# Chapter 6: Transport Layer

Our goals:

- Understand principles behind transport layer services:
  - Multiplexing/demultiplexing
  - Reliable data transfer
  - Flow control
  - Congestion control



- Learn about transport layer protocols in the Internet:
  - UDP: connectionless transport
  - TCP: connection-oriented transport
  - TCP congestion control



# 第6章：传输层

我们的目标：

- 理解传输层服务背后的原理：
  - 复用/解复用
  - 可靠数据传输
  - 流量控制
  - 拥塞控制



- 学习互联网中的传输层协议：
  - UDP：无连接传输
  - TCP：面向连接传输
  - TCP拥塞控制

# Keypoints and Difficulties

Keypoints:

- Port numbers
- TCP

Difficulties:

- TCP connection management
- TCP Reliable data transfer
- TCP congestion control



# 关键点和难点

关键点：

- 端口号
- TCP

难点：

- TCP连接管理
- TCP可靠数据传输
- TCP拥塞控制

# Chapter 6 Outline

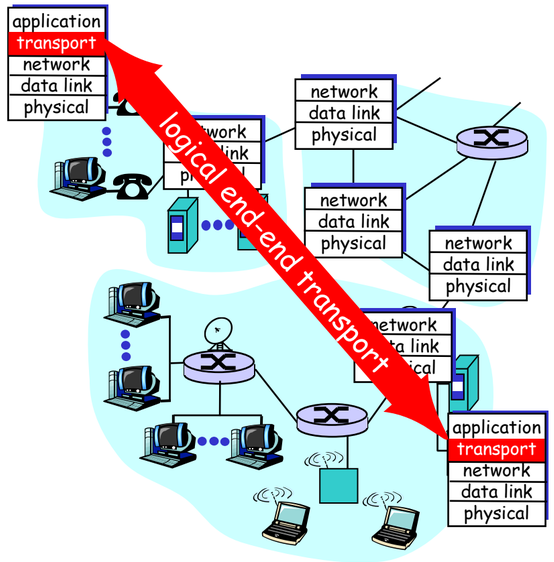
- 6.1 Transport-layer services
- 6.2 Multiplexing and demultiplexing
- 6.3 Connectionless transport: UDP
- 6.4 Connection-oriented transport: TCP
  - Segment structure
  - Connection management
  - Flow control
  - Reliable data transfer
- 6.5 TCP congestion control

# 第6章概要

- 6.1 传输层服务
- 6.2 复用和分用
- 6.3 无连接传输：UDP
- 6.4 面向连接的传输：TCP
  - 分段结构
  - 连接管理
  - 流量控制
  - 可靠数据传输
- 6.5 TCP拥塞控制

# Transport Services and Protocols

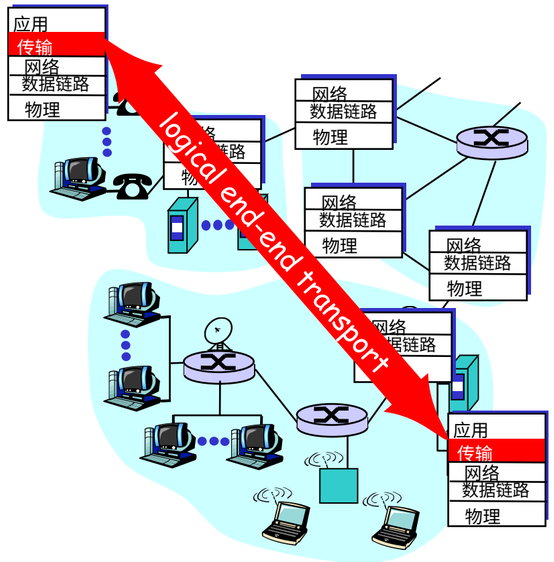
- Provide *logical communication* between app processes running on different hosts
- Transport protocols run in end systems
  - Send side: breaks app messages into segments, passes to network layer
  - Rcv side: reassembles segments into messages, passes to app layer
- More than one transport protocol available to apps
  - Internet: TCP and UDP





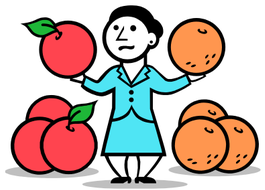

# 传输服务与协议

- 提供逻辑通信，让运行在不同主机上的应用程序进程之间进行
- 传输协议运行在端系统中
  - 发送端：将应用消息分段为分段，传递给网络层 接收端：将分段组装成消息，传递给应用层
- 应用可使用多个传输协议
  - 互联网：TCP 和 UDP



# Transport vs. Network Layer

- *Network layer:*
  - Logical communication between hosts

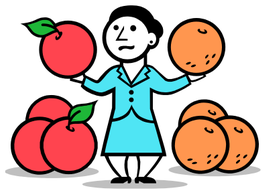


- *Transport layer:*
  - Logical communication between processes
  - Relies on, enhances, network layer services



# 传输层 vs. 网络层

- 网络层：
  - 主机间的逻辑通信主机



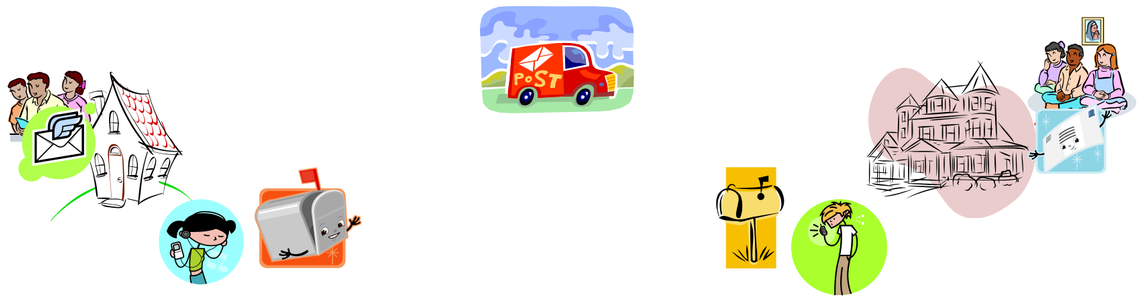
- 传输层：
  - 逻辑通信进程 依赖于，增强网络层服务

# Transport vs. Network Layer

**Household analogy:**

*12 kids sending letters to 12 kids*

- Processes = kids
- App messages = letters in envelopes
- Hosts = houses
- Network-layer protocol = postal service
- Transport protocol = Ann and Bill





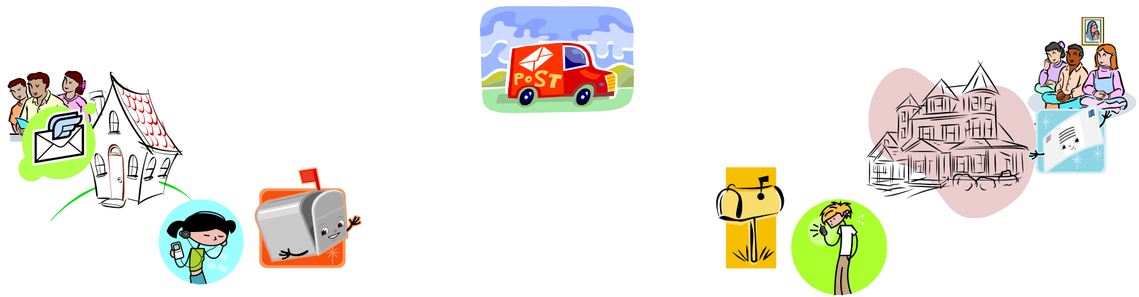

# 传输层与网络层

**家庭类比：**

12个孩子给12个孩子寄信❒ 进程 = 孩子❒

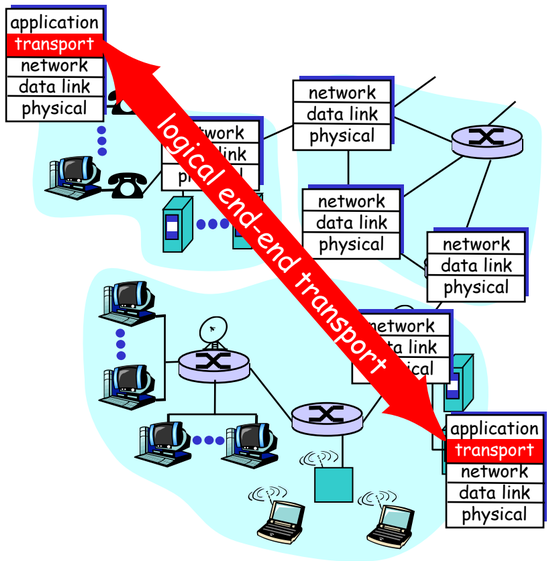
应用消息 = 信封里的信❒ 主机 = 房子❒ 网络

层协议 = 邮政服务❒ 传输协议 = 安和比尔



# Internet Transport-Layer Protocols

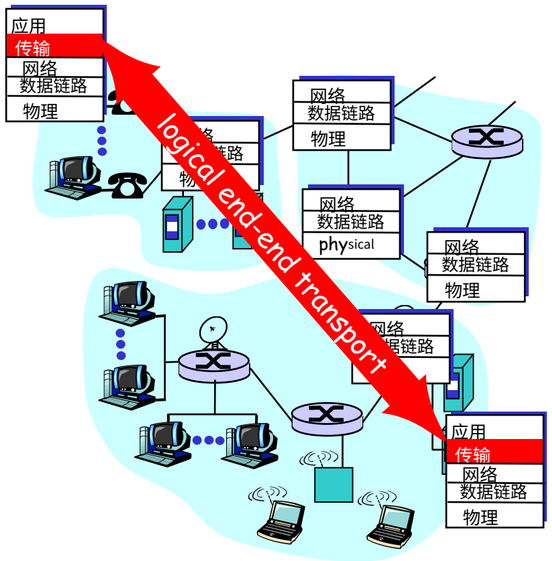
- Reliable, in-order delivery (TCP)
  - Congestion control
  - Flow control
  - Connection setup
- Unreliable, unordered delivery: UDP
  - No-frills extension of "best-effort" IP
- Services not available:
  - Delay guarantees
  - Bandwidth guarantees






# 互联网传输层协议

- 可靠、有序传输（TCP）
  - 拥塞控制 流量控制 连接建立
- 不可靠、无序传输：UDP
  - “尽力而为”IP的简单扩展
- 服务不可用：
  - 延迟保证 带宽保证



# Chapter 6 Outline

- 6.1 Transport-layer services
- 6.2 Multiplexing and demultiplexing
- 6.3 Connectionless transport: UDP
- 6.4 Connection-oriented transport: TCP
  - Segment structure
  - Connection management
  - Flow control
  - Reliable data transfer
- 6.5 TCP congestion control

# 第6章概要

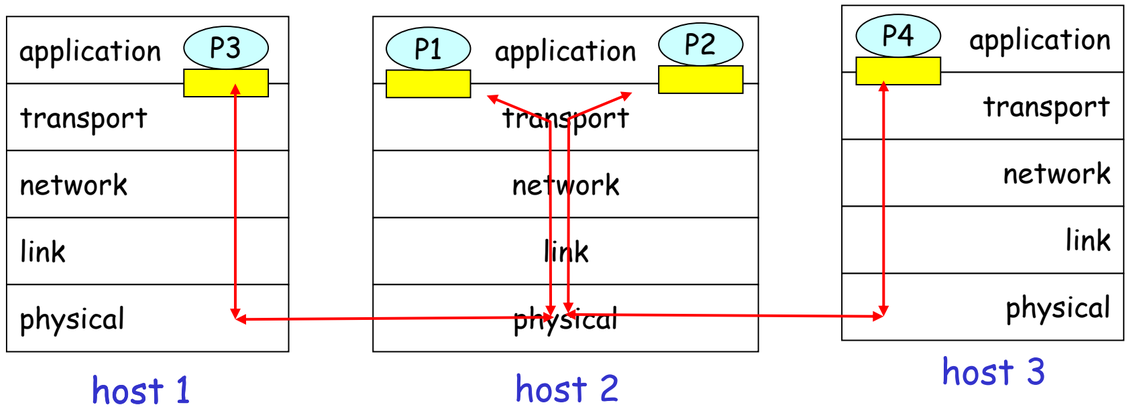
- 6.1 传输层服务
- 6.2 多路复用和分路复用
- 6.3 无连接传输：UDP
- 6.4 面向连接的传输：TCP
  - 分段结构
  - 连接管理
  - 流量控制
  - 可靠数据传输
- 6.5 TCP拥塞控制

# Multiplexing/Demultiplexing

- 1 host ←→ 1 or more processes
- 1 process ←→ 1 or more sockets
- Transport layer interacts with socket

- Demultiplexing at rcv host
- Multiplexing at send host

= socket = process



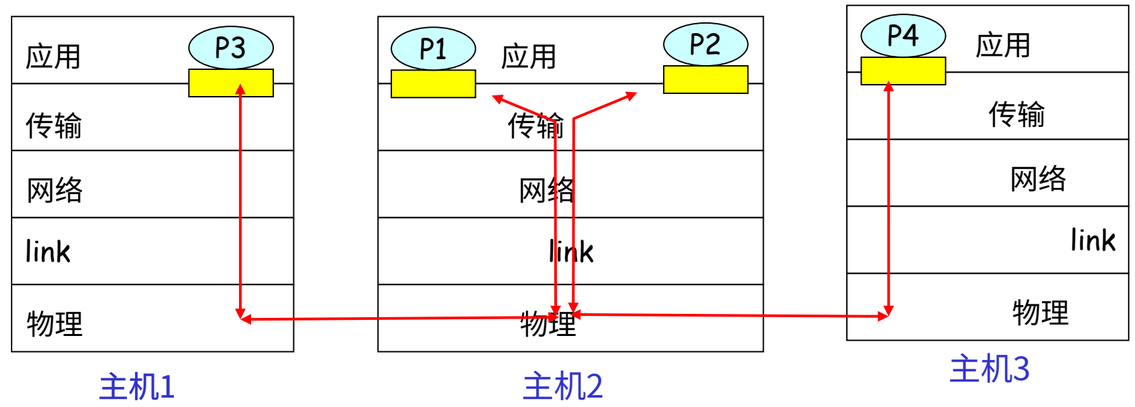



# 复用/解复用

- 1 主机 ←→ 1 或更多进程
- 1 进程 ←→ 1 或更多 套接字
- 传输层与套接字交互

- 在接收主机处解复用
- 发送端的复用

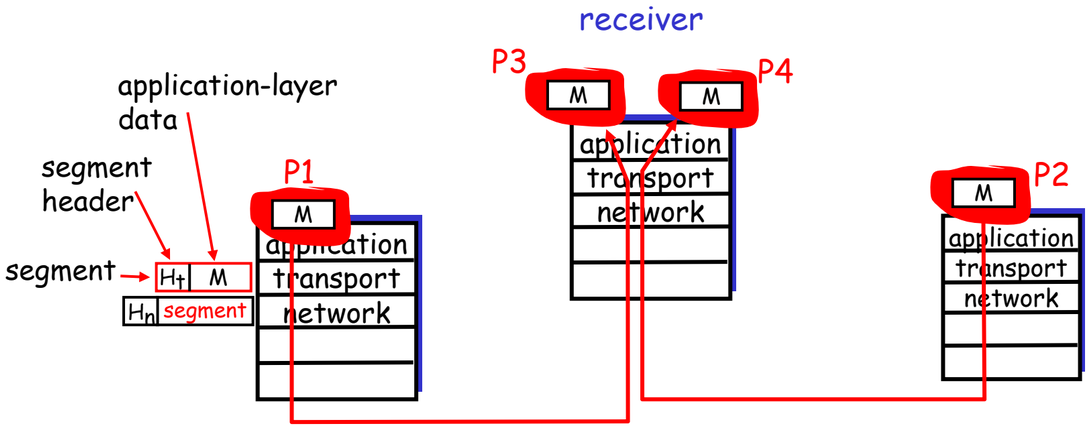
= 套接字 = 处理



# Multiplexing/Demultiplexing

*Segment* - unit of data exchanged between transport layer entities
- aka TPDU: transport protocol data unit

Demultiplexing at rcv host: : delivering received segments to correct app layer processes through socket

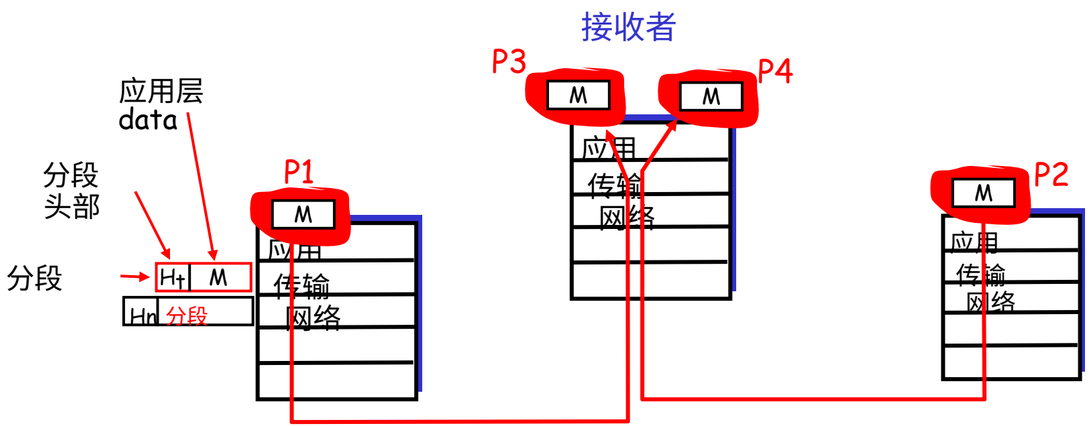





# 复用/解复用

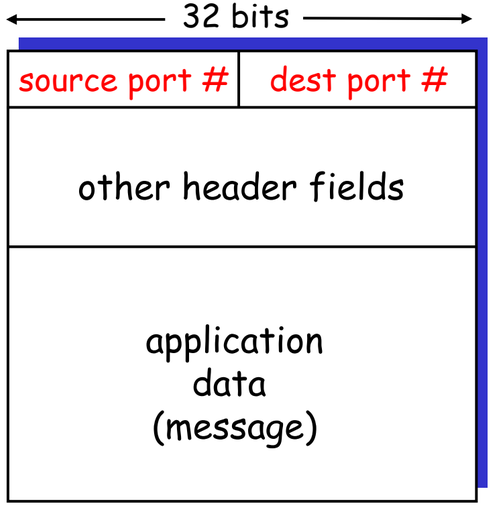
分段 - 传输层实体之间交换的数据单元❍ 又名 TPDU：传输协议数据单元

接收主机处的解复用： 正在传递接收到的 分段 通过套接字正确地传递给应用层进程 套接字



# How Demultiplexing Works

- Host receives IP datagrams
  - Each datagram has source IP address, destination IP address
  - Each datagram carries 1 transport-layer segment
  - Each segment has source, destination port number (recall: well-known port numbers for specific applications)
- Host uses IP addresses & port numbers to direct segment to appropriate socket

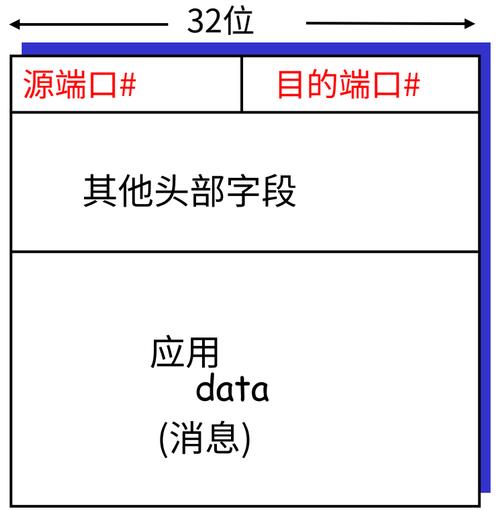



TCP/UDP segment format



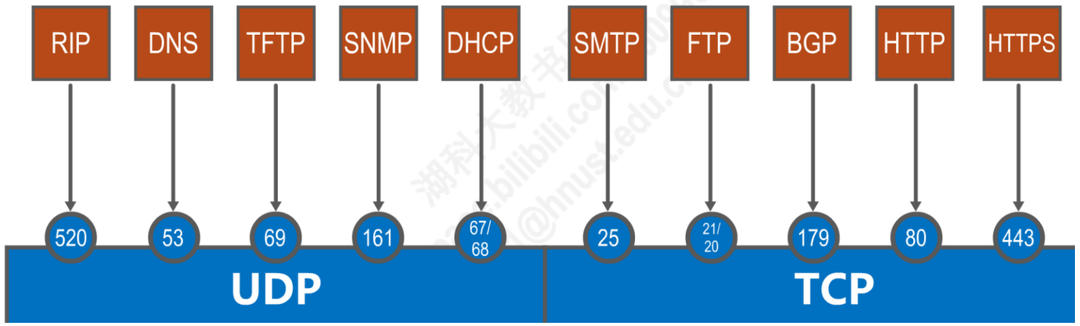
# 解复用如何工作

- 主机接收 IP 数据报
  - 每个<code>数据报</code>都有源IP地址、目的IP地址 每个数据报携带1个传输层的<code>分段</code> 每个<code>分段</code>都有源、目的端口号（回想：特定应用的全名端口）
- 主机使用IP地址和端口号将分段导向正确的套接字



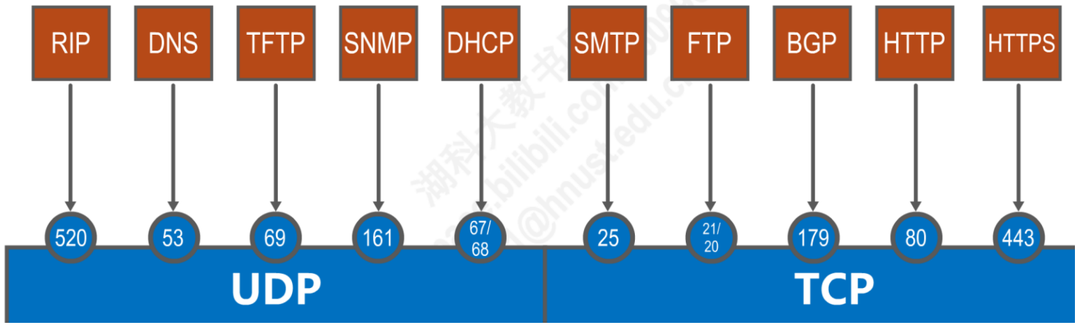

TCP/UDP分段格式

# Well-known port number




The port number only has local meaning, and the same port number in different computers is not related.

# 好-已知的端口号




端口号仅具有本地意义，不同计算机中的相同端口号并不相关。

# Multiplexing/Demultiplexing

Multiplexing at send host:

gathering data from multiple sockets, enveloping data with header (later used for demultiplexing)

- Multiplexing/demultiplexing
  - Based on sender, receiver port numbers, IP addresses
  - <source IP #, dest. IP#, source port #, dest. port#>



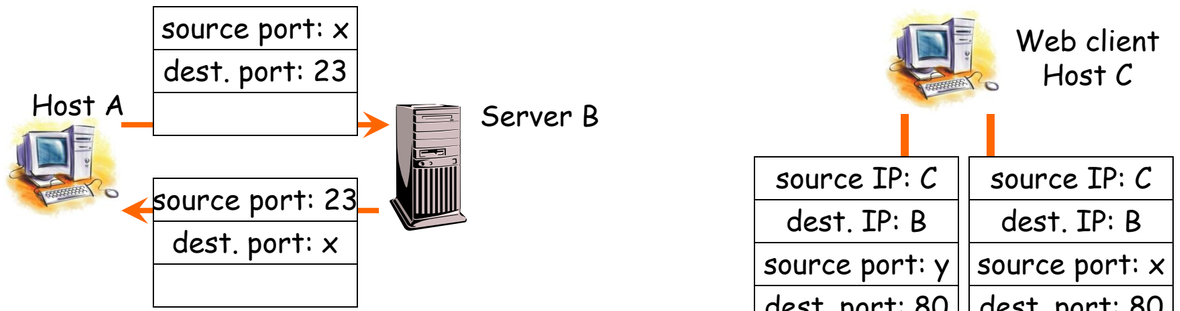
# 复用/解复用

发送方复用：

从多个套接字收集数据，用头部封装数据（稍后用于解复用）

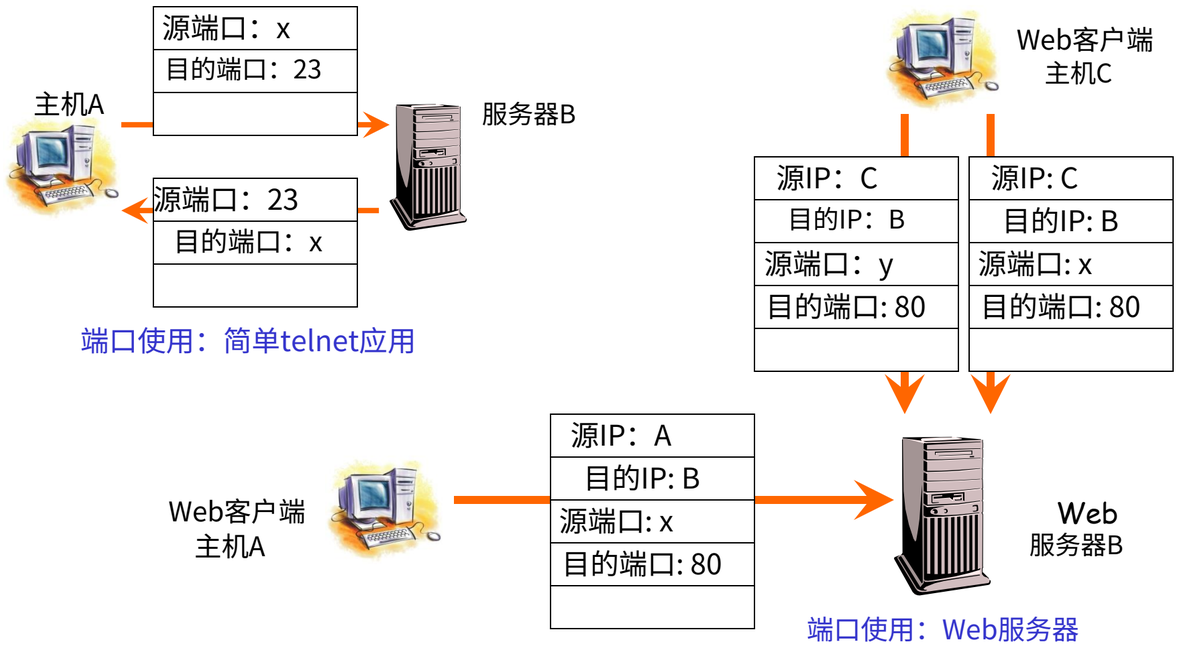
- 复用/解复用
  - 基于发送方、接收者端口号、IP地址
  - <源IP #, 目的IP#, 源端口 #, 目的端口#>

# Multiplexing/Demultiplexing: examples






# 复用/解复用：示例



# Connectionless Demultiplexing

- Create sockets with port numbers:

```
DatagramSocket mySocket1 =
   new DatagramSocket(12534);
DatagramSocket mySocket2 =
   new DatagramSocket(12535);
```

- UDP socket identified by two-tuple:

(dest IP address, dest port number)

- When host receives UDP segment:
  - Checks destination port number in segment
  - Directs UDP segment to socket with that port number
- IP datagrams with different source IP addresses and/or source port numbers directed to same socket



# 无连接解复用

- 使用端口号创建套接字：

数据报套接字 mySocket1 =new 数据报套接字(12534); 数据报套接字 mySocket2 = new 数据报套接字(12535);

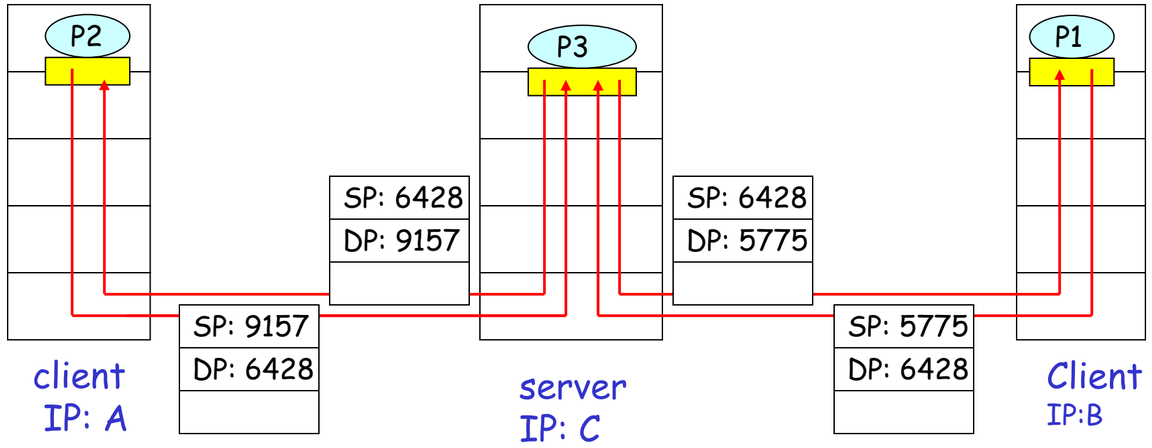
- 由两个元组标识的UDP套接字：

(dest IP地址，目的端口编号)

- 当主机接收UDP分段时：
  - 检查分段中的目的端口编号
  - 将UDP分段导向该端口编号的套接字
- 具有不同源IP地址和/或源端口号的数据报发送到同一个套接字

# Connectionless Demux (cont)

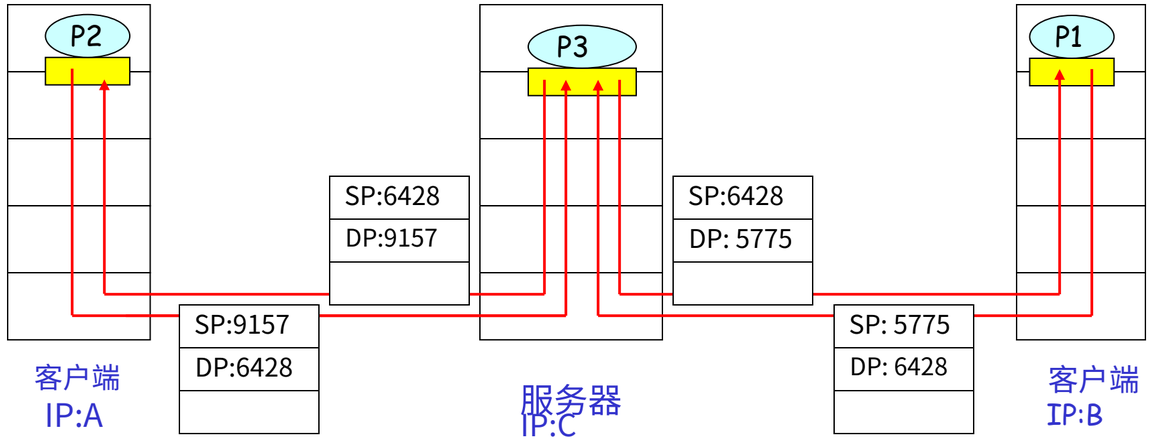
```
DatagramSocket serverSocket = new DatagramSocket(6428);
```




SP provides "return address"

# 无连接多路分解 (续)

数据报套接字 serverSocket = new 数据报套接字(6428);




SP 提供 "返回地址"

# Connection-Oriented Demux

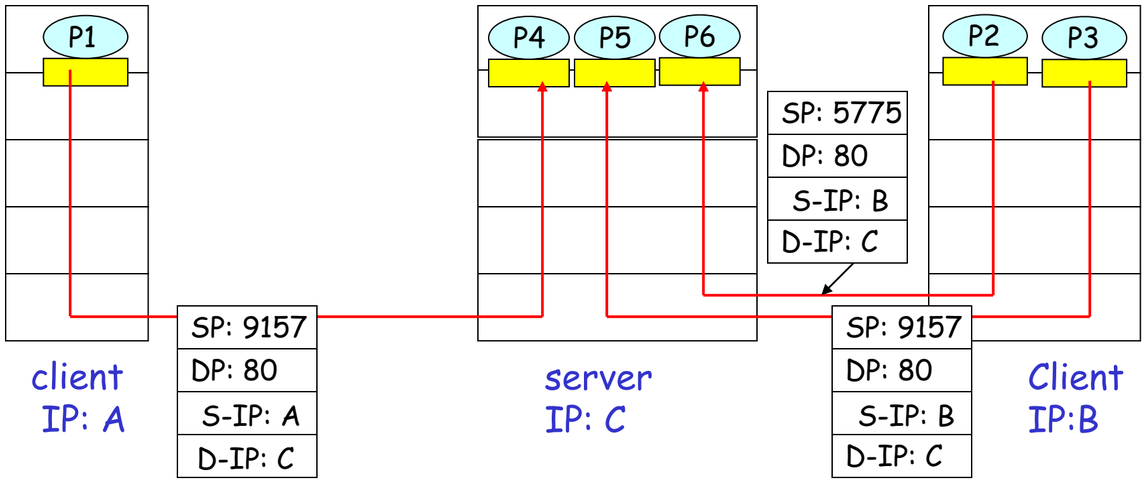
- TCP socket identified by 4-tuple:
  - Source IP address
  - Source port number
  - Dest IP address
  - Dest port number
- Recv host uses all four values to direct segment to appropriate socket
- Server host may support many simultaneous TCP sockets:
  - Each socket identified by its own 4-tuple

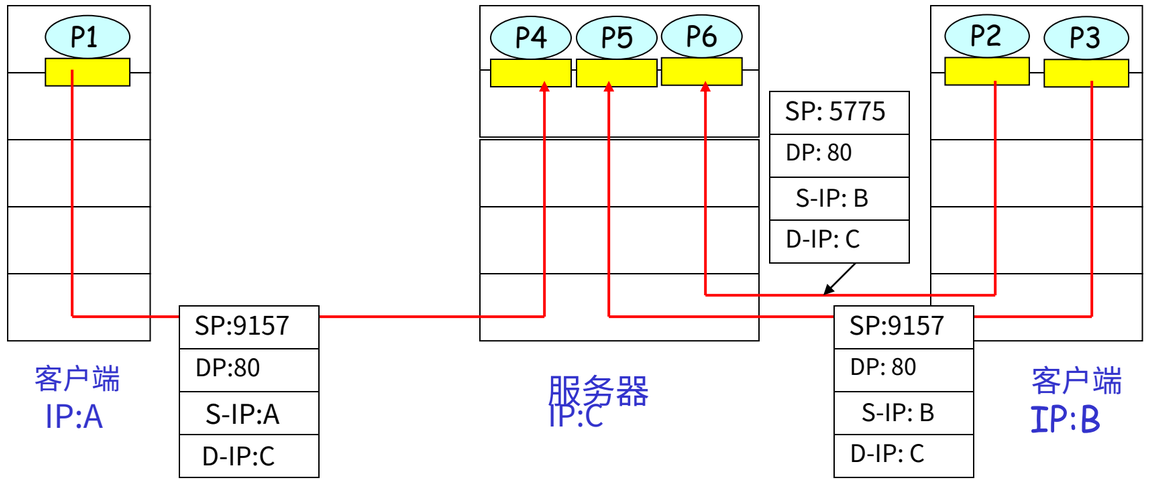
# 面向连接的解复用

- TCP套接字由4元组标识：
  - 源IP地址 源端口号
  - 目的IP地址 目的端口号
- 接收主机使用所有四个值来将分段导向适当的套接字
- 服务器主机可能支持许多同时的TCP套接字：
  - 每个套接字由其自身的4元组标识

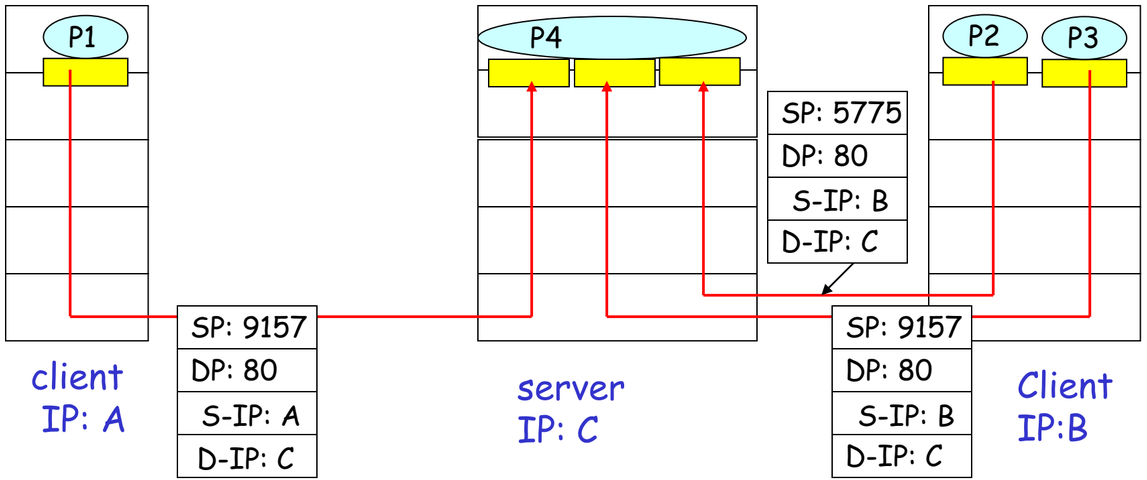
# Connection-Oriented Demux (cont.)

# 面向连接的解复用（续）

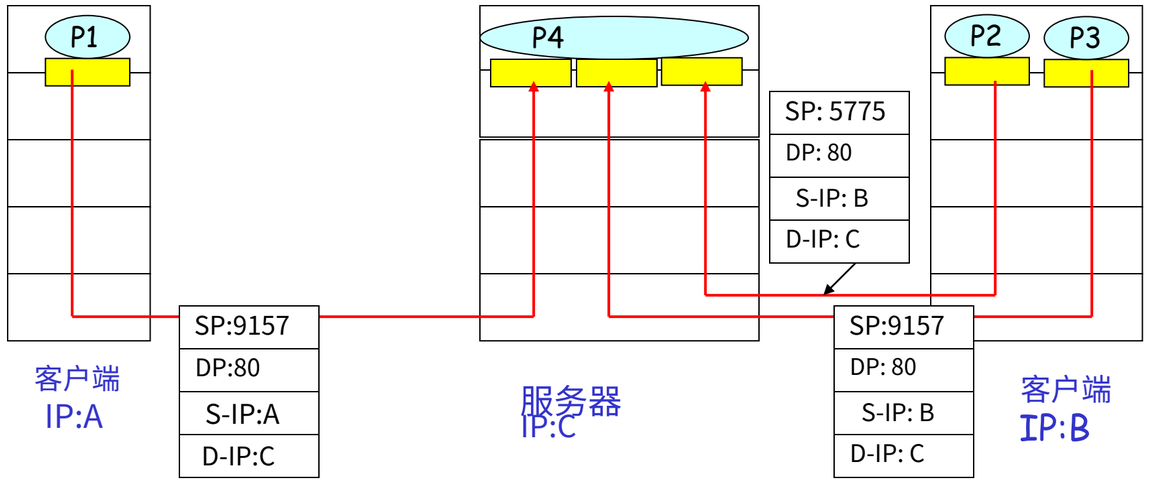


# Connection-Oriented Demux: Threaded Web Server




# 面向连接的解复用：多线程Web服务器



# Chapter 6 Outline

- 6.1 Transport-layer services
- 6.2 Multiplexing and demultiplexing
- 6.3 Connectionless transport: UDP
- 6.4 Connection-oriented transport: TCP
  - Segment structure
  - Connection management
  - Flow control
  - Reliable data transfer
- 6.5 TCP congestion control

# 第6章概要

- 6.1 传输层服务
- 6.2 多路复用和分路复用
- 6.3 无连接传输：UDP
- 6.4 面向连接的传输：TCP
  - 分段结构
  - 连接管理
  - 流量控制
  - 可靠数据传输
- 6.5 TCP拥塞控制

# UDP: User Datagram Protocol [RFC 768]

- "No frills," "bare bones" Internet transport protocol
- "Best effort" service, UDP segments may be:
  - Lost
  - Delivered out of order to app
- *Connectionless:*
  - No handshaking between UDP sender, receiver
  - Each UDP segment handled independently of others

**Why is there a UDP?**

- No connection establishment (which can add delay)
- Simple: no connection state at sender, receiver
- Small segment header
- No congestion control: UDP can blast away as fast as desired



# UDP: 用户数据报协议 [RFC 768]

- "不花哨的"，"精简的"互联网传输协议
- "尽力而为"服务，UDP分段可能：
  - 丢失
  - 乱序交付给应用

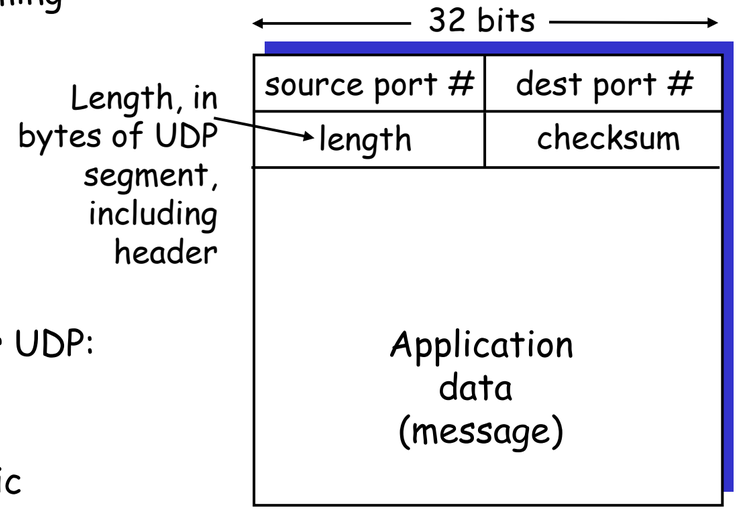
- 无连接：
  - UDP发送方与接收者之间无需握手
  - 每个UDP分段独立于其他分段处理

**为什么要有UDP?**

- 没有连接建立（这可能会增加延迟）
- 简单：发送方和接收方都没有连接状态
- 小的分段头部
- 没有拥塞控制：UDP可以尽可能快地发送

# UDP: more

- Often used for streaming multimedia apps
  - Loss tolerant
  - Rate sensitive
- Other UDP uses
  - DNS
  - SNMP
- Reliable transfer over UDP: add reliability at application layer
  - Application-specific error recovery!

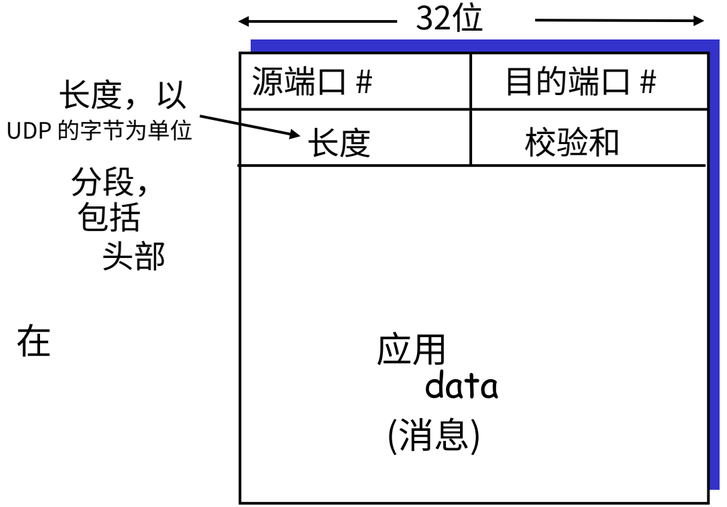



UDP segment format



# UDP：更多

- 常用于流媒体应用
  - 容错
  - 速率敏感
- 其他UDP用途
  - DNS
  - SNMP
- 通过UDP实现可靠传输：在应用层增加可靠性
  - 特定于应用错误恢复！




UDP分段格式

# Chapter 6 Outline

- 6.1 Transport-layer services
- 6.2 Multiplexing and demultiplexing
- 6.3 Connectionless transport: UDP
- 6.4 Connection-oriented transport: TCP
  - Segment structure
  - Connection management
  - Flow control
  - Reliable data transfer
- 6.5 TCP congestion control



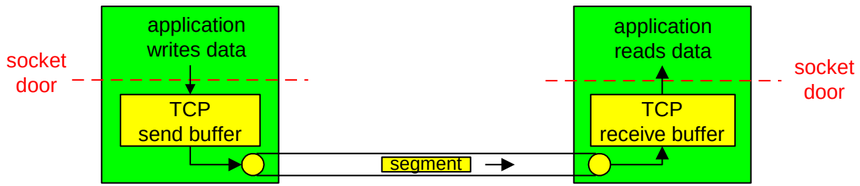
# 第6章概要

- 6.1 传输层服务
- 6.2 复用和分用
- 6.3 无连接传输：UDP
- 6.4 面向连接的传输：TCP
  - 分段结构 连接管理 流量控制 可靠数据传输
- 6.5 TCP拥塞控制

# TCP: Overview

RFCs: 793, 1122, 1323, 2018, 2581

- End-to-End:
  - One sender, one receiver
- In-order *byte steam*:
  - No "message boundaries"
- Reliable :
  - TCP congestion and flow control set window size
- Connection-oriented:
  - Handshaking (exchange of control msgs) init's sender, receiver state before data exchange
- Full duplex data:
  - Bi-directional data flow in same connection
  - MSS: maximum segment size

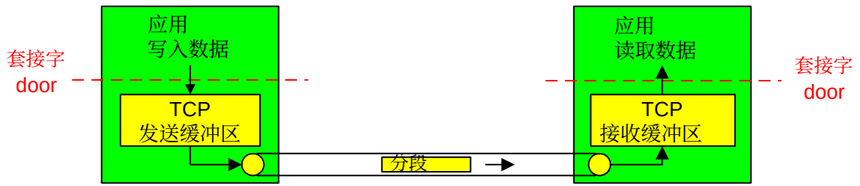





# TCP：概述

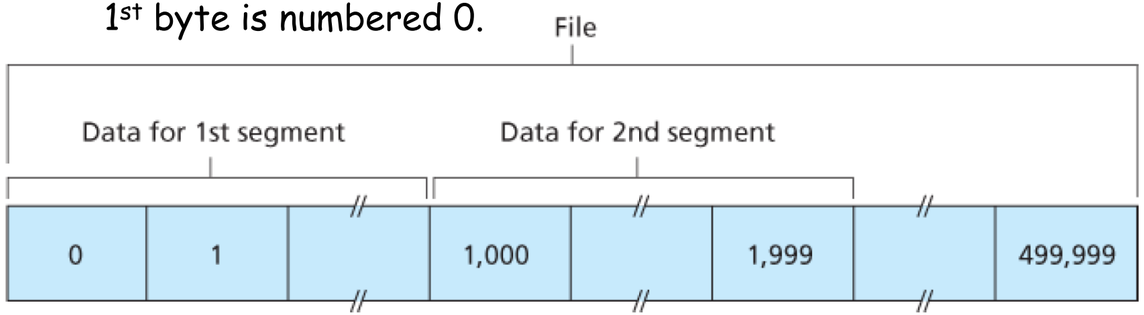
RFCs：793、1122、1323、2018、2581

- 端到端：
  - 一个发送方，一个接收者
- 有序字节流：
  - 没有"消息边界"
- 可靠：
  - TCP拥塞和流控制窗口大小
- 面向连接：
  - 握手（控制消息交换）初始化发送方、接收者状态，数据交换前
- 全双工数据：
  - 双向数据流在同一连接
  - MSS：最大分段大小



# TCP: byte stream

- Each byte is numbered in segment's data
  - E.g., Suppose that a process in Host A wants to send a stream of data (a file of 500,000 bytes) to a process in Host B over a TCP connection. MSS is 1,000 bytes, and the 1st byte is numbered 0.

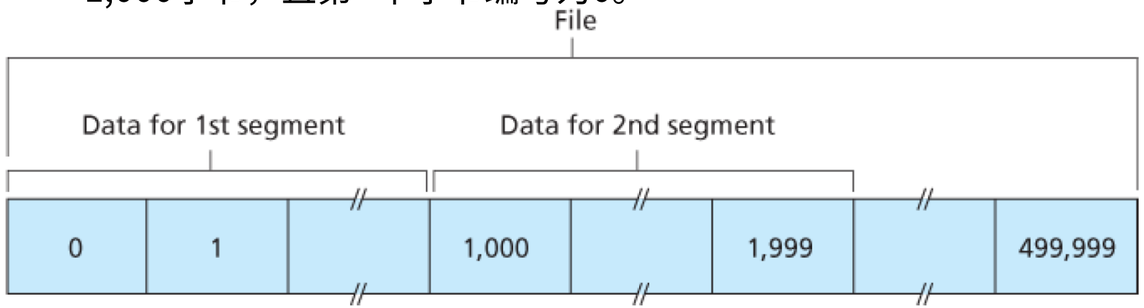



  - Seq # of the 1st segment: 0; Seq # of the 2nd segment: 1000;
  - Seq # of the 3rd segment: ?



# TCP：字节流

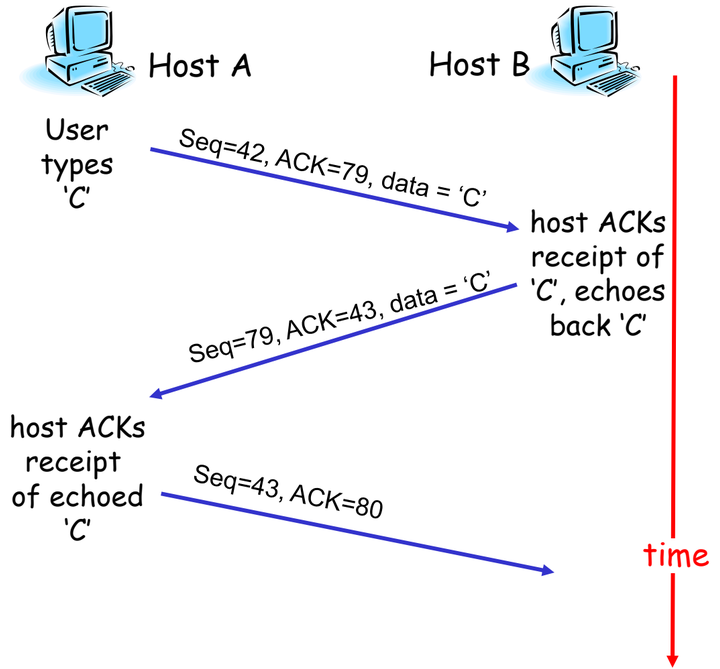
- 每个字节都在分段的数据中编号
  - 例如，假设主机A中的某个进程想要通过TCP连接将一个数据流（500,000字节大小的文件）发送到主机B中的某个进程。MSS为1,000字节，且第一个字节编号为0。




  - 第一个分段的序列号：0；第二个分段的序列号：1,000；
  - 第3分段序列号：？

# TCP Seq. #'s and ACKs

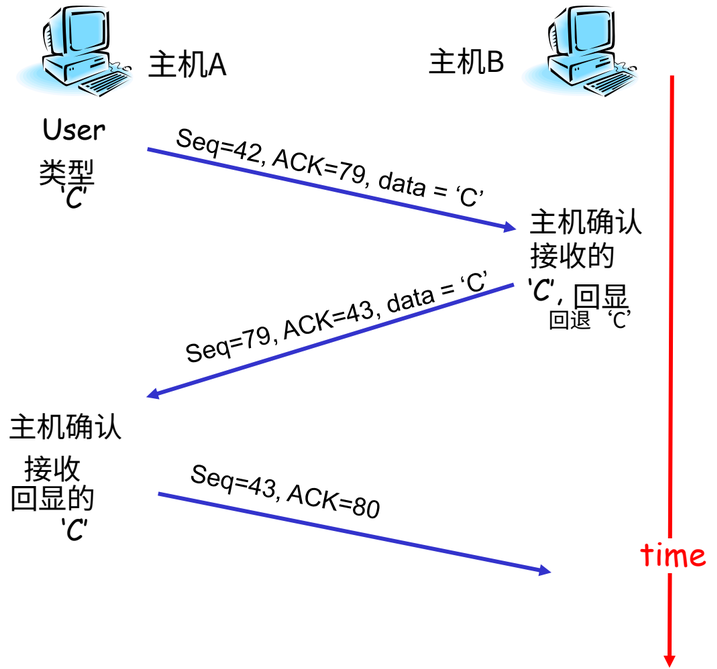
Simple telnet scenario



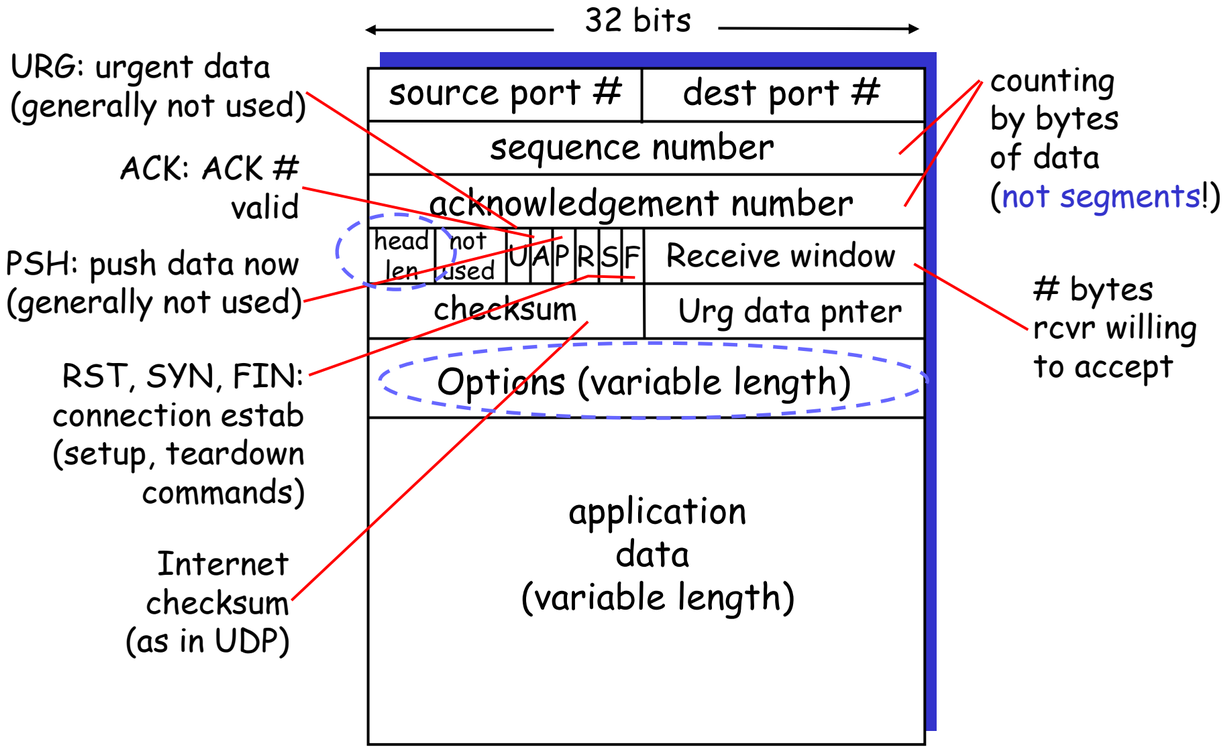



# TCP序列号和确认

简单的telnet场景

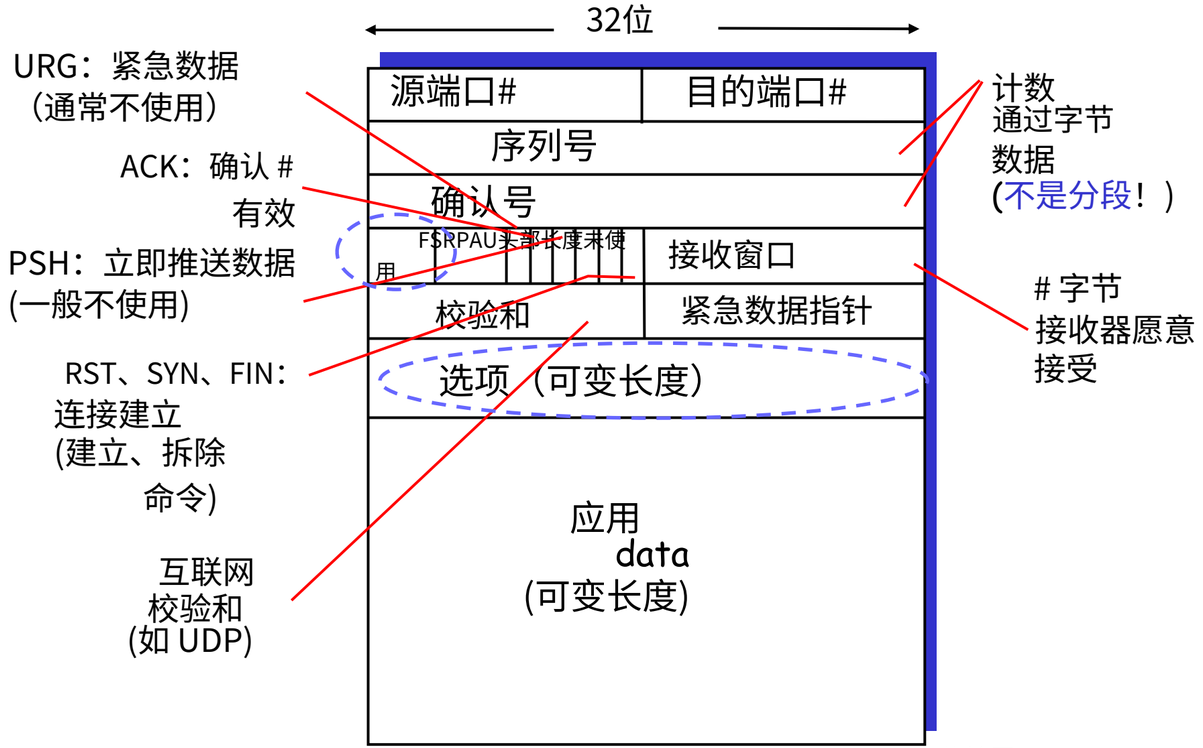


# TCP Segment Structure






# TCP分段结构



# Chapter 6 Outline

- 6.1 Transport-layer services
- 6.2 Multiplexing and demultiplexing
- 6.3 Connectionless transport: UDP
- 6.4 Connection-oriented transport: TCP
  - Segment structure
  - Connection management
  - Flow control
  - Reliable data transfer
- 6.5 TCP congestion control

# 第6章概要

- 6.1 传输层服务
- 6.2 多路复用和分路复用
- 6.3 无连接传输：UDP
- 6.4 面向连接的传输：TCP
  - 分段结构
  - 连接管理
  - 流量控制
  - 可靠数据传输
- 6.5 TCP拥塞控制

# TCP Connection Management

Recall: TCP sender, receiver establish "connection" before exchanging data segments

- Initialize TCP variables:
  - Seq. #s
  - Buffers, flow control info (e.g. `RcvWindow`)
- *Client*: connection initiator

```
Socket clientSocket
  = new Socket("hostname", "port number");
```

- *Server*: contacted by client

```
Socket connectionSocket
  = welcomeSocket.accept();
```





# TCP连接管理

回顾：TCP发送方、接收者建立“连接”后再交换数据分段

- 初始化TCP变量：
  - 序列号{s}缓冲区，流量控制信息（例如<code>RcvWindow</code>）
- 客户端：连接发起者Socket clientSocket= new Socket("hostname", "port number");

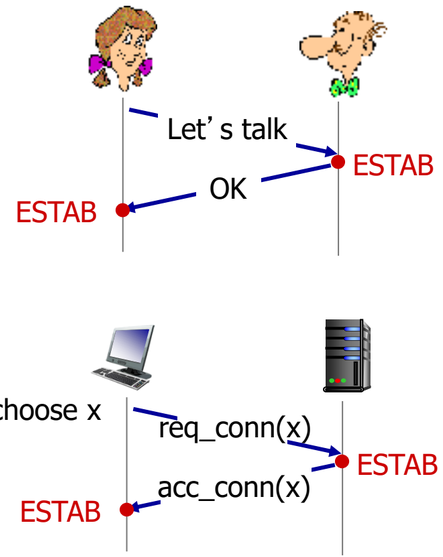
- 服务器：被客户端连接

套接字连接Socket= 欢迎套接字.accept();



# Agreeing to establish a connection

2-way handshake:

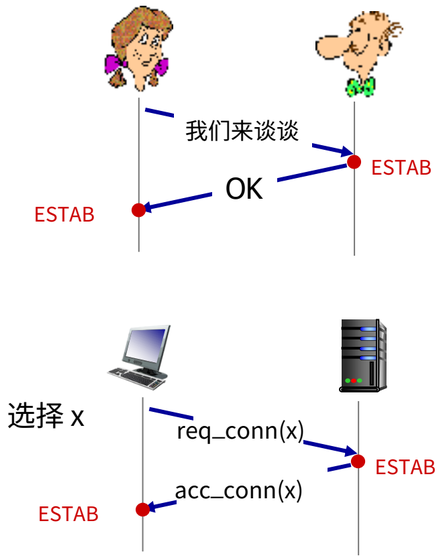



Q: will 2-way handshake always work in network?

- variable delays
- retransmitted messages (e.g. req_conn(x)) due to message loss
- message reordering
- can't "see" other side

# 同意建立连接

双向握手：

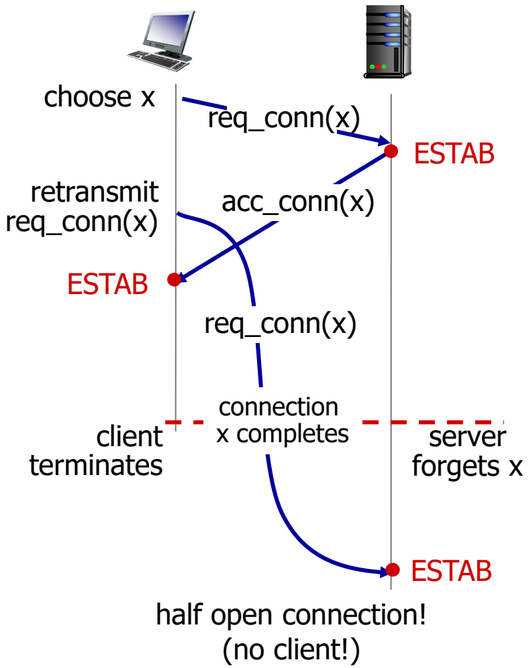


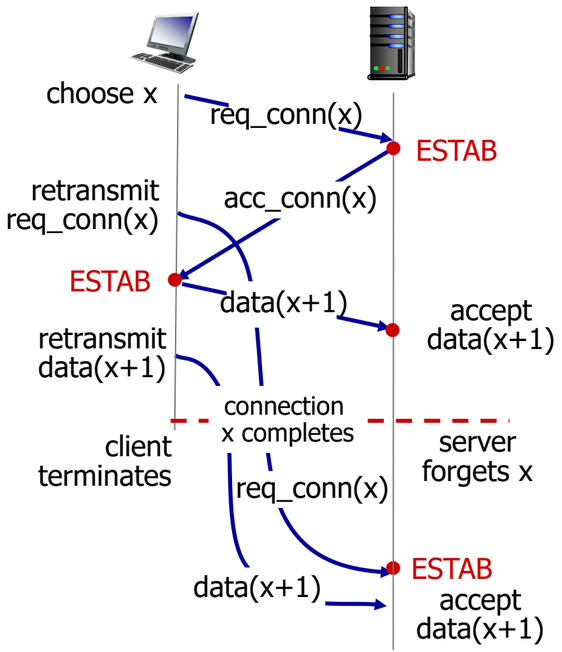

Q: 双向握手在网络中是否总是有效?

- 变量延迟
- 重传消息（例如req_conn(x)）由于消息丢失
- 消息重排
- 无法"看到"另一端

# Agreeing to establish a connection

2-way handshake failure scenarios:

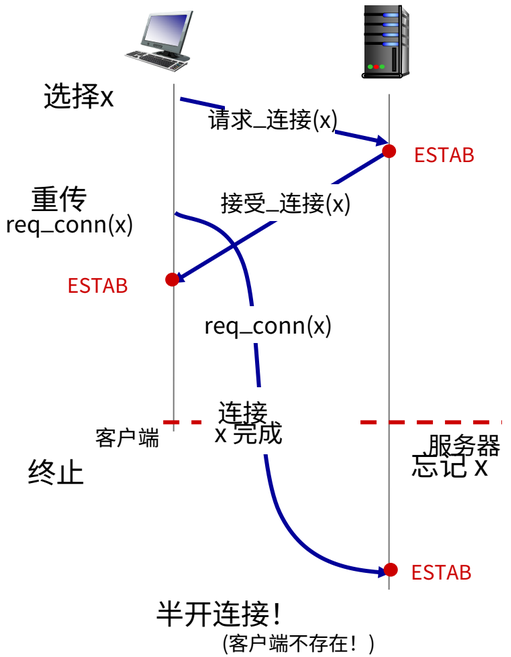


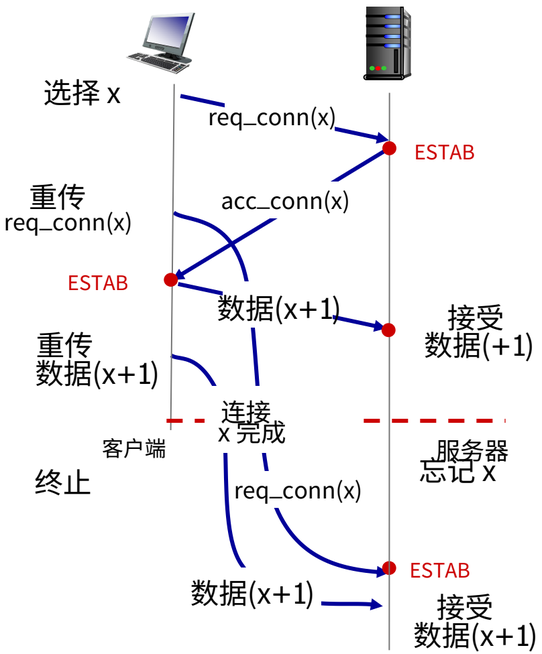






# 同意建立连接

双向握手失败场景：



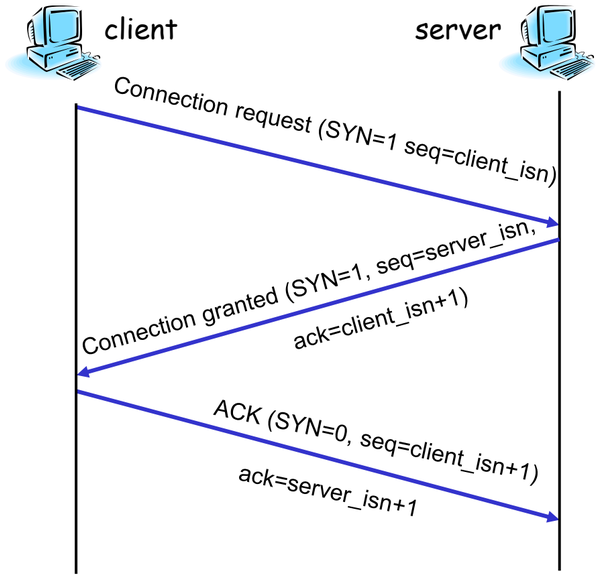
# TCP Connection Management (Cont.)

## Three way handshake:

Step 1: client host sends TCP SYN segment to server
- Specifies client initial seq #
- No application data

Step 2: server host receives SYN, replies with SYNACK segment
- Server allocates buffers
- Specifies server initial seq. #

Step 3: client receives SYNACK, replies with ACK segment, which may contain data





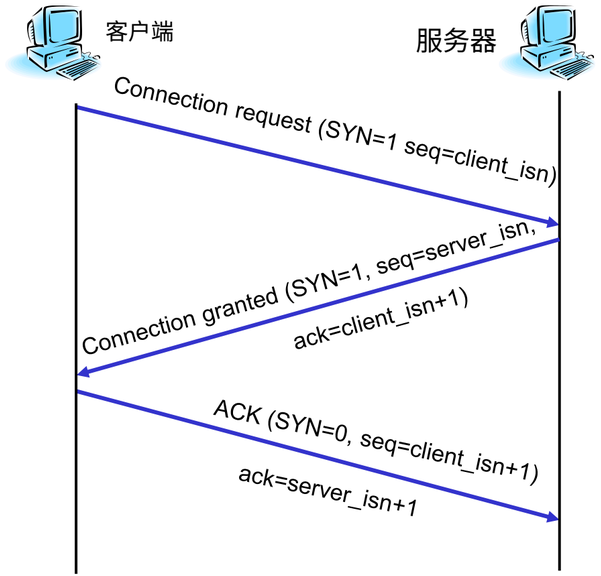

# TCP连接管理（续）

## 三路握手：

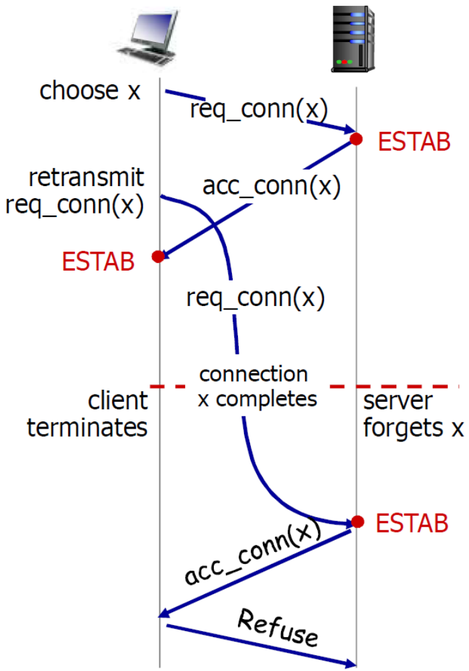
步骤1：客户端主机向服务器发送TCP SYN分段
- 指定客户端初始序列号
- 无应用数据

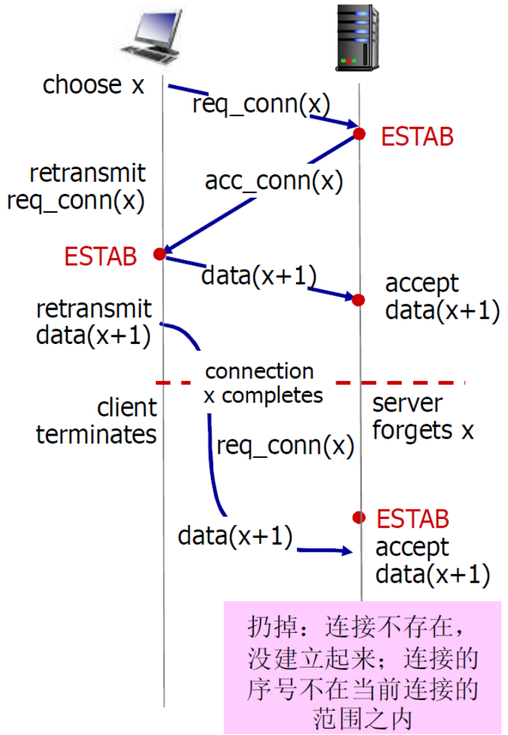
步骤2：服务器主机接收SYN，回复SYNACK分段
- 服务器分配缓冲区
- 指定服务器初始序列号

步骤 3： 客户端接收 SYNACK，回复 ACK 分段，该分段可能包含数据



# TCP Connection Management (Cont.)

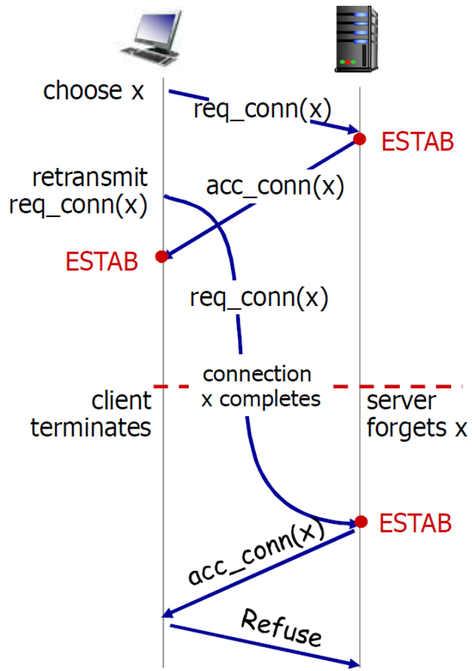


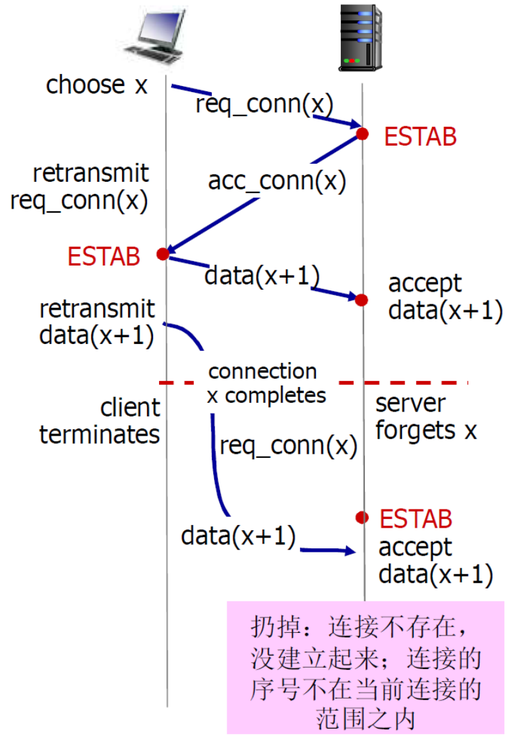




扔掉：连接不存在，没建立起来；连接的序号不在当前连接的范围之内



# TCP连接管理（续）




扔掉：连接不存在，没建立起来；连接的序号不在当前连接的范围之内

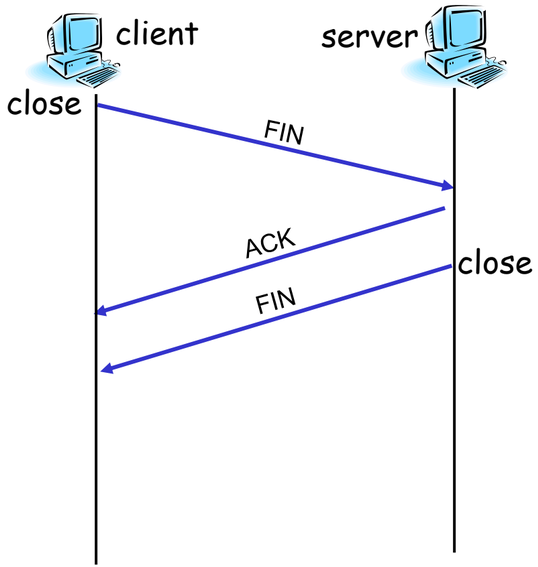
# TCP Connection Management (cont.)

**Closing a connection:**

client closes socket:
`clientSocket.close();`

**Step 1:** client end system sends TCP FIN control segment to server

**Step 2:** server receives FIN, replies with ACK. Closes connection, sends FIN





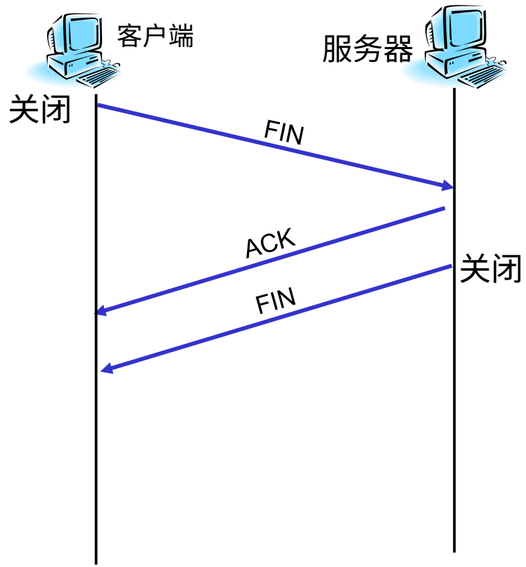

# TCP连接管理（续）

**关闭连接：**

客户端关闭套接字：
**clientSocket.close();**

**步骤1：** 客户端端系统向服务器发送TCP FIN控制段

**步骤 2：** 服务器收到 FIN，回复确认。关闭连接，发送 FIN



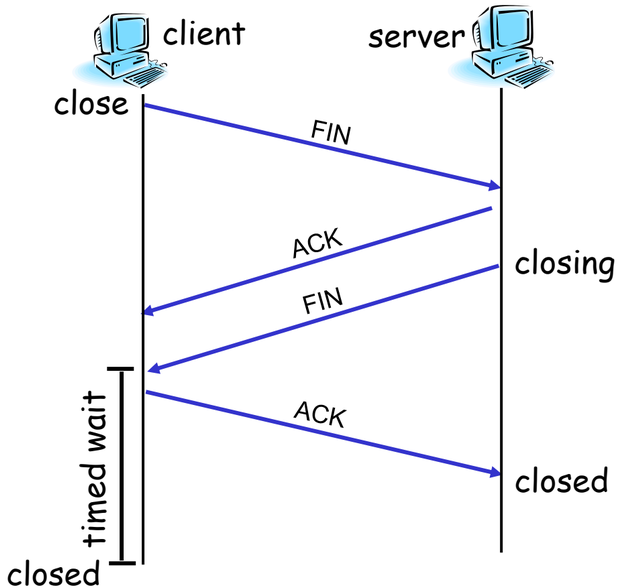
# TCP Connection Management (cont.)

**Step 3:** client receives FIN, replies with ACK.

- Enters "timed wait" - will respond with ACK to received FINs

**Step 4:** server, receives ACK. Connection closed.

**Note:** with small modification, can handle simultaneous FINs





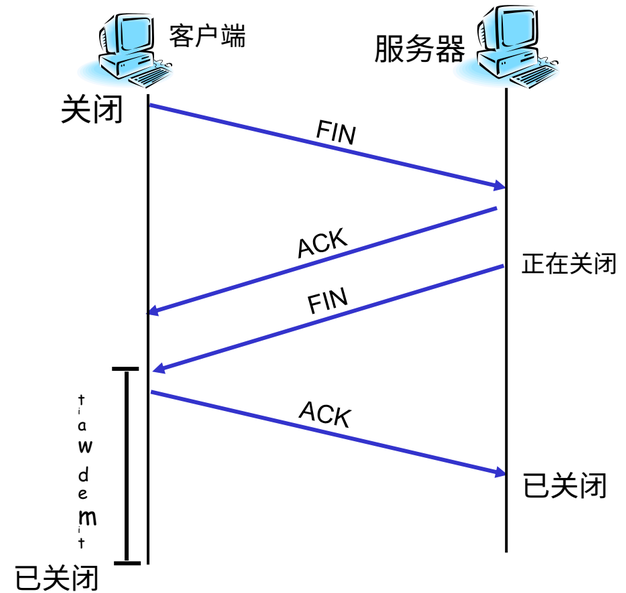

# TCP连接管理（续）

**步骤 3：** 客户端收到 FIN，回复确认。

- 进入“计时等待”状态 - 将对收到的 FIN 发送确认

**步骤4：** 服务器，收到确认。连接关闭。

**注意：** 稍作修改后，即可处理同时到达的FIN



# Exercises-1

1. Host A sends a TCP segment (SYN=1,seq=11220) to host B and expects to establish a TCP connection with host B. if host B accepts the connection request, the correct TCP segment sent by host B to host A may be ( )

(1)SYN=0,ACK=0,seq=11221,ack=11221

(2)SYN=1,ACK=1,seq=11220,ack=11220

(3)SYN=1,ACK=1,seq=11221,ack=11221

(4)SYN=0,ACK=0,seq=11220,ack=11220



# 练习-1

1. 主机A向主机B发送一个TCP数据段（SYN=1，seq=11220），期望与主机B建立TCP连接。如果主机B接受连接请求，主机B向主机A发送的正确TCP数据段可能是（）

(1)SYN=0,确认=0,序列号=11221,确认=11221

(2)SYN=1,确认=1,序列号=11220,确认=11220

(3)SYN=1,确认=1,序列号=11221,确认=11221

(4)SYN=0,确认=0,序列号=11220,确认=11220

# Chapter 6 Outline

- 6.1 Transport-layer services
- 6.2 Multiplexing and demultiplexing
- 6.3 Connectionless transport: UDP
- 6.4 Connection-oriented transport: TCP
  - Segment structure
  - Connection management
  - Flow control
  - Reliable data transfer
- 6.5 TCP congestion control

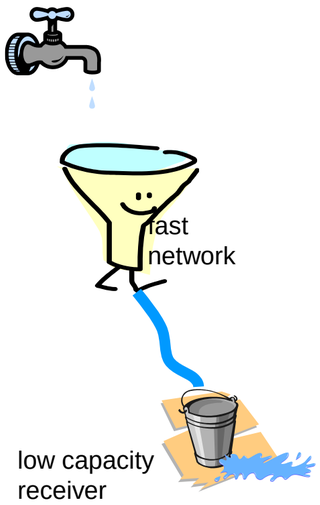


# 第6章概要

- 6.1 传输层服务
- 6.2 复用和分用
- 6.3 无连接传输：UDP
- 6.4 面向连接的传输：TCP
  - 分段结构 连接管理
  - 流量控制 可靠数据传输
- 6。5 TCP拥塞控制

# TCP Flow Control

- Sender should not overwhelm receiver's capacity to receive data
- If necessary, sender should slow down transmission rate to accommodate receiver's rate
- Different from Congestion Control whose purpose was to handle congestion in network
- But both congestion control and flow control work by slowing down data transmission




flow control problem



# TCP流控制

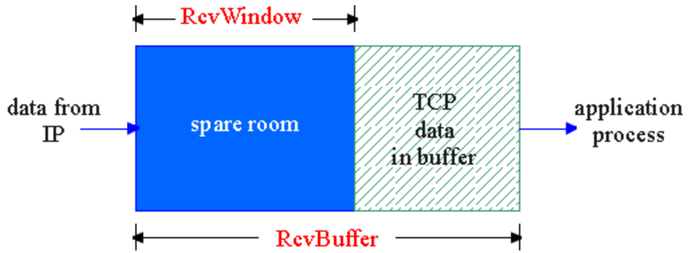
- 发送方不应压倒接收者的接收数据能力
- 如果需要，发送方应降低传输速率以适应接收者的速率
- 与拥塞控制不同，其目的是处理网络中的拥塞
- 但拥塞控制和流量控制都是通过减缓数据传输来实现的




流量控制问题

# TCP Flow Control

- Receive side of TCP connection has a receive buffer:




- App process may be slow at reading from buffer

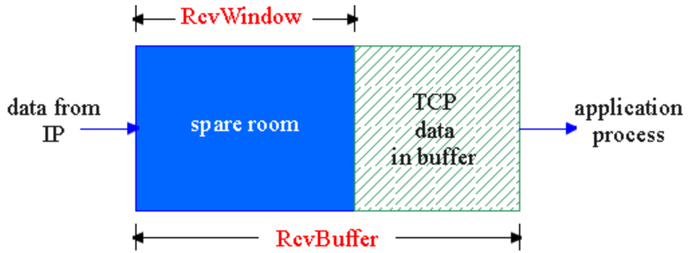
**flow control**

sender won't overflow receiver's buffer by transmitting too much, too fast

- Speed-matching service: matching the send rate to the receiving app's drain rate

# TCP流控制

- TCP连接的接收端有一个接收缓冲区：



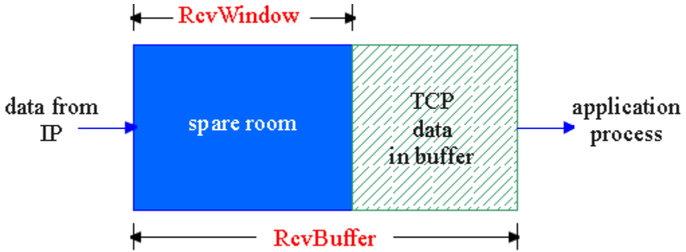

- 应用进程在从缓冲区读取时可能很慢

**流量控制**

发送方不会因传输过多、过快而使接收方的缓冲区溢出

- 速率匹配服务：将发送速率匹配到接收应用的消耗速率

# TCP Flow Control: How it Works




(Suppose TCP receiver discards out-of-order segments)

- Spare room in buffer

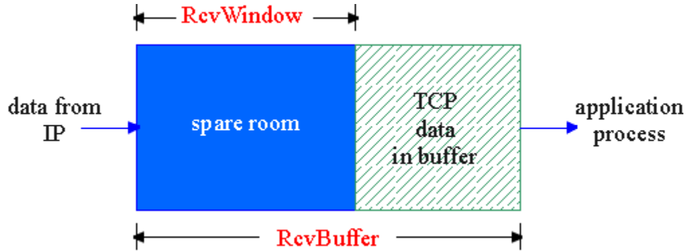
= **RcvWindow**

= **RcvBuffer-[LastByteRcvd - LastByteRead]**

- Rcvr advertises spare room by including value of **RcvWindow** in segments
- Sender limits unACKed data to **RcvWindow**
  - Guarantees receive buffer doesn't overflow



# TCP流控制：它是如何工作的




(假设TCP接收器丢弃乱序分段)

- 缓冲区空闲空间=

**RcvWindow= RcvBuffer-[ LastByteRcvd - LastByteRead]**

- 接收器通过在分段中包含<样式 id='2'> 接收窗口 </ 样式 > 的值来宣布空闲空间- 发送方将未确认数据限制在<样式 id='5'>接收窗口</样式>- 保证接收缓冲区不会溢出

# Technical Issue

- Suppose RcvWindow=0 and that receiver has already ACK'd ALL packets in buffer
- Sender does not transmit new packets until it hears RcvWindow>0
- Receiver never sends RcvWindow>0 since it has no new ACKS to send to Sender
- **DEADLOCK!**



- Solution:
  - TCP specs require sender to continue sending packets with one data byte while RcvWindow=0, just to keep receiving ACKS from receiver
  - At some point the receiver's buffer will empty and RcvWindow>0 will be transmitted back to sender



# 技术问题

- <样式 id='1'>假设</样式><样式 id='3'>接收窗口</样式>=0 <样式 id='6'>并且接收器已经确认了缓冲区中的所有数据包</样式>
- 发送方在收到接收窗口>0之前不会发送新的数据包

- 接收者永远不会发送 接收窗口>0，因为它没有新的确认要发送给发送方
- **死锁！**



- 解决方案：
  - TCP规范要求发送方继续发送包含一个数据字节的包，同时接收窗口=0，只是为了持续接收来自接收者的确认
    - 某个时刻，接收方的缓冲区会空，接收窗口>0将被传回给发送方

# Exercises-2

1. A TCP connection is established between host A and host B, and the MSS length of TCP is 1000 bytes. If the current congestion window of host A is 4000 bytes, after host A sends two MSS segments to host B continuously, host A successfully receives the confirmation segment of the first segment sent by host B, and the size of the receiving window in the confirmation segment is 2000 bytes, then the maximum number of bytes that host A can also send to host B is ( )

(1)1000
(2)2000
(3)3000
(4)4000



# 练习-2

1. 主机A和主机B之间建立了一个TCP连接，TCP的MSS长度为1000字节。如果主机A当前的拥塞窗口为4000字节，主机A连续向主机B发送两个MSS分段后，主机A成功收到了主机B发送的第一个分段的确认分段，确认分段中的接收窗口大小为2000字节，那么主机A最多还可以向主机B发送多少字节（）

(1)1000(2)2000(3)3000(4)4000

# Chapter 6 Outline

- 6.1 Transport-layer services
- 6.2 Multiplexing and demultiplexing
- 6.3 Connectionless transport: UDP
- 6.4 Connection-oriented transport: TCP
  - Segment structure
  - Connection management
  - Flow control
  - Reliable data transfer
- 6.5 TCP congestion control



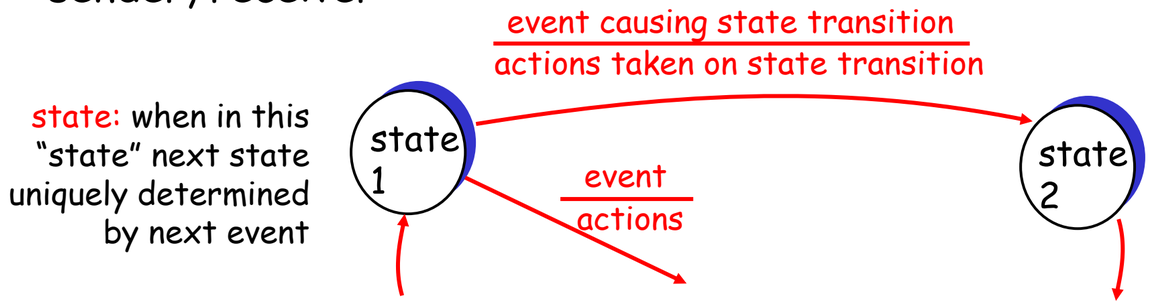
# 第6章概要

- 6.1 传输层服务
- 6.2 复用和分用
- 6.3 无连接传输：UDP
- 6.4 面向连接的传输：TCP
  - 分段结构 连接管理 流量控制 可靠数据传输
- 6.5 TCP拥塞控制

# Reliable Data Transfer: Getting Started

We'll:

- Incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)
- Consider only unidirectional data transfer
  - But control info will flow on both directions!
- Use finite state machines (FSM) to specify sender, receiver

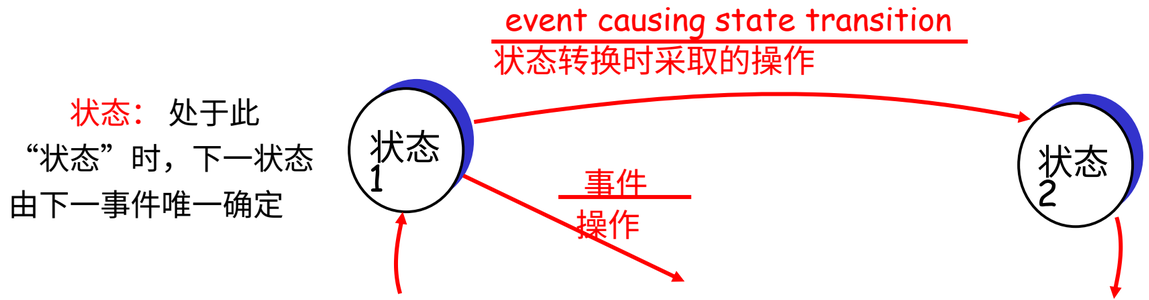





# 可靠数据传输：入门指南

我们将：

- 逐步开发可靠数据传输协议（rdt）的发送方和接收方
- 仅考虑单向数据传输
  - 但控制信息将在两个方向上流动！
- 使用有限状态机（FSM）来指定发送方、接收者



# Summary of the Protocols

- Rdt1.0: all packets arrive correctly
- Rdt2.0: all packets arrive, with possible errors only in data packets, and introducing ACK and NAK (no error)
- Rdt2.1: corrupted ACKs/NAKs
- Rdt2.2: similar to rdt2.1 remove NAKs
- Rdt3.0: Allows packets to be lost and errors

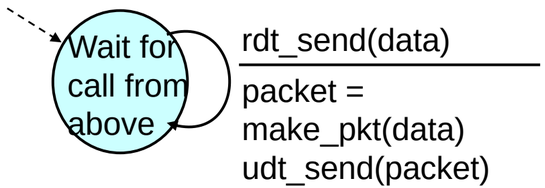


# 协议总结

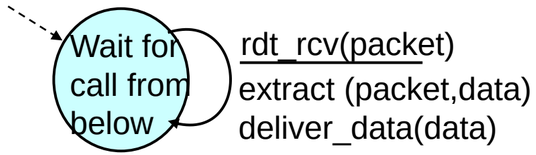
- Rdt1.0：所有数据包均正确到达
- Rdt2.0：所有数据包都能到达，但可能只有数据包会出错，并引入了确认（ACK）和否认（NAK）（无错误）
- Rdt2.1：损坏的确认（ACK）或否认（NAK）
- Rdt2.2：与rdt2.1类似，移除了否认（NAK）
- Rdt3.0：允许丢包和错误

# Rdt1.0: Reliable Transfer over a Reliable Channel

- Underlying channel perfectly reliable
  - No bit errors
  - No loss of packets
- Separate FSMs for sender, receiver:
  - Sender sends data into underlying channel
  - Receiver read data from underlying channel




sender

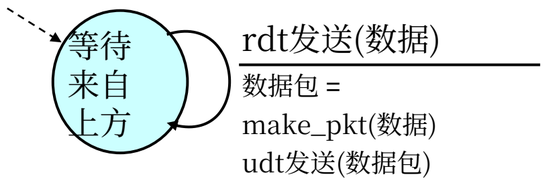



receiver

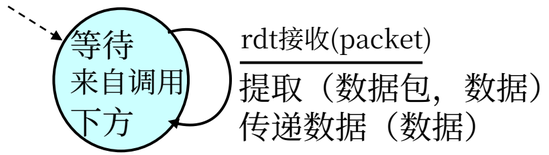


# Rdt1.0：在可靠信道上的可靠传输

- 底层信道完全可靠
  - 无比特错误 无丢包
- 发送方和接收方使用独立的有限状态机：
  - 发送方将数据发送到底层信道 接收方从底层信道读取数据




发送方




接收者

# Rdt2.0: Channel with Bit Errors

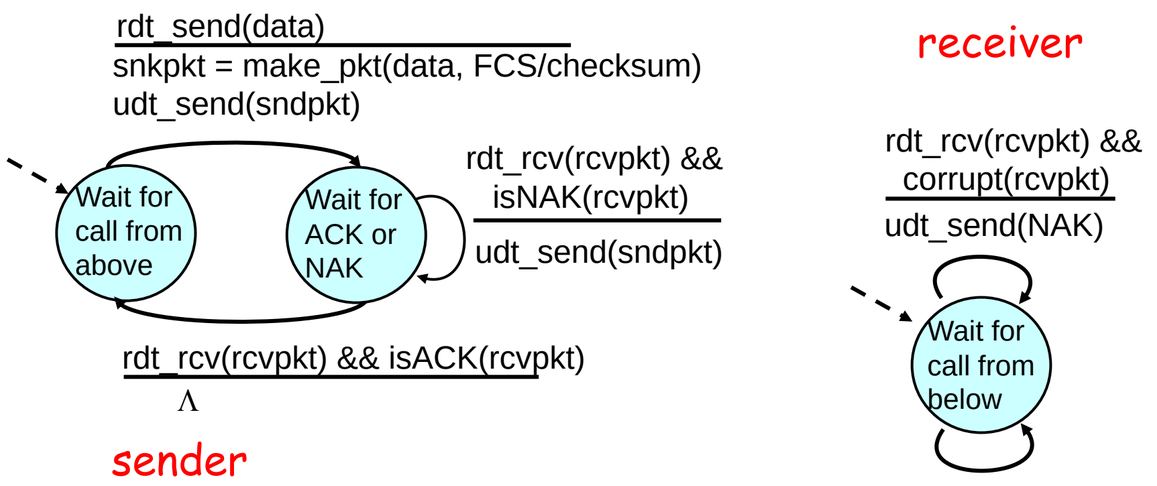
- Underlying channel may flip bits in packet
  - FCS/Checksum to detect bit errors
- *The question:* how to recover from errors:
  - *Acknowledgements (ACKs):* receiver explicitly tells sender that pkt received OK
  - *Negative acknowledgements (NAKs):* receiver explicitly tells sender that pkt had errors
  - Sender retransmits pkt on receipt of NAK
- New mechanisms in **rdt2.0** (beyond **rdt1.0**):
  - Error detection
  - Receiver feedback: control msgs (ACK,NAK) rcvr->sender



# Rdt2.0：带位错误的信道

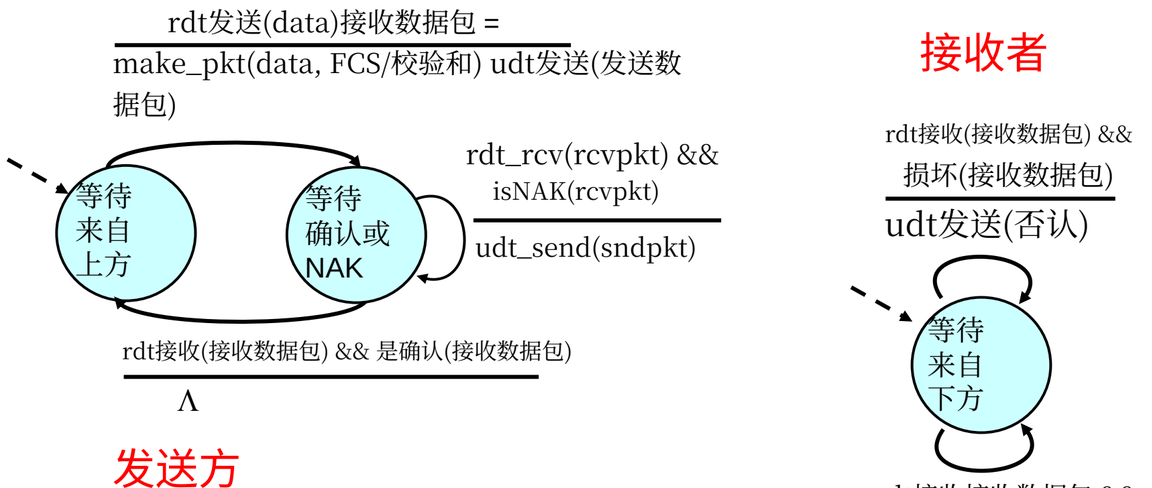
- 底层信道可能会在数据包中翻转比特 FCS/校验和用于检测比特错误
- 问题：如何从错误中恢复：
  - 确认 (ACKs)：接收者明确告诉发送者数据包接收正常
  - 否认 (NAKs)：接收者明确告诉发送者数据包存在错误
  - 发送者在收到否认时重传数据包
- rdt2.0中的新机制（超越rdt1.0）：
  - 错误检测 接收器反馈：控制消息（确认，否认）接收者->发送方

# Rdt2.0: FSM specification

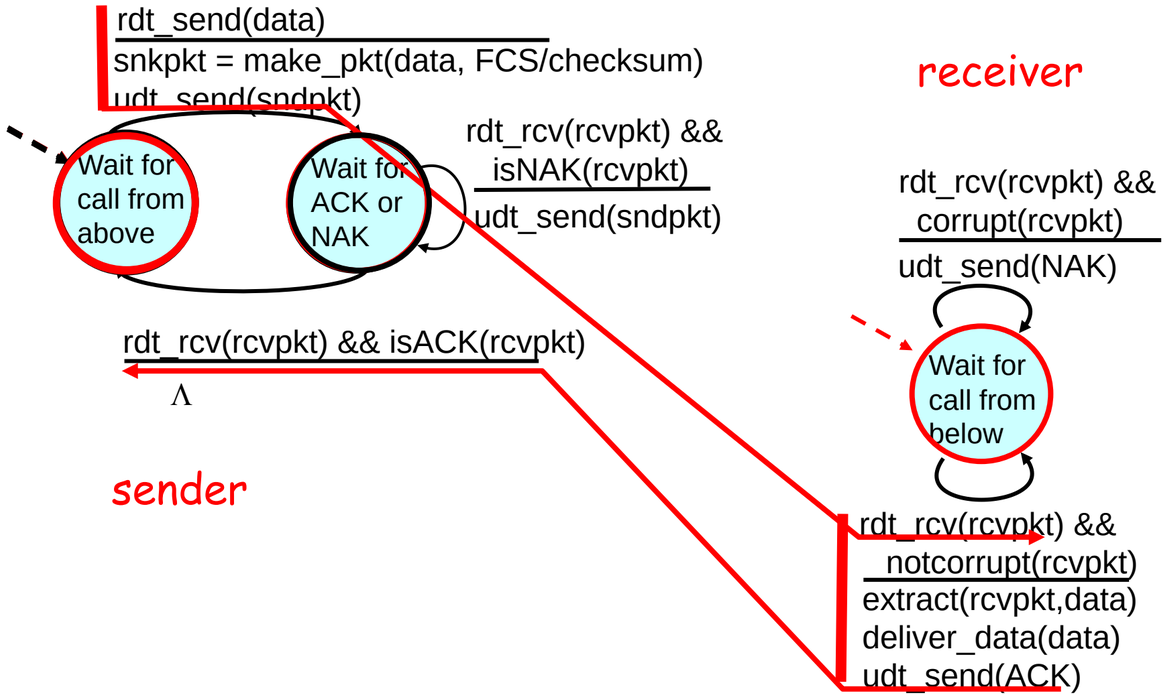





# Rdt2.0：FSM规范

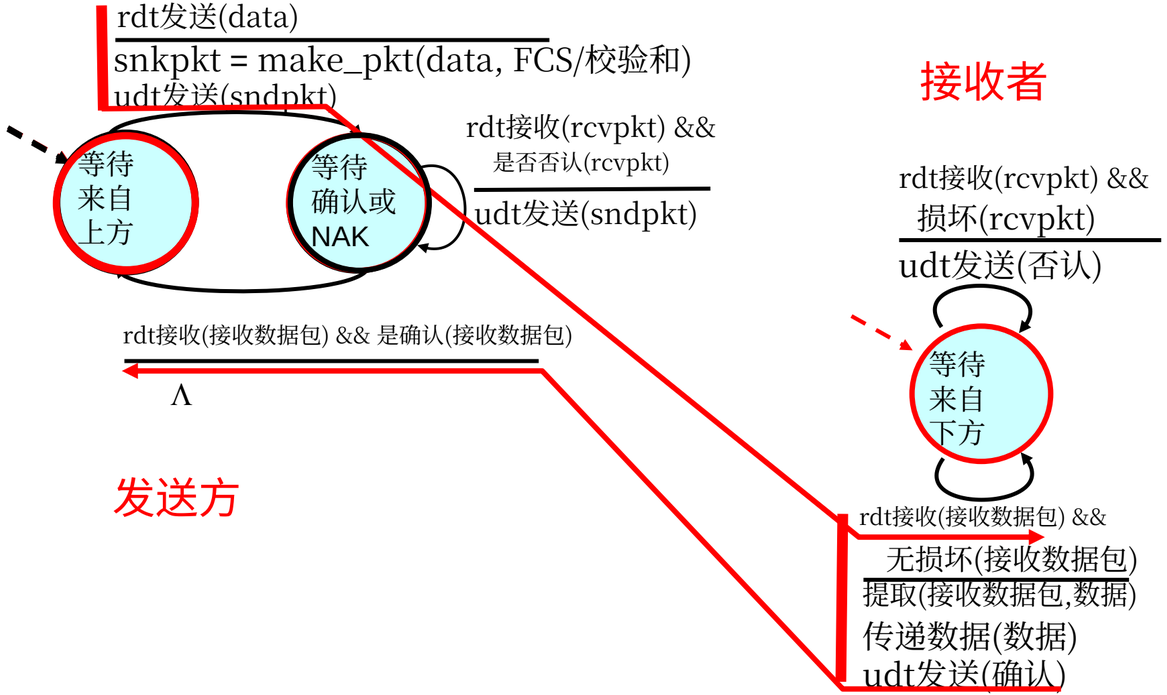


# Rdt2.0: Operation with No Errors

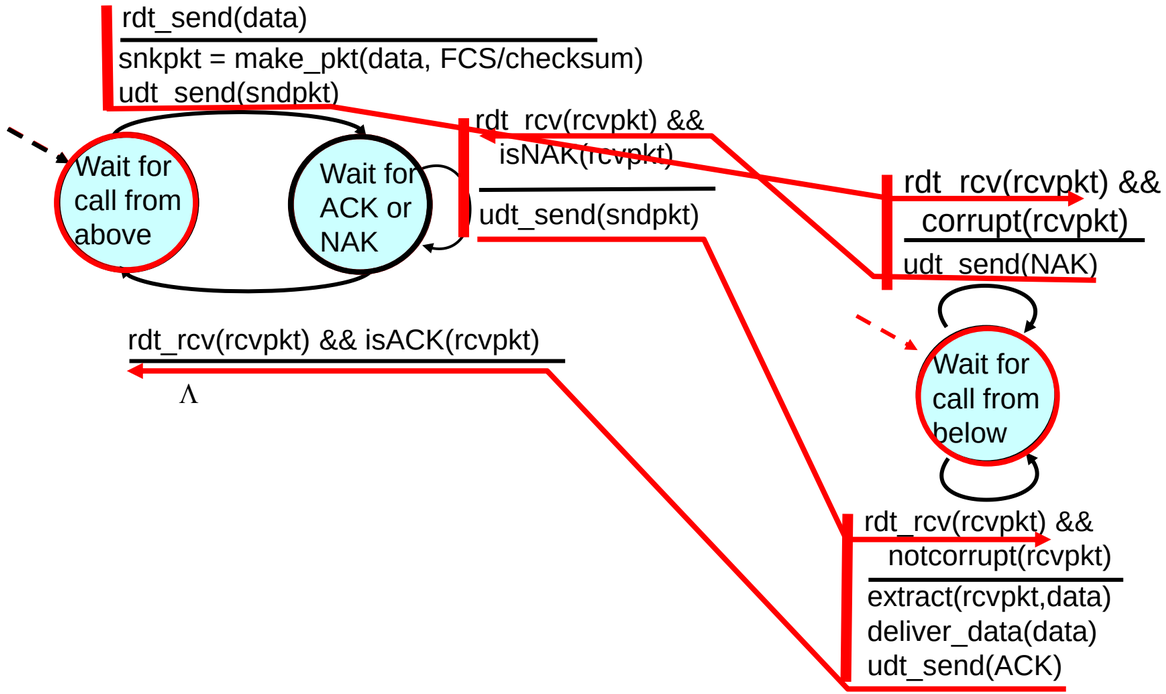





# Rdt2.0：无错误操作

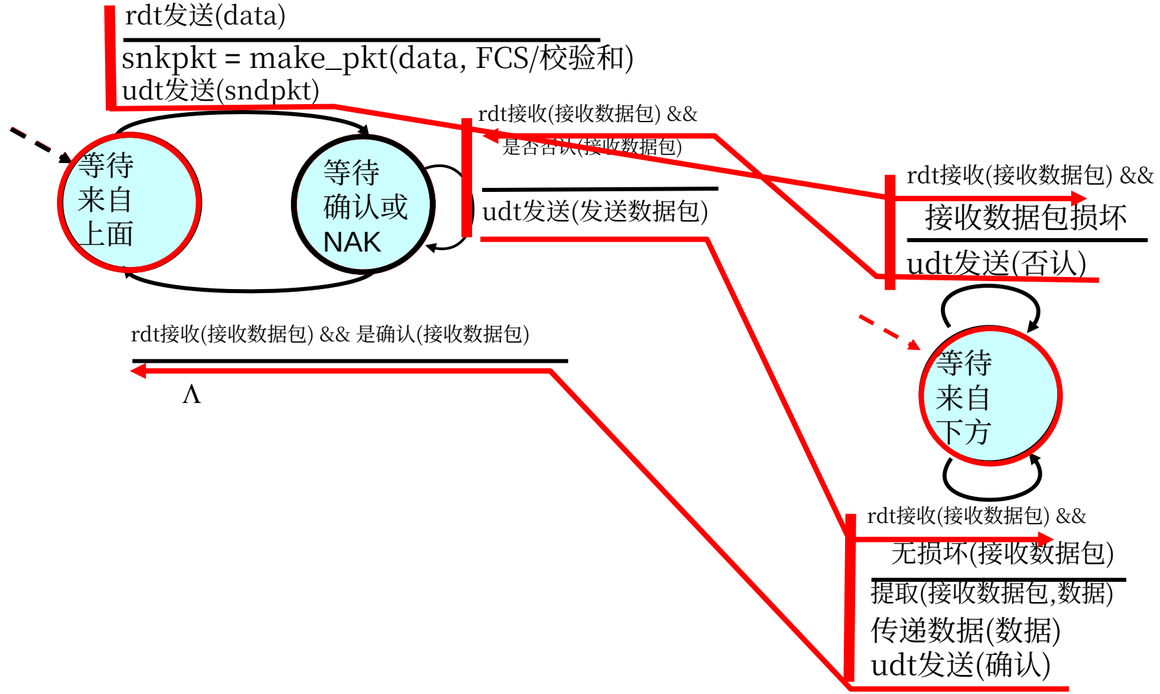


# Rdt2.0: Error Scenario






# Rdt2.0：错误场景



# Rdt2.0 Has a Fatal Flaw!

## What happens if ACK/NAK corrupted?

- Sender doesn't know what happened at receiver!
- Can't just retransmit: possible duplicate



## Handling duplicates:

- Sender retransmits current pkt if ACK/NAK garbled
- Sender adds *sequence number* to each pkt
- Receiver discards (doesn't deliver up) duplicate pkt

**stop and wait**
Sender sends one packet, then waits for receiver response



# Rdt2.0 存在致命缺陷！

## 如果 ACK/NAK 损坏会怎样？

- 发送方不知道接收方发生了什么！
- 不能仅重传：可能存在重复



## 处理重复数据包：

- 发送方若确认/否认信息损坏，则重传当前数据包
- 发送方为每个数据包添加序列号
- 接收方丢弃（不向上传递）重复数据包

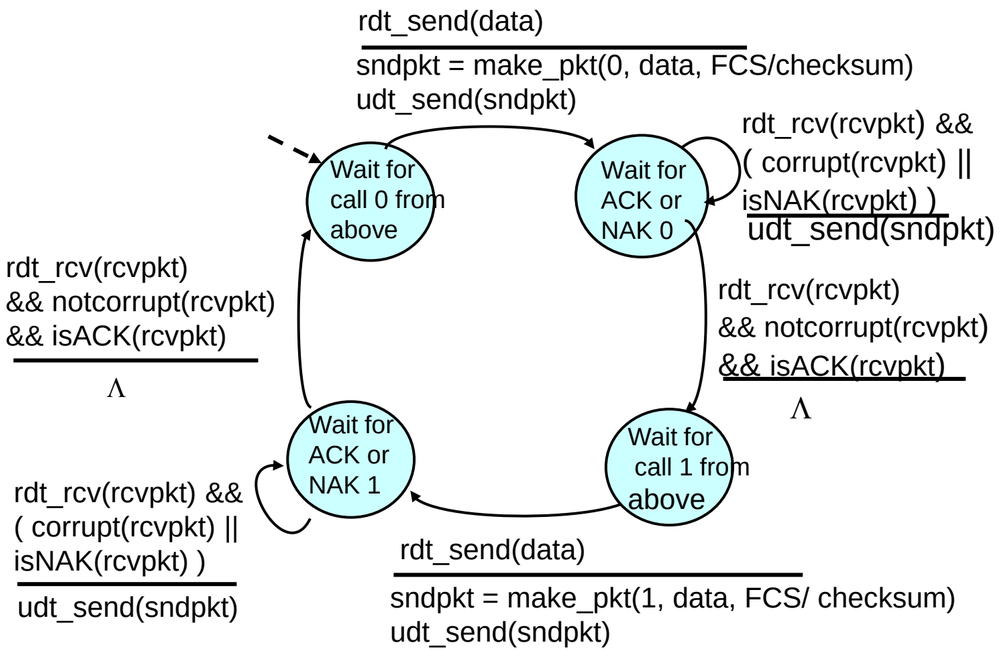
**停止等待**
发送方发送一个数据包，然后等待接收方响应

- Sender: whenever sender receives control message it sends a packet to receiver
  - A valid ACK: Sends next packet (if exists) with new sequence #
  - A NAK or corrupt response: resends old packet
- Receiver: sends ACK/NAK to sender
  - If received packet is corrupt: send NAK
  - If received packet is valid and has different sequence # as prev packet: send ACK and deliver new data up
  - If received packet is valid and has same sequence # as prev packet, i.e., is a retransmission of duplicate: send ACK
- Note: ACK/NAK **do not** contain sequence #



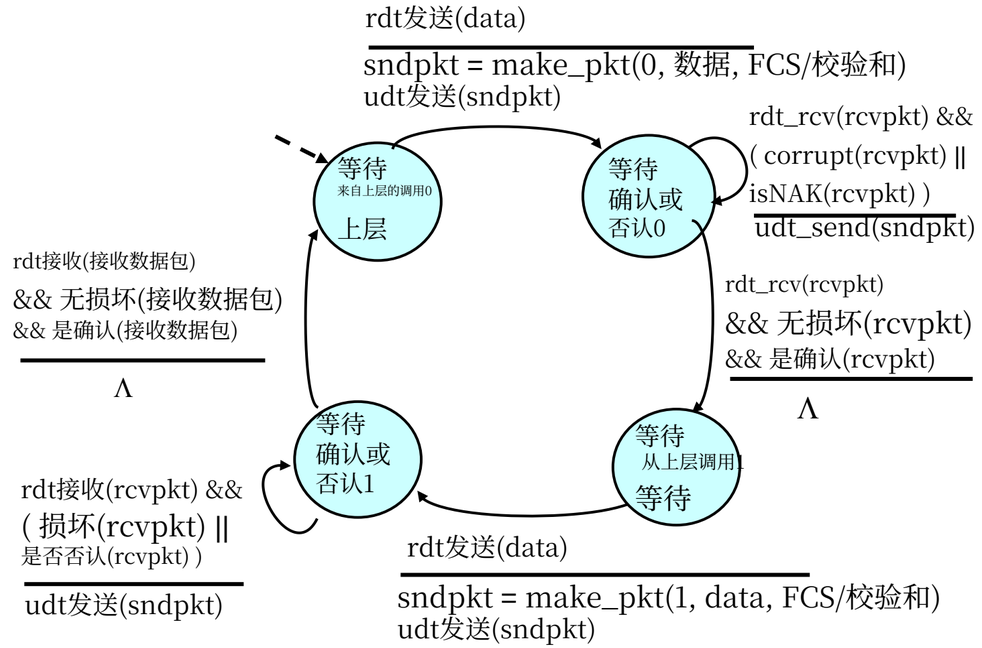
- 发送方：每当发送方收到控制消息时，它就会向接收者发送一个数据包
  - 有效的确认：发送下一个数据包（如果存在），并使用新的序列号 #
  - 无效的确认或损坏的响应：重传旧数据包
- 接收者：向发送方发送确认/否认
  - 如果接收到的数据包损坏：发送否认
  - 如果接收到的数据包有效且序列号与前一数据包不同：发送确认并传递新数据
  - 如果接收到的数据包有效且序列号与前一数据包相同，即重复传输：发送确认
- 注意：确认/否认 **不** 包含序列号

# Rdt2.1: Sender, Handles Garbled ACK/NAKs

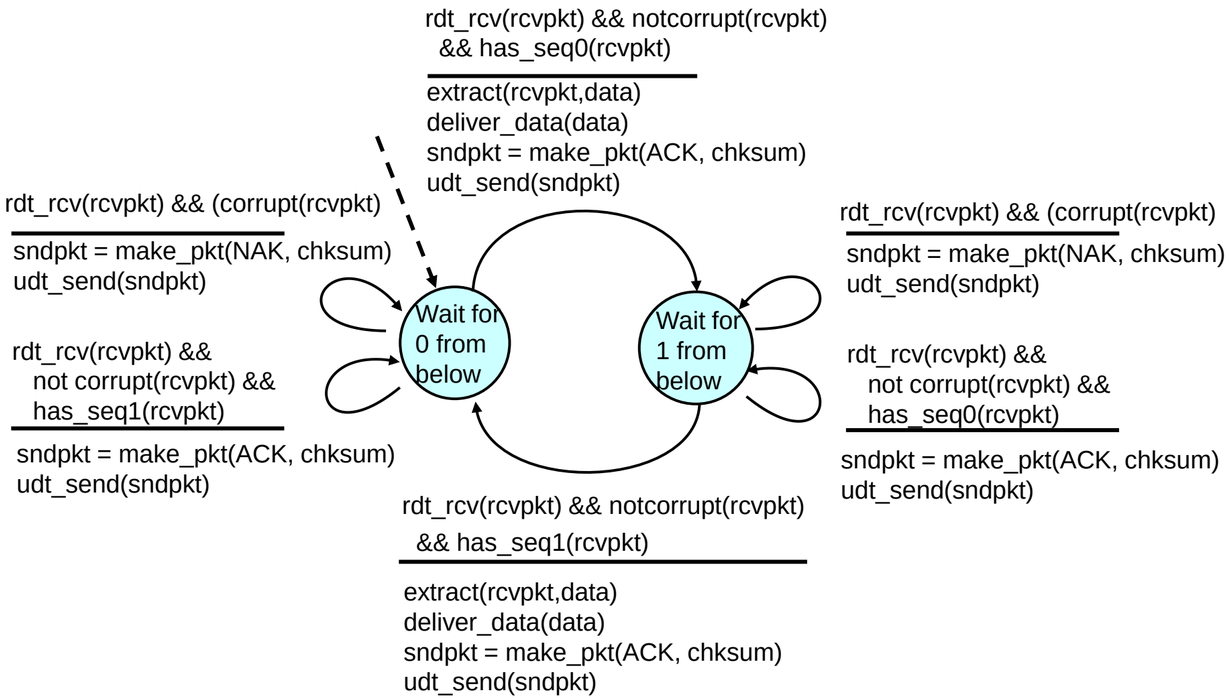





# Rdt2.1：发送方，处理损坏的确认/否认

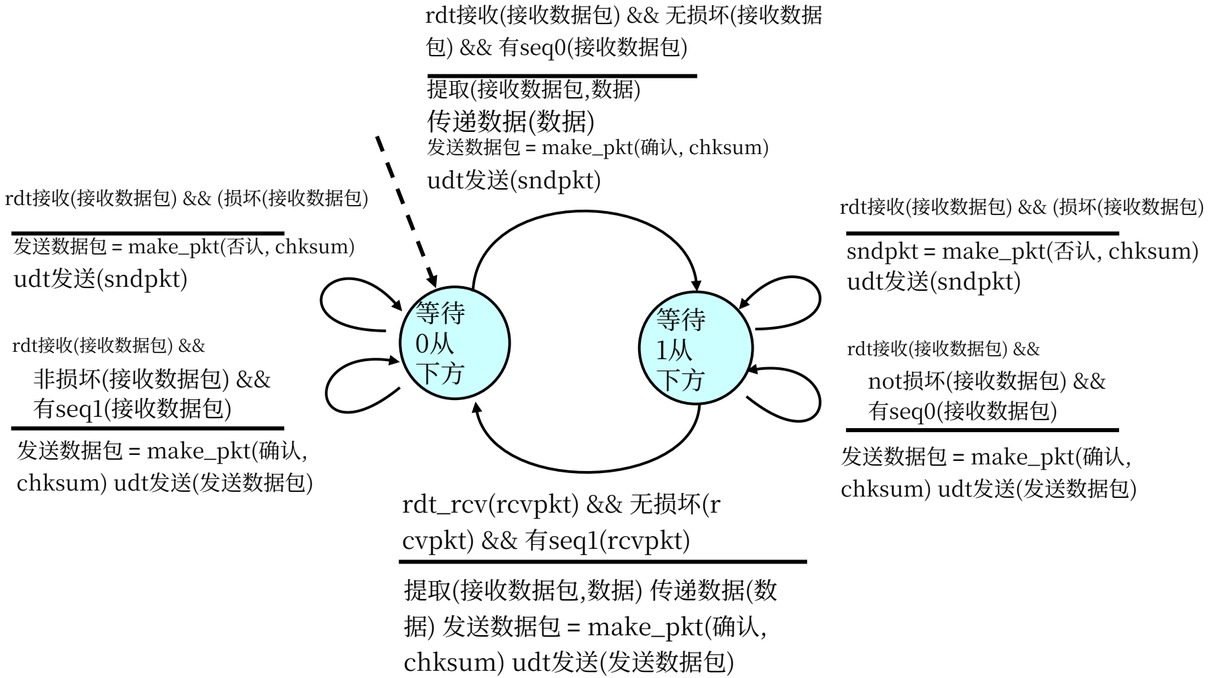


# Rdt2.1: Receiver, Handles Garbled ACK/NAKs






# Rdt2.1：接收者，处理损坏的确认/否认ACK/NAKs



# Rdt2.1: Discussion

Sender:

- Seq # added to pkt
- Two seq. #'s (0,1) will suffice. Why?
- Must check if received ACK/NAK corrupted
- Twice as many states
  - State must "remember" whether "current" pkt has 0 or 1 seq. #

Receiver:

- Must check if received packet is duplicate
  - State indicates whether 0 or 1 is expected pkt seq #
- Note: receiver can *not* know if its last ACK/NAK received OK at sender



# Rdt2.1：讨论

发送方：

- 序列号添加到数据包
- 两个序列号（0，1）就足够了。为什么?
- 必须检查接收到的确认/否认是否损坏
- 状态数翻倍
  - 状态必须“记住”当前数据包是否具有 0 或 1 个序列号。#

接收者：

- 必须检查收到的数据包是否是重复的
  - 状态指示预期接收 0 或 1 的数据包序列号
- 注意：接收者无法知道其收到的最后一个确认/否认是否在发送方成功接收

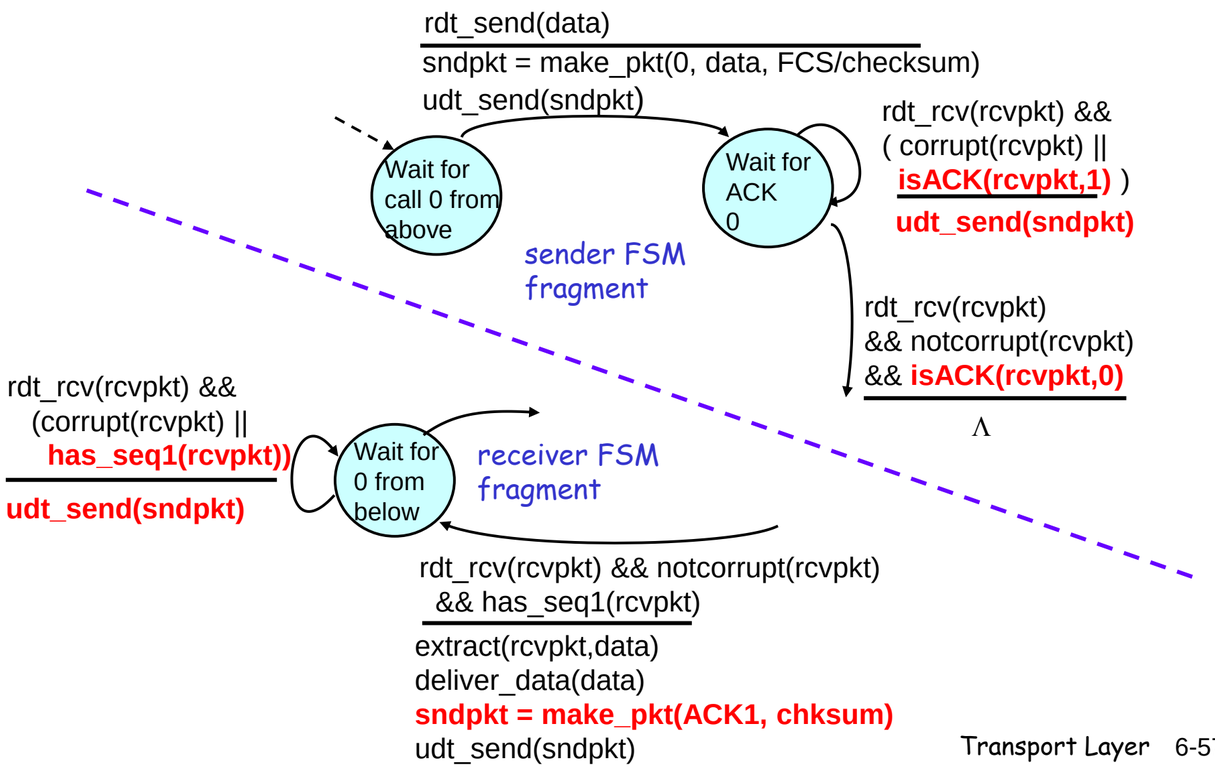
# Rdt2.2: a NAK-free Protocol

- Same functionality as rdt2.1, using ACKs only
- Instead of NAK, receiver sends ACK for last pkt received OK
  - Receiver must *explicitly* include seq # of pkt being ACKed
- Duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

# Rdt2.2: 一个无否认协议-free Protocol

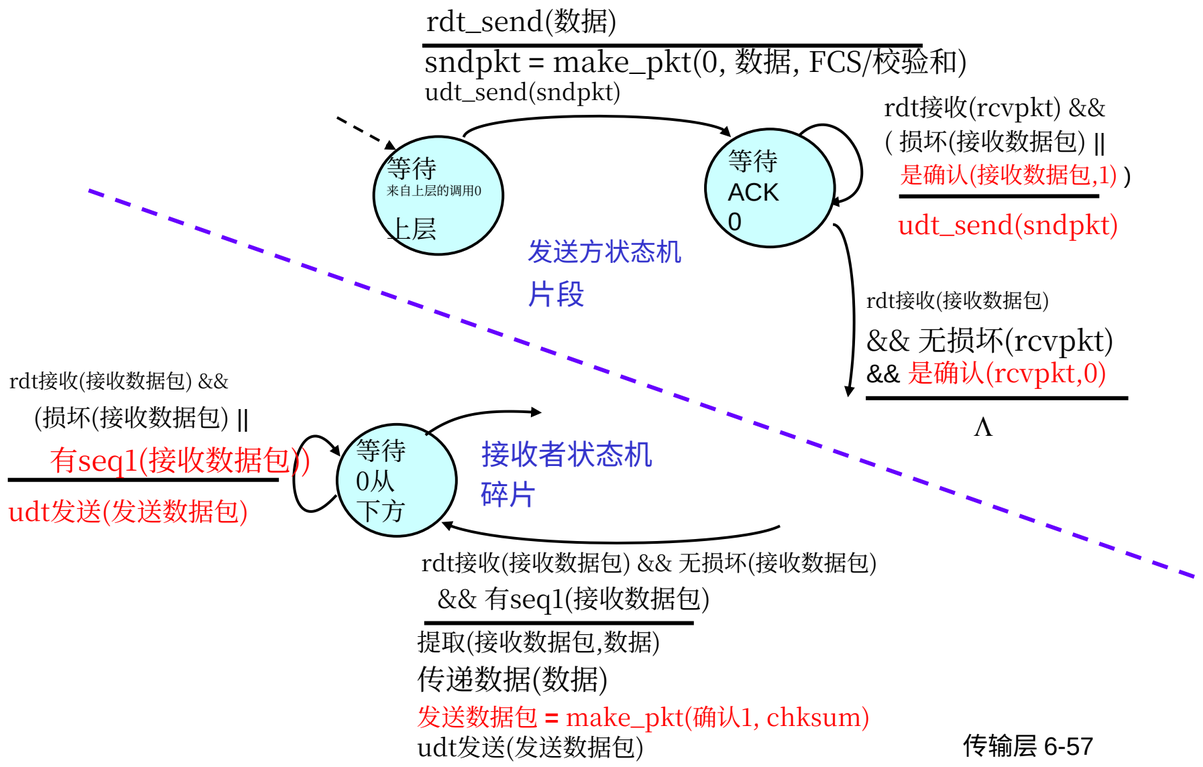
- 与 rdt2.1 具有相同的功能，仅使用确认
- 发送方不再发送否认，而是对收到的最后一个数据包发送确认
  - 接收方必须 明确包含正在确认的数据包的序列号
- 发送方收到的重复确认与否认的作用相同：重传当前数据包

# Rdt2.2: Sender, Receiver Fragments






# Rdt2.2: 发送方，接收者片段



# Rdt3.0: Channels with Errors and Loss

New assumption: Underlying channel can also lose packets (data or ACKs)

- Checksum, seq. #, ACKs, retransmissions will be of help, but not enough

Q: how to deal with loss?

- Sender waits until certain data or ACK lost, then retransmits
- Any drawbacks?

Approach: sender waits "reasonable" amount of time for ACK

- Retransmits if no ACK received in this time
- If pkt (or ACK) just delayed (not lost):
  - Retransmission will be duplicate, but use of seq. #'s already handles this
  - Receiver must specify seq # of pkt being ACKed
- Requires countdown timer



# Rdt3.0：存在错误和丢包的信道

新的假设：底层信道也可能丢包（数据或确认）

- 校验和、序列号、确认、重传会有帮助，但还不够

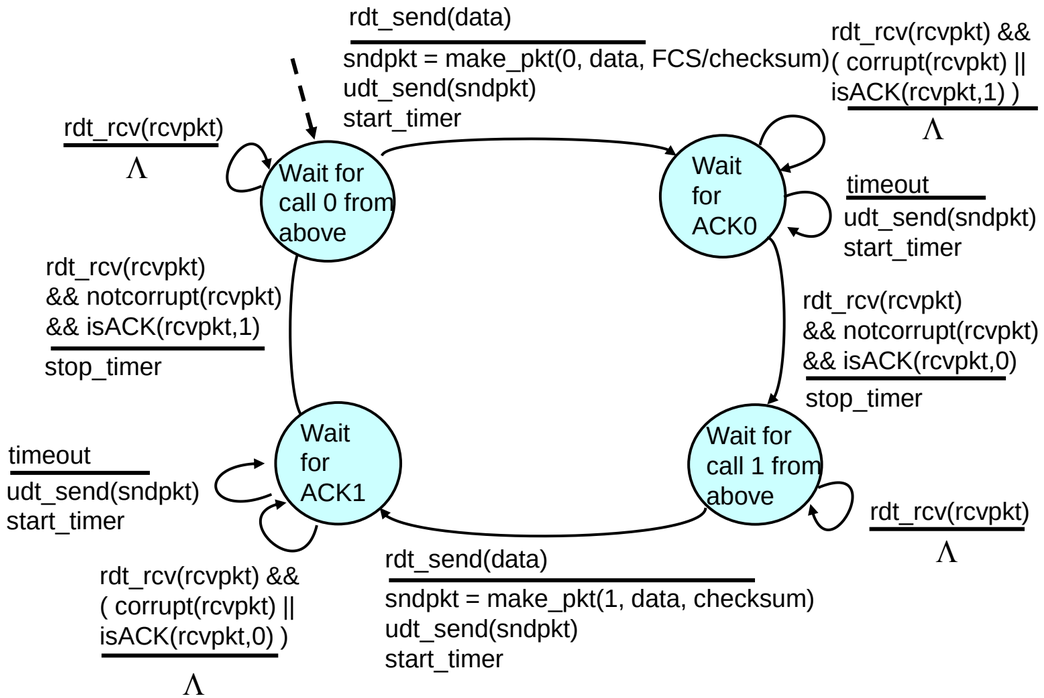
Q：如何处理丢包？

- 发送方等待特定数据或确认丢失后，再重传○ 有什么缺点?

方法：发送方等待“合理”的时间确认❒ 如果在规定时间内未收到确认，则重传❒ 如果数据包（或确认）只是延迟（未丢失）：○ 重传将是重复的，但序列号的使用已经处理了这种情况○ 接收方必须指定正在确认的数据包的序列号

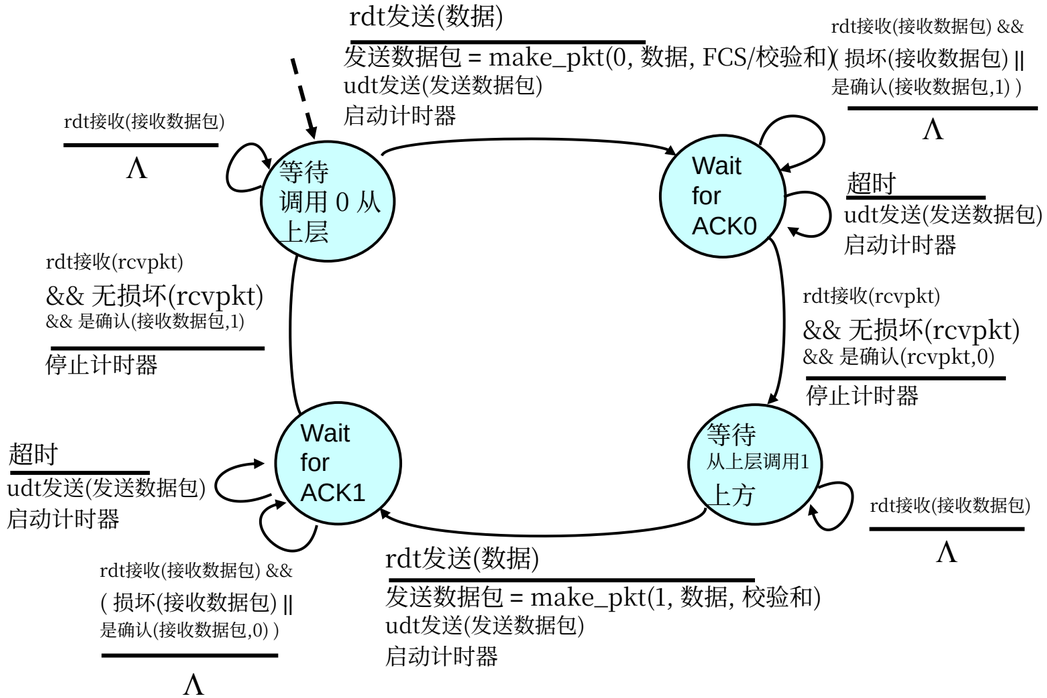
- 需要倒计时计时器

# Rdt3.0 Sender

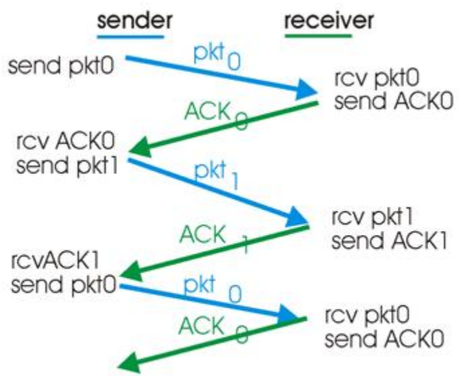




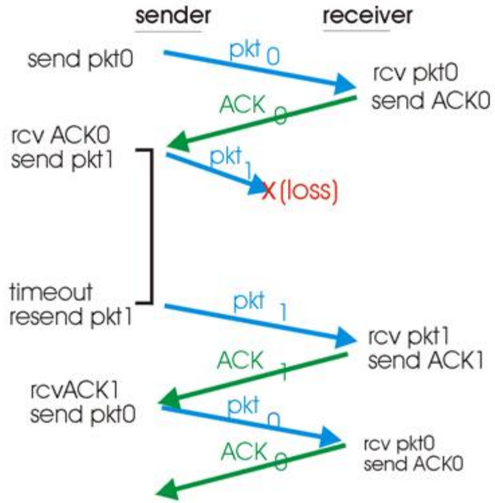

# Rdt3.0 发送方



# Rdt3.0 In Action

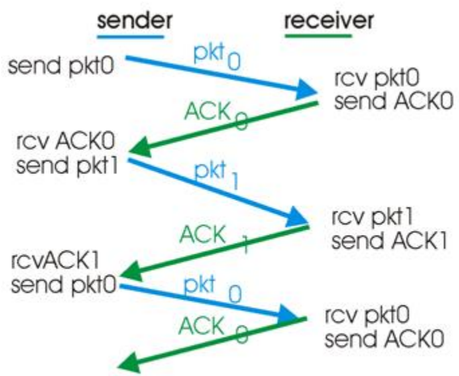



(a) operation with no loss

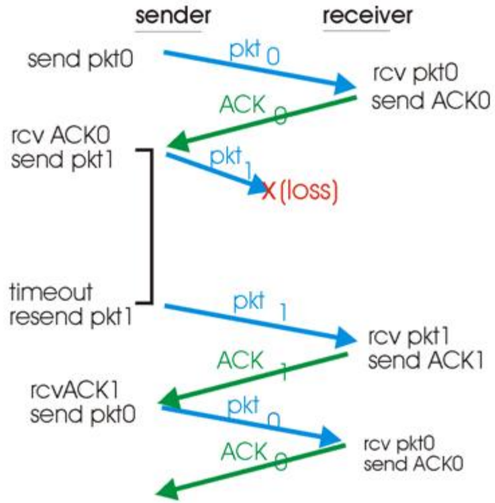



(b) lost packet

# Rdt3.0 在行动

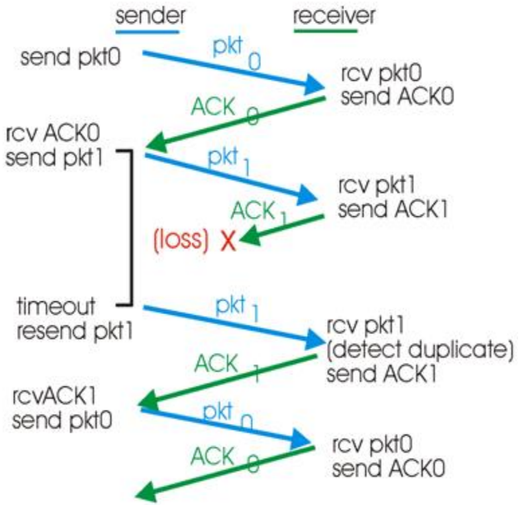



(a) operation with no loss

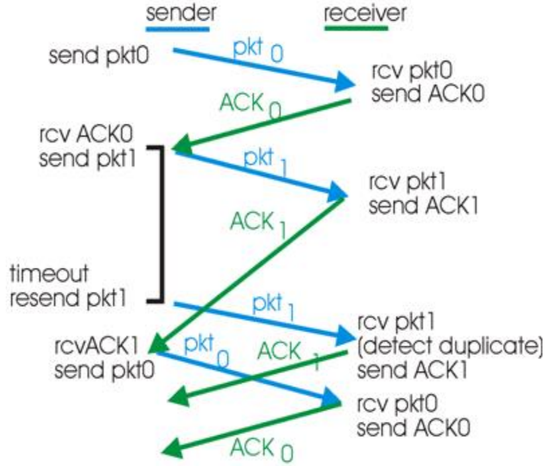



(b) lost packet

# Rdt3.0 In Action

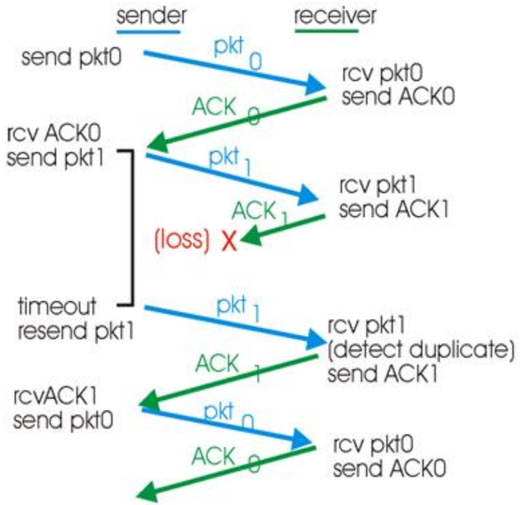



(c) lost ACK

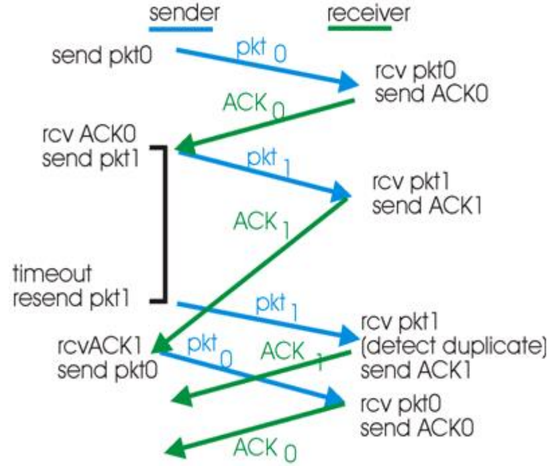



(d) premature timeout

# Rdt3.0 在行动




(c) lost ACK




(d) premature timeout

# Performance of rdt3.0



- Rdt3.0 works, but performance stinks
- Example: 1 Gbps link, 15 ms e-e prop. delay, 1KB packet:

$$T_{transmit} = \frac{L \text{ (packet length in bits)}}{R \text{ (transmission rate, bps)}} = \frac{8kb/pkt}{10^{**}9 \text{ b/sec}} = 8 \text{ microsec}$$

  - $U_{sender}$: utilization – fraction of time sender busy sending

$$U_{sender} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

  - 1KB pkt every 30 msec -> 33kB/sec thruput over 1 Gbps link
  - Network protocol limits use of physical resources!



# rdt3.0 的性能



- Rdt3.0 可以工作，但性能很差- 示例：1 Gbps 链路，15 ms 传播延迟，1KB 数据包：

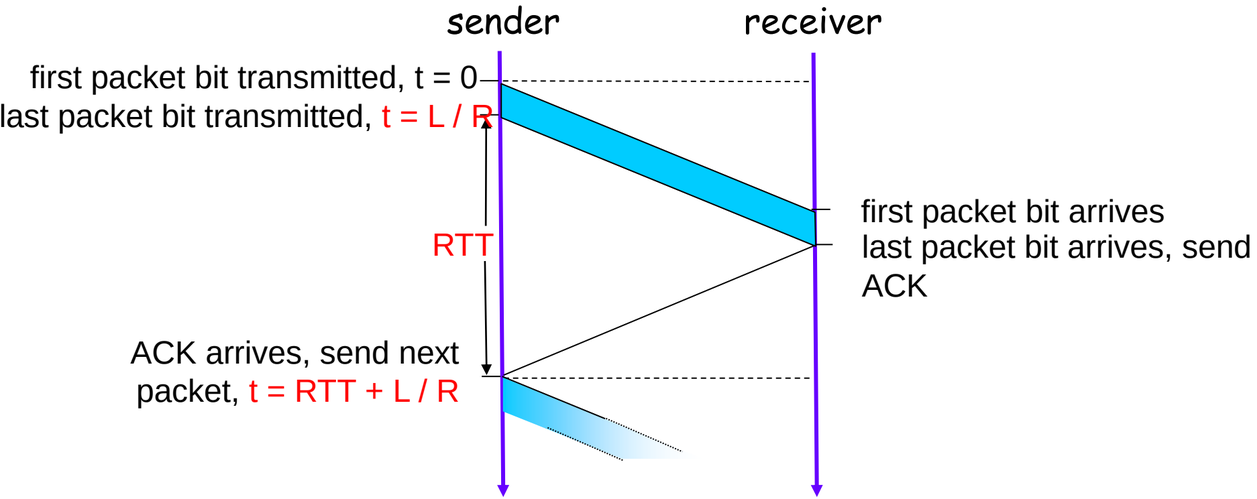
Ttransmit= L (数据包长度，比特) / R (传输速率，比特/秒) = 8kb/数据包 10**9 b/秒 = 8 微秒

  - $U_{发送方}$: 利用率- 发送方忙碌发送的时间比例

$$U_{发送方} = \frac{L/R}{往返时间 + L/R} = \frac{.008}{30.008} = 0.00027 \text{ 微秒}$$

  - 1每 30 ms 发送 1KB 数据包 -> 33超过 1 Gbps 链路的 kB/秒 吞吐量- 网络协议限制了对物理资源的使用！

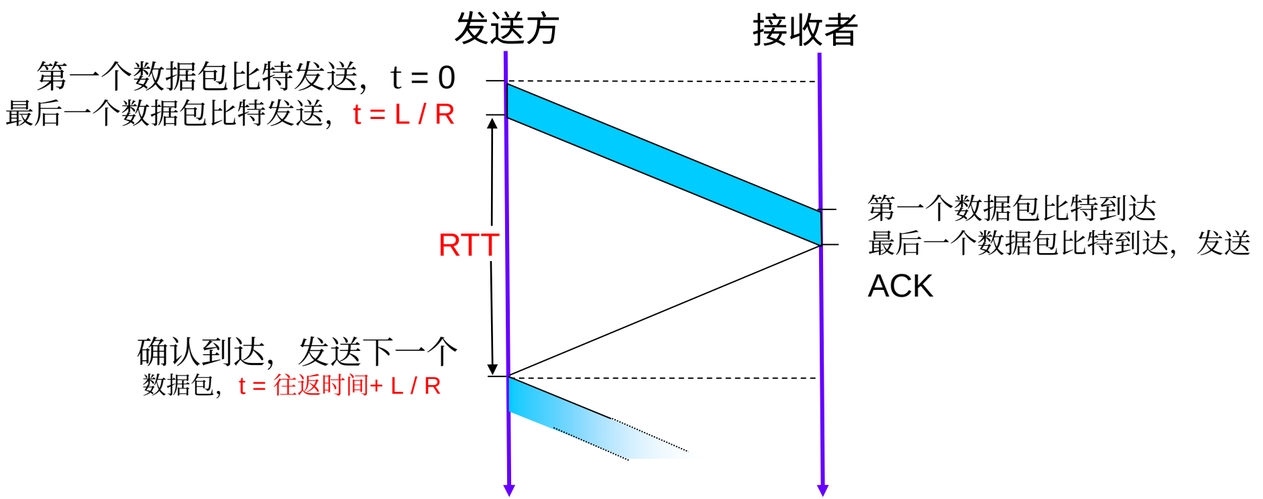
# Rdt3.0: Stop-and-Wait Operation




$$U_{sender} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$



# Rdt3.0: 停止-等待操作



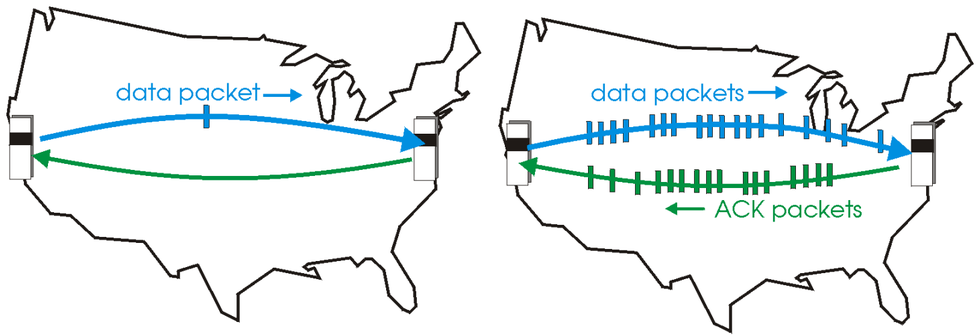

$$U_{发送方} = \frac{L/R}{往返时间 + L/R} = \frac{.008}{30.008} = 0.00027$$

微秒

# Pipelined Protocols

Pipelining: sender allows multiple, "in-flight", yet-to-be-acknowledged pkts

- Range of sequence numbers must be increased
- Buffering at sender and/or receiver



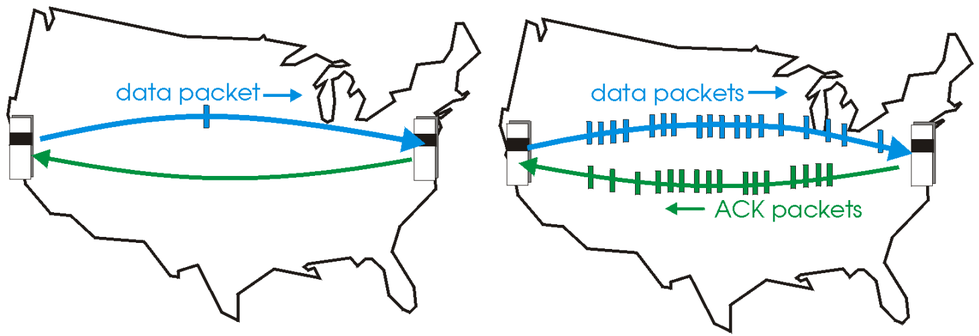

(a) a stop-and-wait protocol in operation (b) a pipelined protocol in operation



# 流水线协议

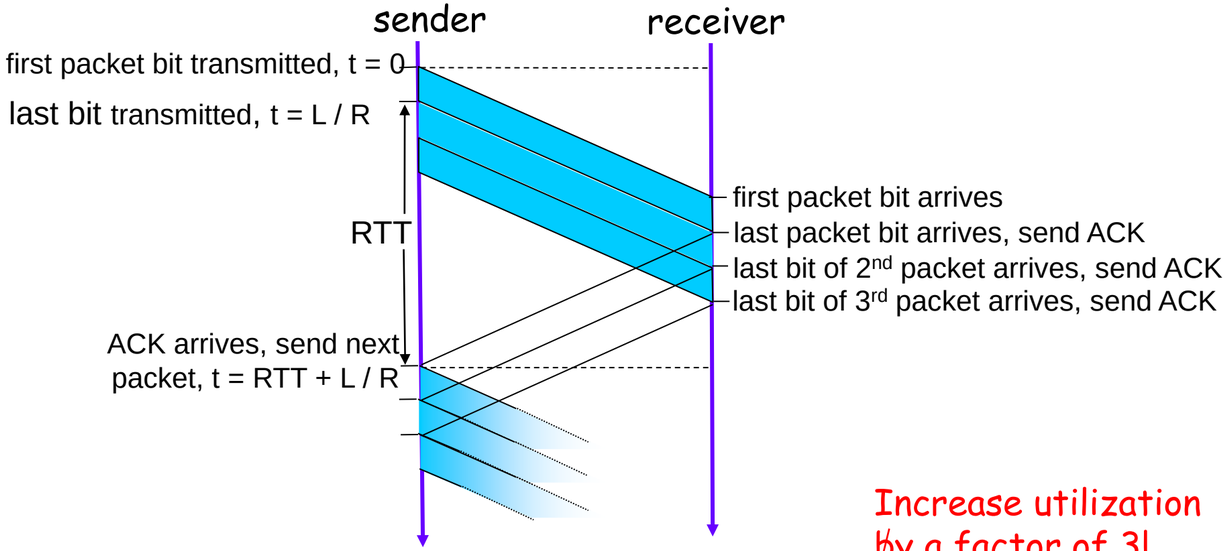
流水线技术：发送方允许多个，“-在飞中”，尚未确认的数据包

- 序列号的范围必须增加
- 发送方和/或接收方的缓冲




(a) a stop-and-wait protocol in operation (b) a pipelined protocol in operation

# Pipelining: Increased Utilization

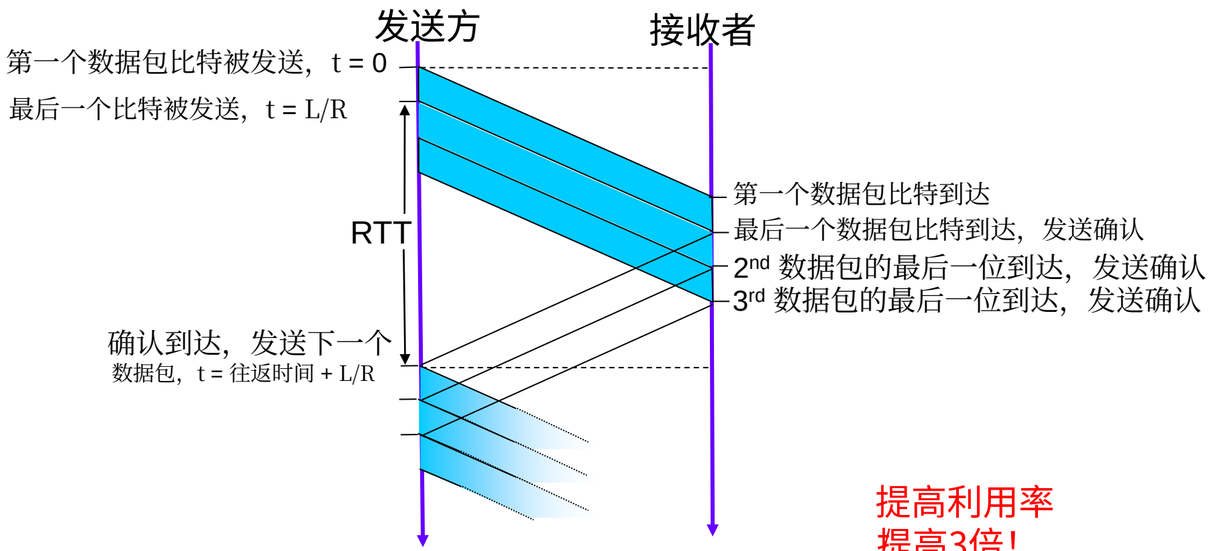



Increase utilization by a factor of 3!

$$U_{sender} = \frac{3 * L / R}{RTT + L / R} = \frac{.024}{30.008} = 0.0008$$



# 流水线技术：提高利用率




提高利用率提高3倍！

$$U_{发送方} = \frac{3 * L/R}{往返时间 + L/R} = \frac{.024}{30.008} = 0.0008$$

微秒

# Pipelined Protocols

- Advantage: much better bandwidth utilization than stop-and-wait
- Disadvantage: More complicated to deal with reliability issues, e.g., corrupted, lost, out of order data.
  - Two generic approaches to solving this
    - *Go-Back-N protocols*
    - *Selective repeat protocols*



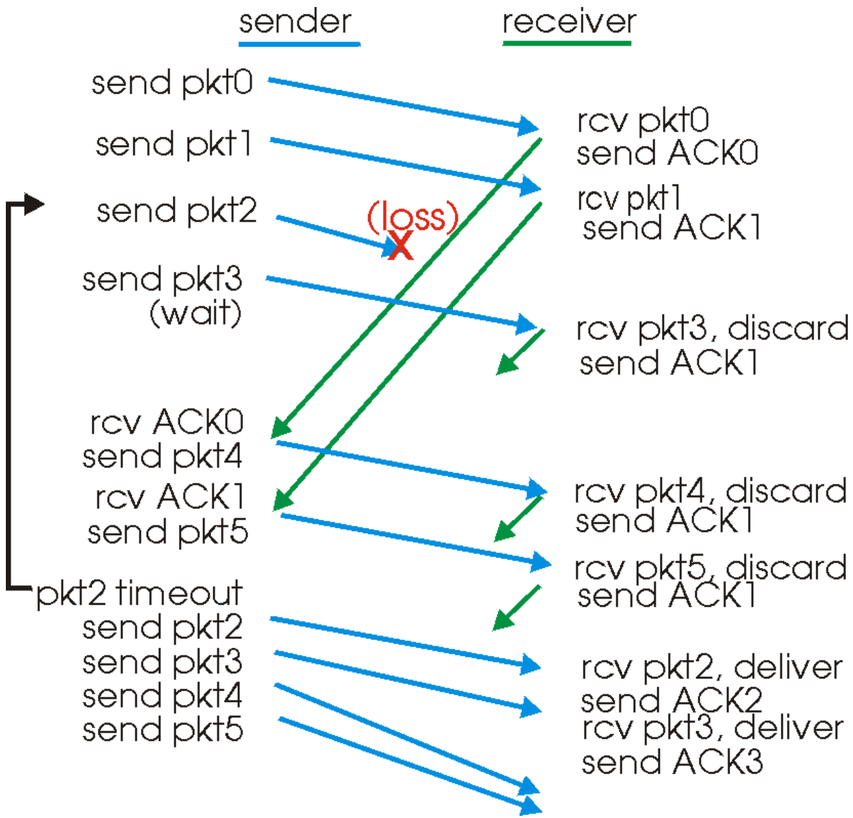


# 流水线协议

- 优点：比停止等待协议的带宽利用率好得多
- 缺点：处理可靠性问题更复杂，例如损坏、丢失、乱序数据
  - 解决这个问题的两种通用方法
    - 回退N 协议
    - 选择重发 协议

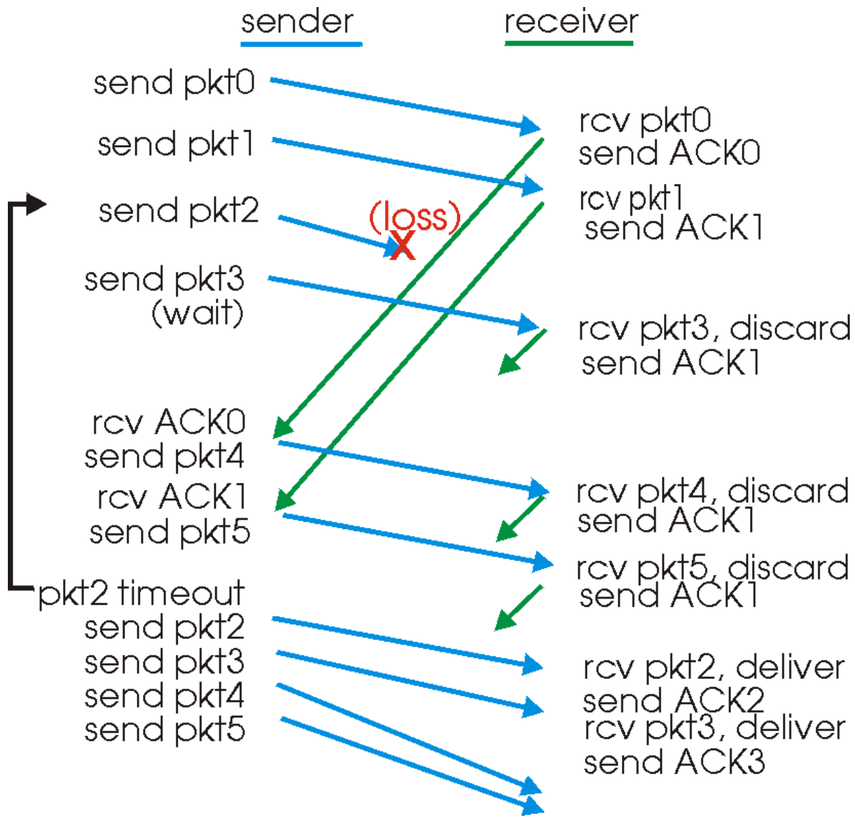


# GBN In Action





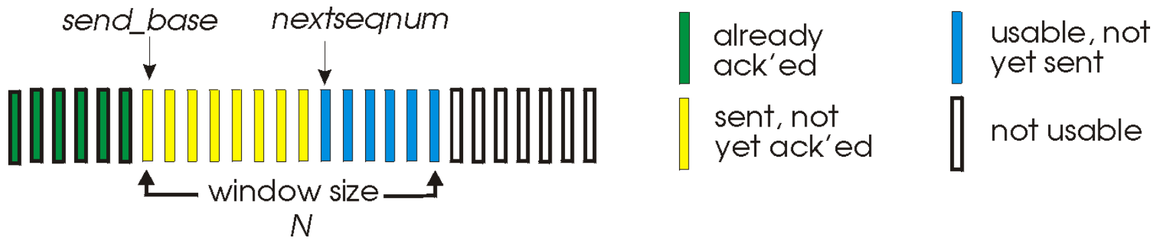

# GBN 在行动中



# Go-Back-N

Sender:

- K-bit seq # in pkt header
- "Window" of up to N, consecutive unack'ed pkts allowed



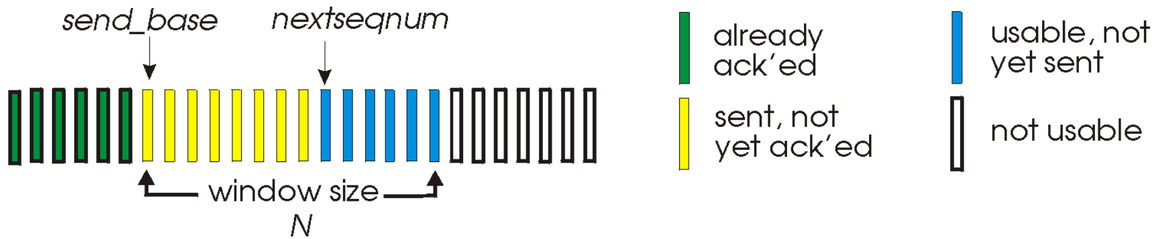

- ACK(n): ACKs all pkts up to, including seq # n - "cumulative ACK"
  - May receive duplicate ACKs (see receiver)
- Timer for each in-flight pkt
- *Timeout(n)*: retransmit pkt n and all higher seq # pkts in window
- Called a sliding-window protocol



# 去-反向N

发送方：

- 数据包头部中的K位序列号 “窗口”大小最多为N，允许连续未确认的数据包




- ACK(n)：确认编号为n及之前的所有数据包—— “累积确认” "
  - 可能收到重复确认（参见接收方） 每个在飞数据包的计时器
- 超时(n)：重传编号为n及窗口中所有更高序列号的数据包

w

- 被称为滑动窗口协议

# GBN: Sender

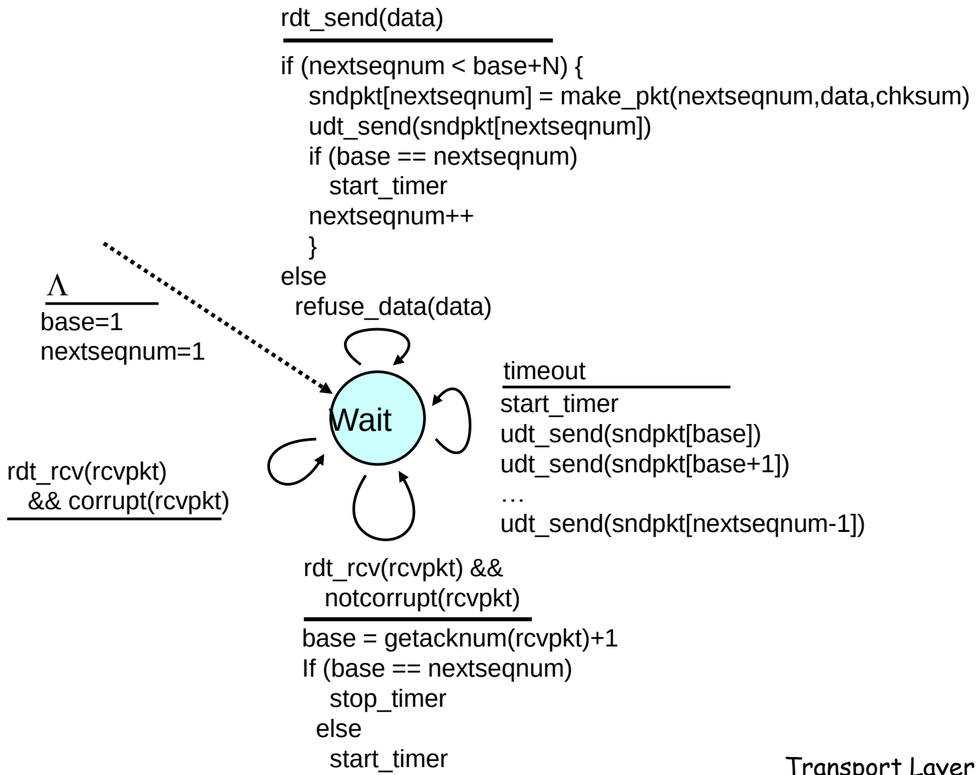
- rdt_Send() called: checks to see if window is full
  - No: send out packet
  - Yes: return data to upper level
- Receipt of ACK(n): cumulative acknowledgement that all packets up to and including *n* have been received. Updates window accordingly and restarts timer
- Timeout: resends ALL packets that have been sent but not yet acknowledged.
  - This is only event that triggers resend



# GBN: 发送方

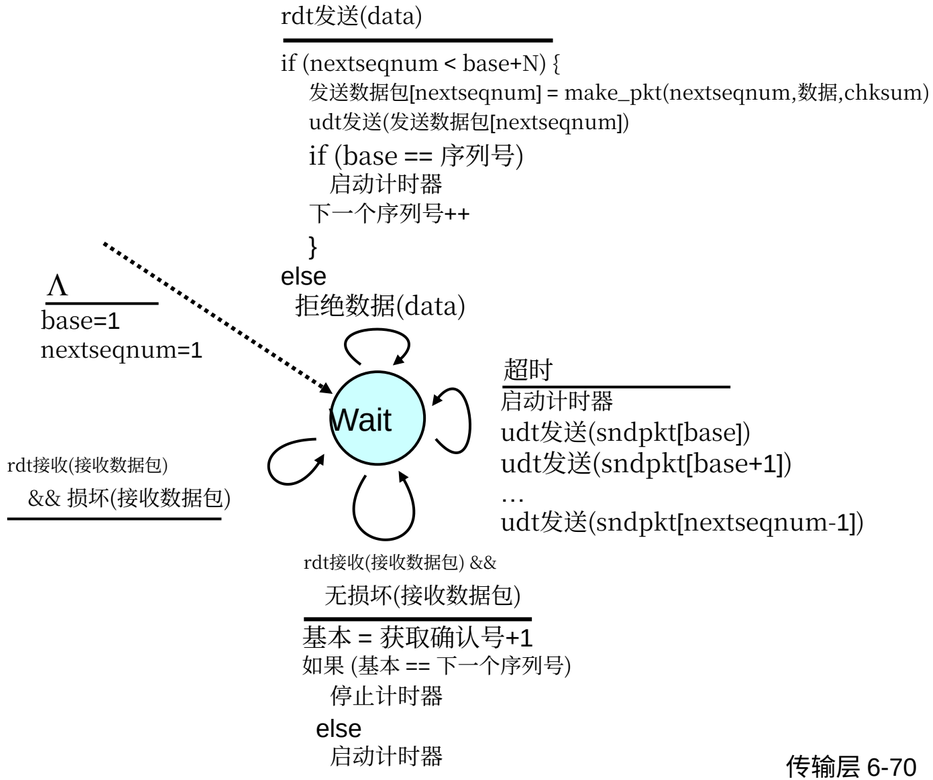
- rdt_Send() 被调用：检查窗口是否已满
  - 否： 发送数据包 是： 返回数据给上层
- 接收确认(n)： 累积确认，表示所有编号至包括 *n* 的数据包均已接收。更新窗口并重新启动计时器
- 超时： 重发所有已发送但尚未确认的数据包。
  - 这是唯一触发重发的事件

# GBN: Sender Extended FSM

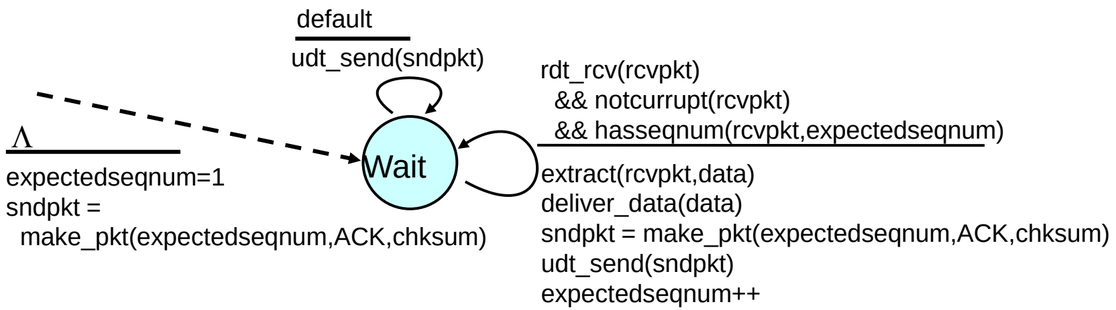





# GBN：发送方扩展状态机



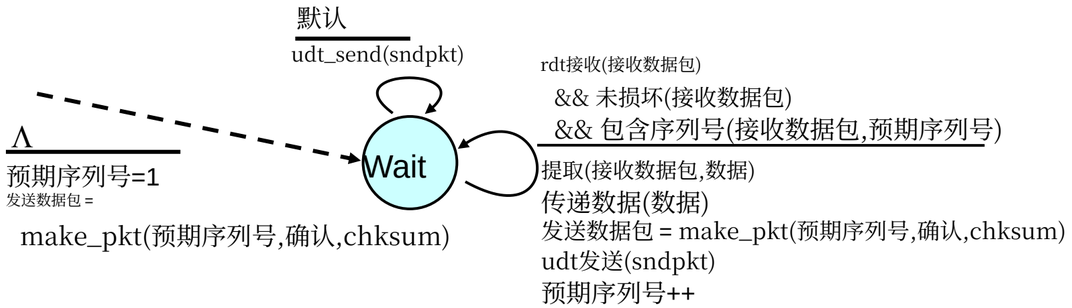
# GBN: Receiver Extended FSM




- If expected packet received:
  - Send ACK and deliver packet upstairs
- If out-of-order packet received:
  - Discard (don't buffer) -> no receiver buffering!
  - Re-ACK pkt with highest in-order seq #
  - May generate duplicate ACKs



# GBN: 接收器扩展有限状态机




- 如果 预期数据包 被接收：
  - 发送确认并向上传递数据包
- 如果 乱序数据包被接收：
  - 丢弃（不缓冲）-> 无接收器缓冲！ - 重新确认具有最高顺序号的数据包，可能产生重复确认

# More on Receiver

- ACK-only: the receiver always sends ACK for last correctly received packet with highest *in-order* seq #
- Receiver only sends ACKS (no NAKs)
- Need only remember **expectedseqnum**
- Can generate duplicate ACKs



# 更多关于接收者

- 仅ACK：接收器始终发送确认给最后正确接收的数据包，该数据包具有最高顺序号的
- 接收者仅发送确认（不发送否认）- 只需记住**预期序列号**- 可以生成重复确认

- GBN is easy to code but might have performance problems

- In particular, if many packets are in pipeline at one time (bandwidth-delay product large) then one error can force retransmission of huge amounts of data!

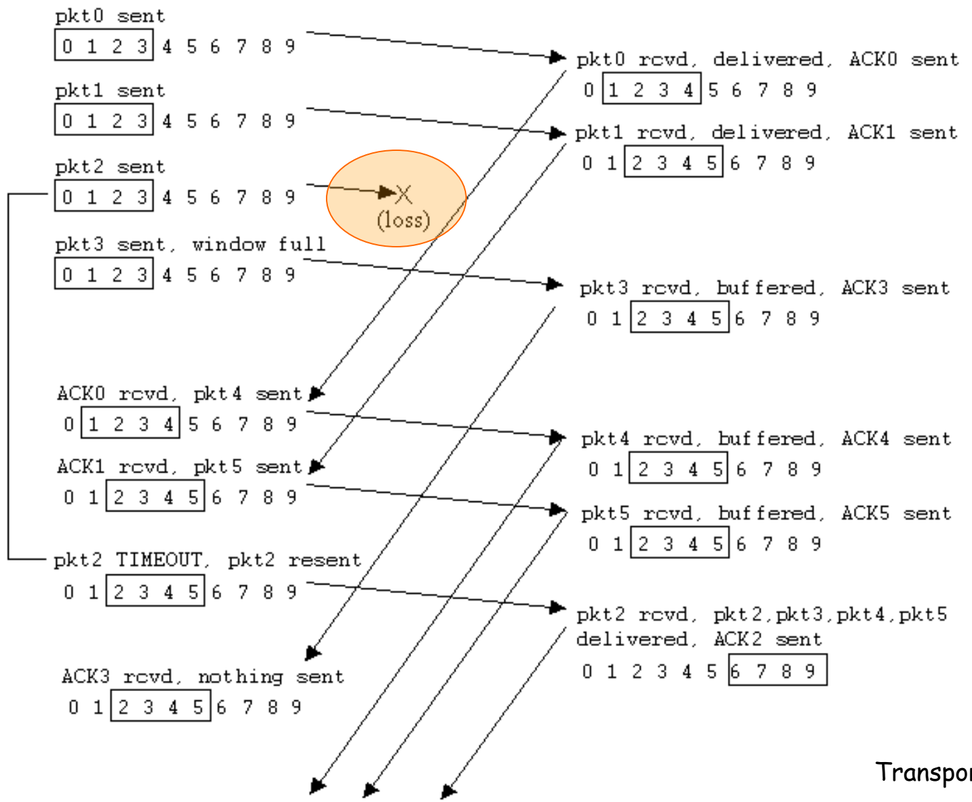
- Selective Repeat protocol allows receiver to buffer data and only forces retransmission of required packets

- GBN 易于编码但可能存在性能问题

- 特别是，如果同一时间有很多数据包在管道中（带宽时延积很大），那么一个错误就可能导致重传大量数据！

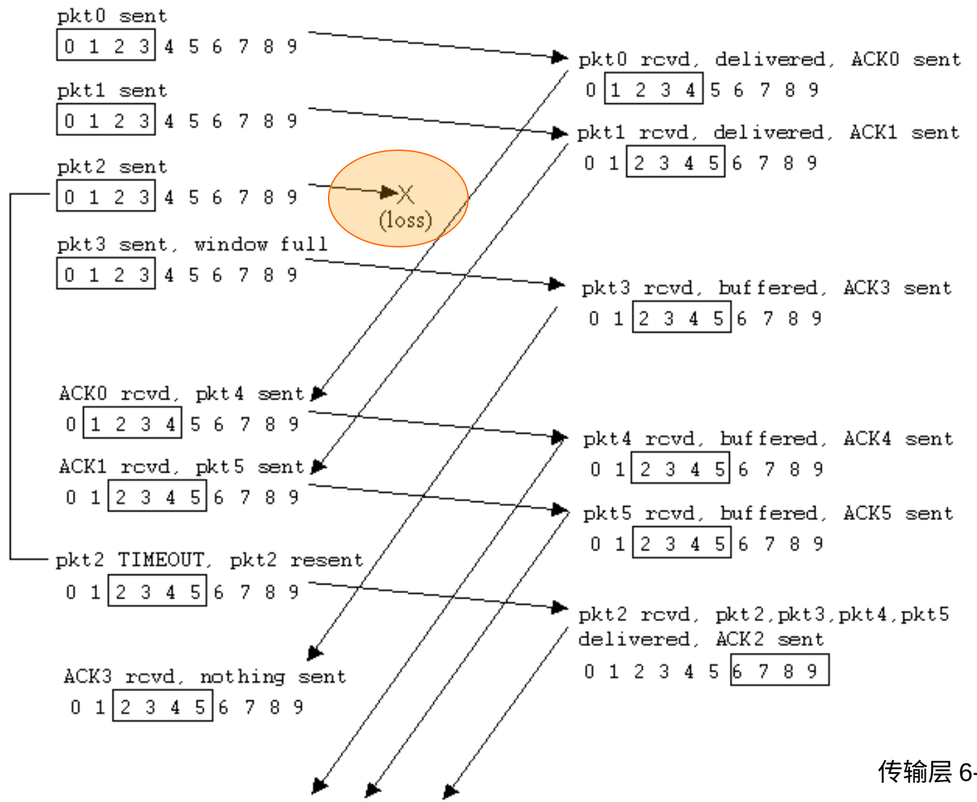
- 选择重发协议允许接收者缓存数据，并且只强制重传所需的数据包

# Selective Repeat In Action






# 选择重发 在行动中



# Selective Repeat

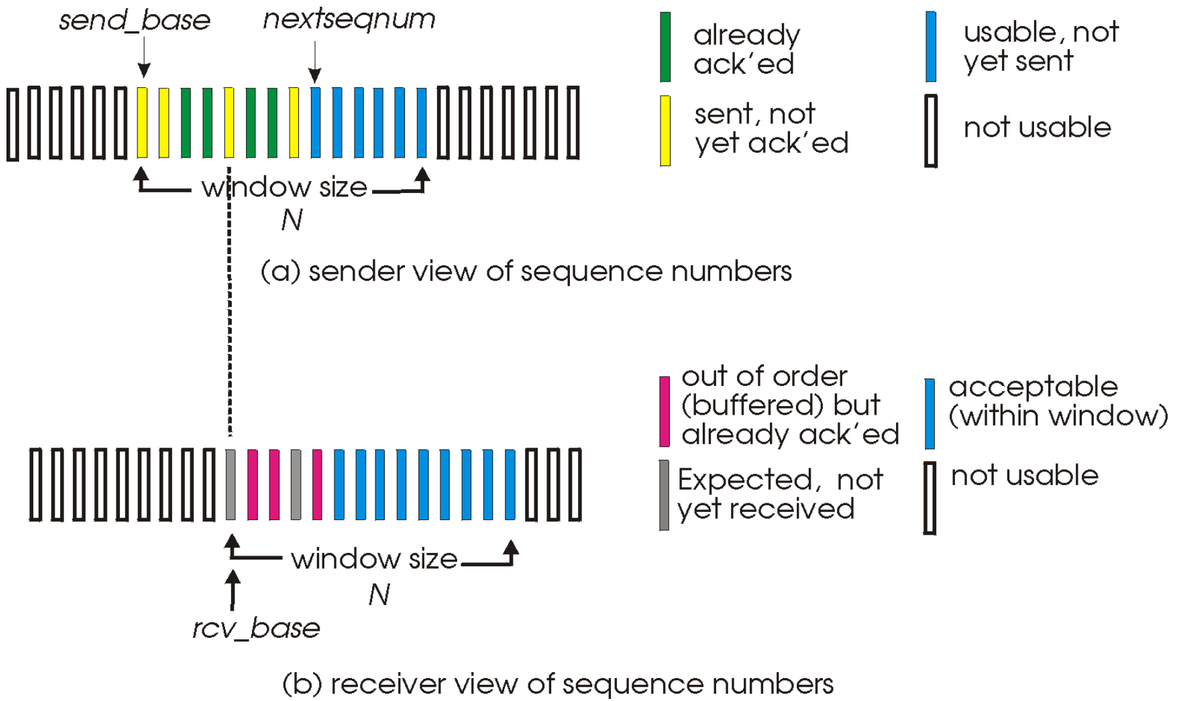
- Receiver *individually* acknowledges all correctly received pkts
  - Buffers pkts, as needed, for eventual in-order delivery to upper layer
- Sender only resends pkts for which ACK not received
  - Sender timer for each unACKed pkt
  - Compare to GBN which only had timer for base packet
- Sender window
  - N consecutive seq #'s
  - Again limits seq #s of sent, unACKed pkts
  - Important: Window size < seq # range



# 选择重发

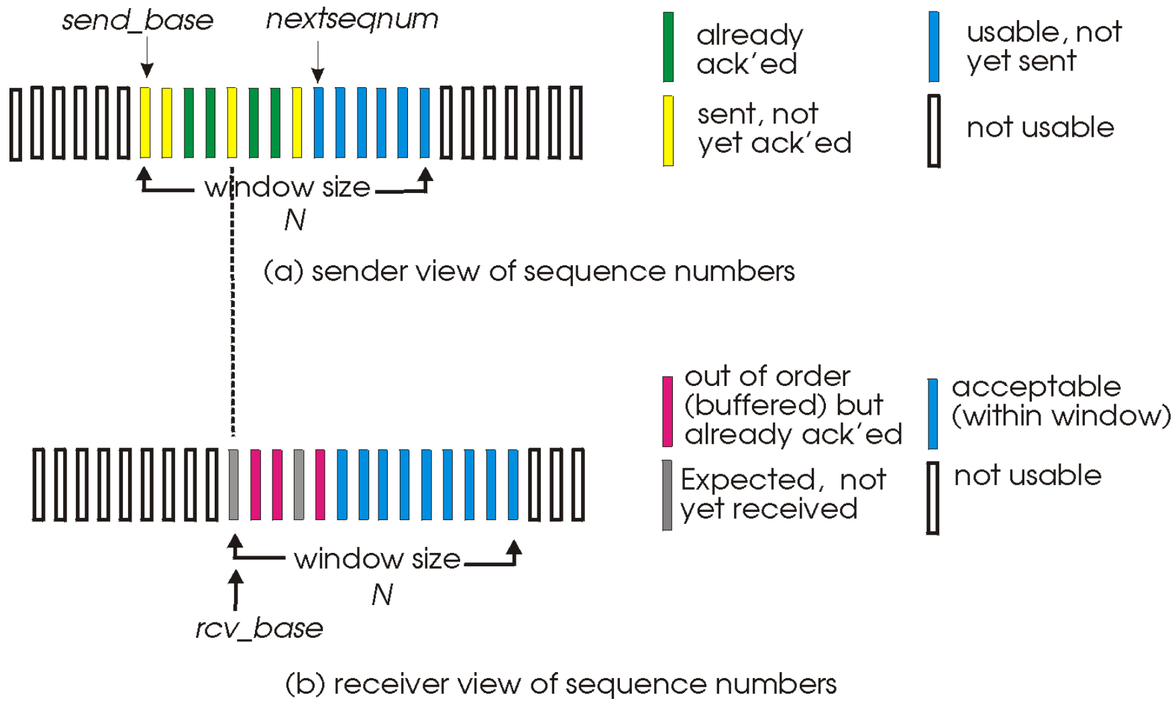
- 接收者 单独确认所有正确接收的数据包
  - 根据需要缓存数据包，以便最终按顺序向上层交付
- 发送方仅重发未收到确认的数据包
  - 发送方为每个未确认的数据包设置计时器❍ 与GBN相比，GBN仅对基本数据包设置计时器
- 发送窗口
  - N个连续的序列号❍ 再次限制已发送但未确认的数据包的序列号❍ 重要： 窗口大小< 序列号范围

# Selective Repeat: Sender, Receiver Windows






# S可选 重复：发送方、接收者窗口



# Selective Repeat

**sender**

Data from above :

- If next available seq # in window, send pkt

Timeout(n):

- Resend pkt n, restart timer

ACK(n) in [sendbase,sendbase+N]:

- Mark pkt n as received
- If n smallest unACKed pkt, advance window base to next unACKed seq #

**receiver**

pkt n in [rcvbase, rcvbase+N-1]

- Send ACK(n)
- Out-of-order: buffer
- In-order: deliver (also deliver buffered, in-order pkts), advance window to next not-yet-received pkt

pkt n in [rcvbase-N,rcvbase-1]

- ACK(n)

Otherwise:

- Ignore



# 选择重发

**发送方**

来自上方的数据：

- 如果窗口中的下一个可用序列号，发送数据包

超时(n)：

- 重发数据包n，重启计时器

确认(n) 在 [发送窗口基址，发送窗口基址+N]:

- 将数据包n标记为已接收
- 如果n是最小的未确认数据包，则将窗口基址前进到下一个未确认序列号

**接收者**

数据包n在 [接收窗口基址，接收窗口基址+N-1]

- 发送确认(n)
- 乱序：缓存
- 有序：投递（也投递缓存的有序数据包），将窗口前进到下一个未接收的数据包

数据包n在[接收窗口基址-N，接收窗口基址-1]

- 确认(n)

否则：

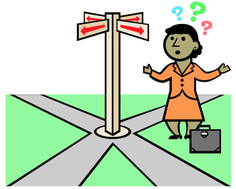
- 忽略

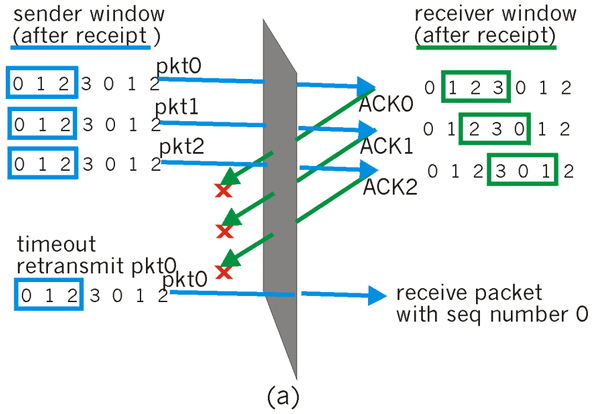
# Selective Repeat: Dilemma

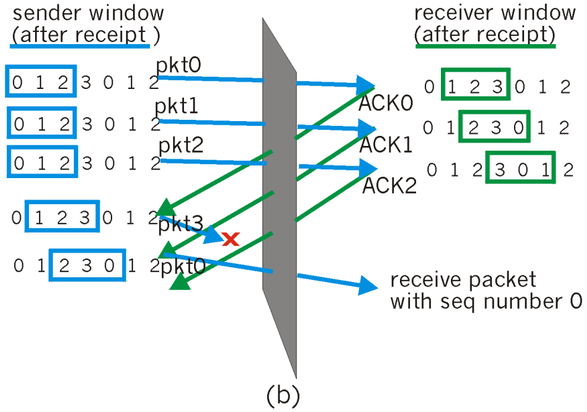
Example:

- seq #'s: 0, 1, 2, 3
- window size=3
- Receiver sees no difference in two scenarios!
- Incorrectly passes duplicate data as new in (a)

Q: what relationship between seq # size and window size?





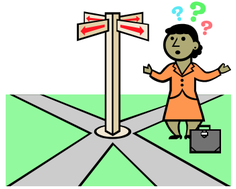


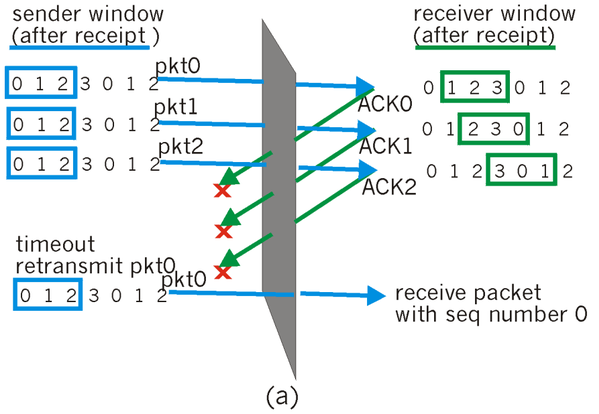

# 选择重发：困境

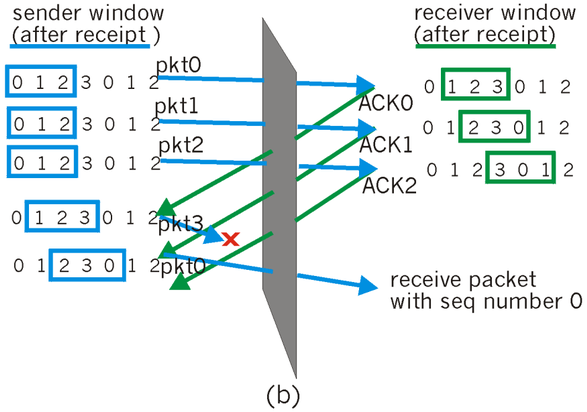
示例：

- 序列号：0，1，2，3
- 窗口大小=3
- 接收者在两种场景下没有区别！
- (a)中错误地将重复数据作为新数据传递

Q: 序列号大小和窗口大小之间有什么关系？

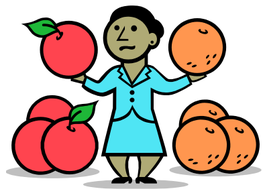




# GBN vs. Selective Repeat

- Selective repeat is more complicated as it needs buffering at the receiver, but only retransmit packets required for retransmission

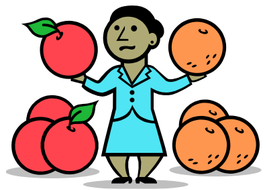


- GBN is simpler, but can lead to large number of unnecessary retransmission



# GBN 与 选择重发

- 选择重发更复杂，因为它需要在接收者处进行缓冲，但只重传需要重传的数据包



- GBN 更简单，但可能导致大量不必要的重传

# TCP Reliable Data Transfer

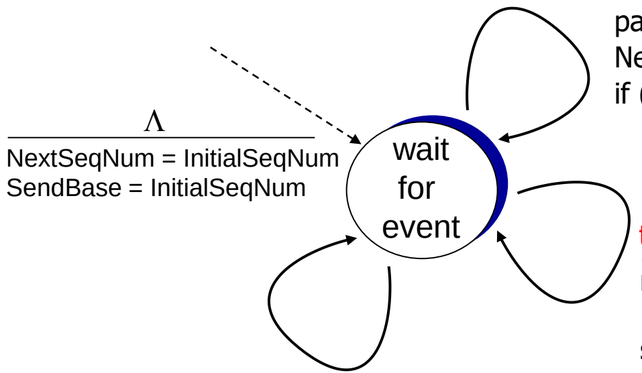
- TCP creates rdt service on top of IP's unreliable service
- Pipelined segments
- Cumulative acks
- TCP uses single retransmission timer
- Retransmissions are triggered by:
  - Timeout events
  - Duplicate acks
- Initially consider simplified TCP sender:
  - Ignore duplicate acks
  - Ignore flow control, congestion control



# TCP可靠数据传输

- TCP 在 IP 的不可靠服务之上创建了 rdt 服务
- 流水线分段
- 累积确认 TCP使用单个重传计时器
- 重传由以下触发：
  - 超时事件 重复确认
- 最初考虑简化的TCP发送方：
  - 忽略重复确认 忽略流量控制、拥塞控制

# TCP sender (simplified)




data received from application above

create segment, seq. #: NextSeqNum
pass segment to IP (i.e., "send")
NextSeqNum = NextSeqNum + length(data)
if (timer currently not running)
start timer

timeout

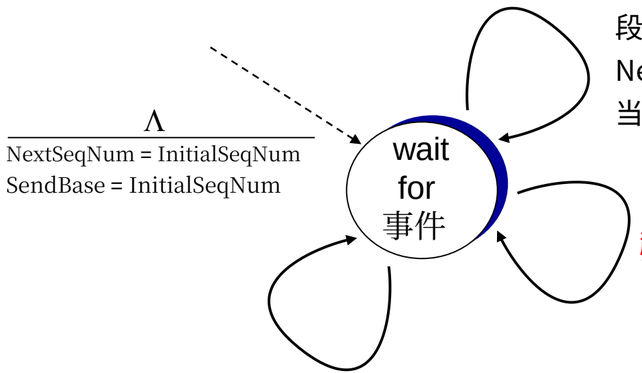
retransmit not-yet-acked segment
with smallest seq. #
start timer

ACK received, with ACK field value y

```
if (y > SendBase) {
   SendBase = y   %cumulative acks
   /* SendBase–1: last cumulatively ACKed byte */
   if (there are currently not-yet-acked segments)
       start timer
   else stop timer
   }
```



# TCP发送方 (简化)




从上层应用接收到的数据

创建分段，序列号 #：NextSeqNum 传递分段给IP（即，“发送”） NextSeqNum = NextSeqNum + length(data) 如果（计时器当前未运行）启动计时器

超时

重传序列号最小的未确认分段，启动计时器

确认收到，确认字段值为 y

if (y > SendBase) { SendBase = y %累积确认 /* SendBase–1: 最后一个累积确认的字节 */ if (当前没有尚未确认的分段) 启动计时器 else 停止计时器
}

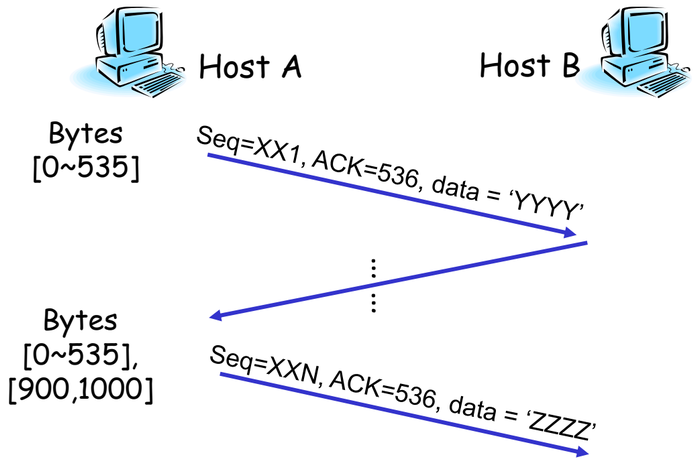
# TCP Seq. #'s and ACKs

**Seq. #'s:**

- Byte stream "number" of first byte in segment's data

**ACKs:**

- Seq # of next byte expected from other side
- Cumulative ACK: TCP only acknowledge bytes up to the first missing byte in the stream




Q: how receiver handles out-of-order segments?

A: TCP spec doesn't say, - up to implementer



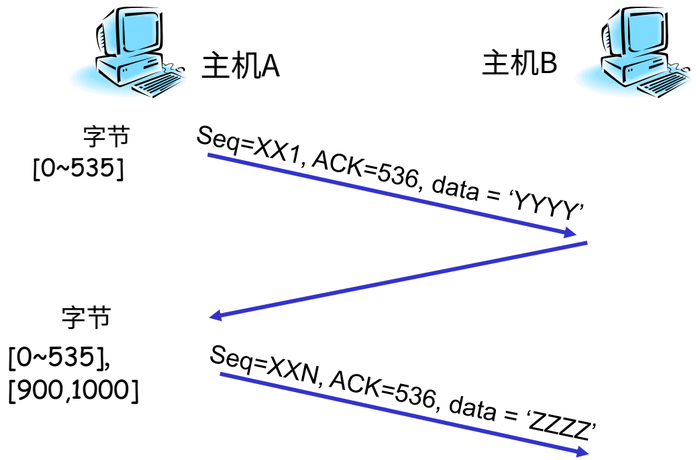
# TCP序列号和确认

**序列号：**

- 字节流"number"为分段数据中的第一个字节

**确认：**

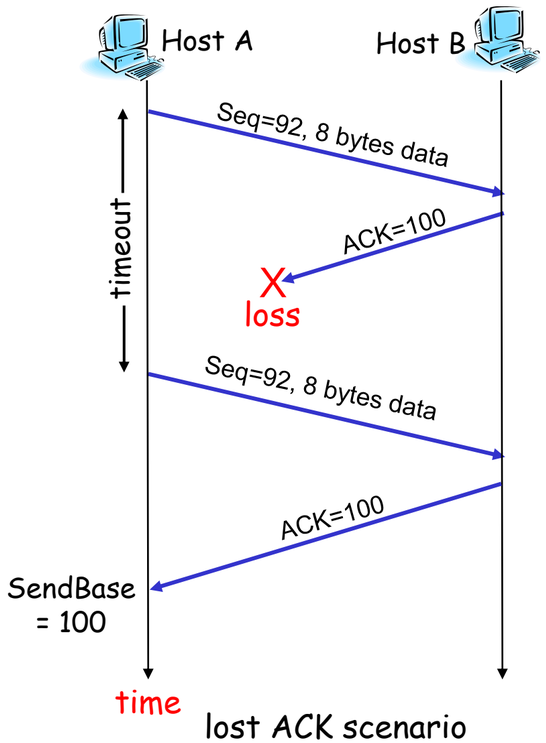
- 期望从对方接收的下字节序列号
- 累积确认：TCP 仅确认流中第一个缺失字节之前的字节



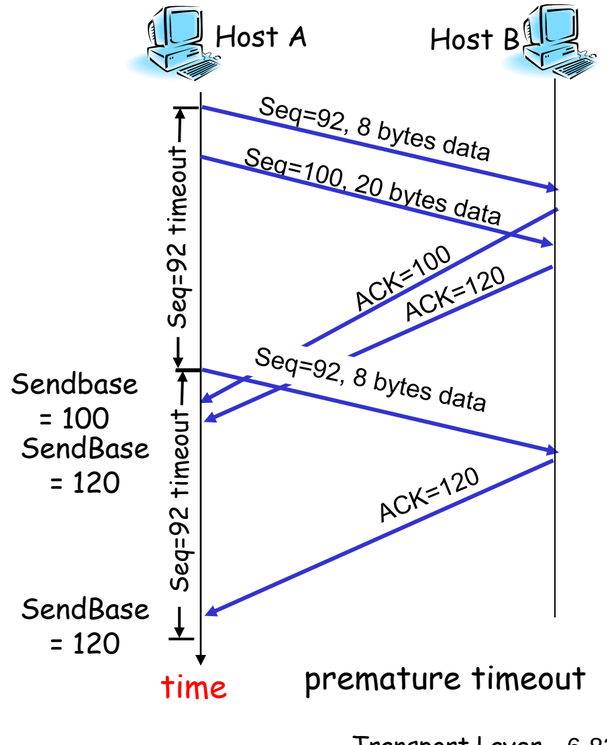

Q: 接收者如何处理乱序分段?

A:TCP规范没有说明，- 由实现者决定

# TCP: Retransmission Scenarios



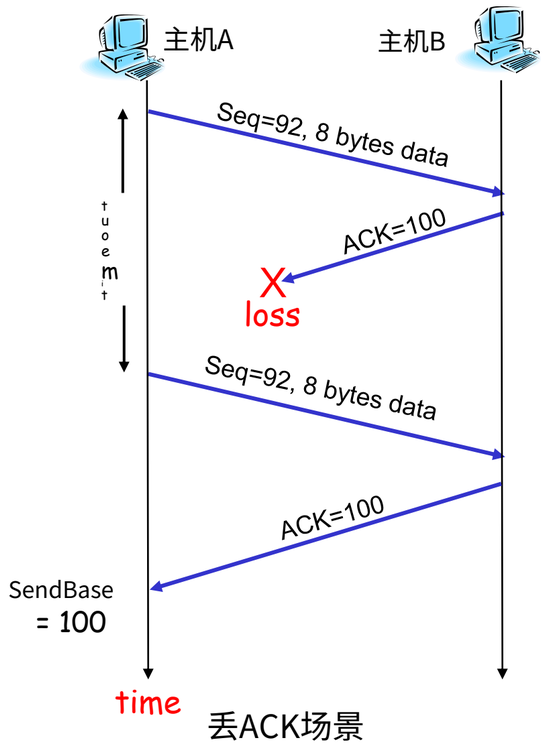

lost ACK scenario



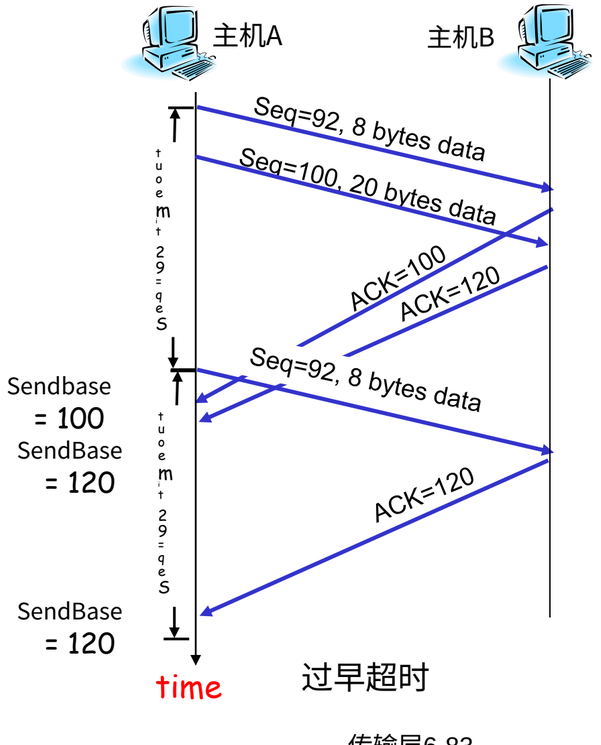

premature timeout



# TCP: 重传场景

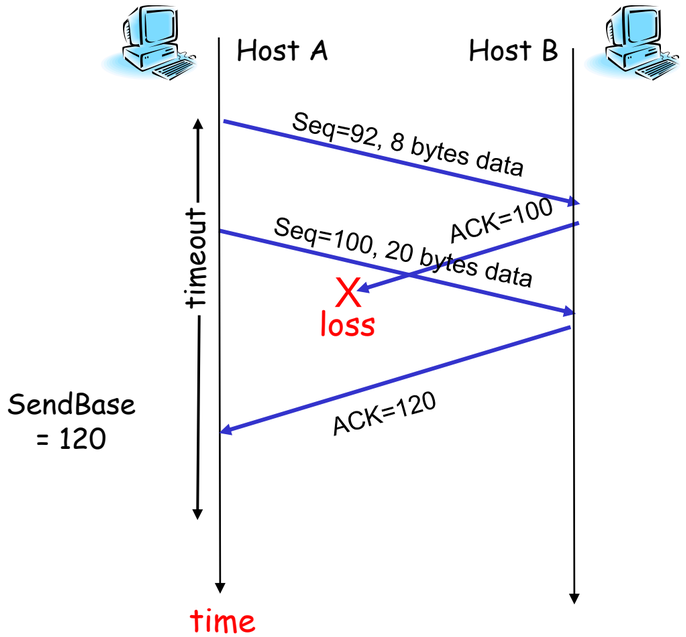



丢ACK场景



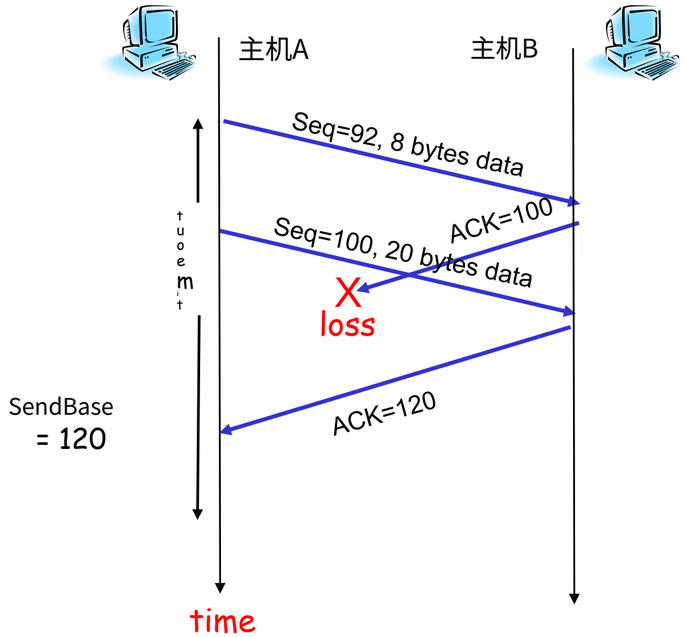

过早超时

# TCP Retransmission Scenarios (more)




Cumulative ACK scenario

# TCP重传场景 (更多)




累积确认场景

# TCP Round Trip Time and Timeout

Q: how to set TCP timeout value?

- Longer than RTT
  - But RTT varies
- Too short: premature timeout
  - Unnecessary retransmissions
- Too long: slow reaction to segment loss

Q: how to estimate RTT?

- **SampleRTT**: measured time from segment transmission until ACK receipt
  - Ignore retransmissions
- **SampleRTT** will vary, want estimated RTT "smoother"
  - Average several recent measurements, not just current **SampleRTT**





# TCP往返时间与超时

Q：如何设置TCP超时值?

- 超过往返时间，但往返时间会变化
- 太短：过早超时
  - 不必要的重传
- 太长：对分段丢失反应缓慢

Q: 如何估算往返时间?

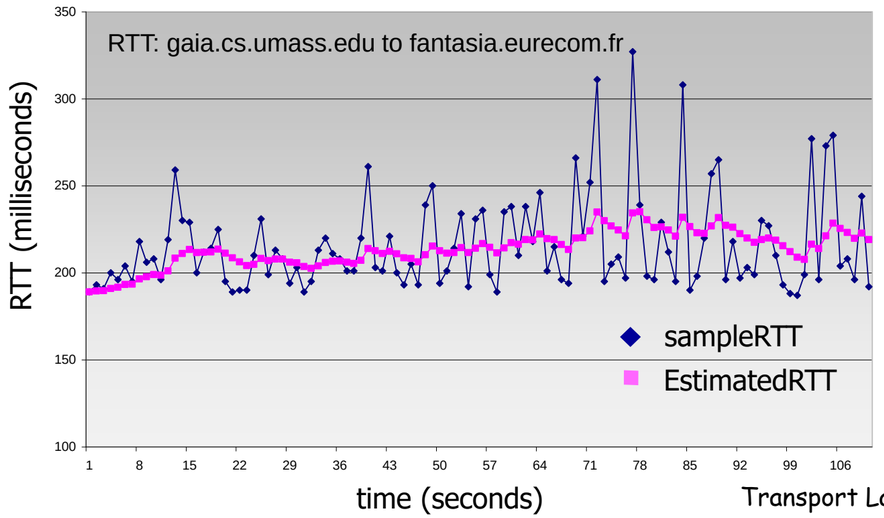
- **样本往返时间**：从分段传输到确认接收的测量时间，忽略重传
- **样本往返时间** 会有变化，想要更平滑的估计往返时间
  - 对多个近期测量值进行平均，而不仅仅是当前**样本往返时间**



# TCP round trip time, timeout

**EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT**

- Exponential Weighted Moving Average
- influence of past sample decreases exponentially fast
- typical value: α = 0.125

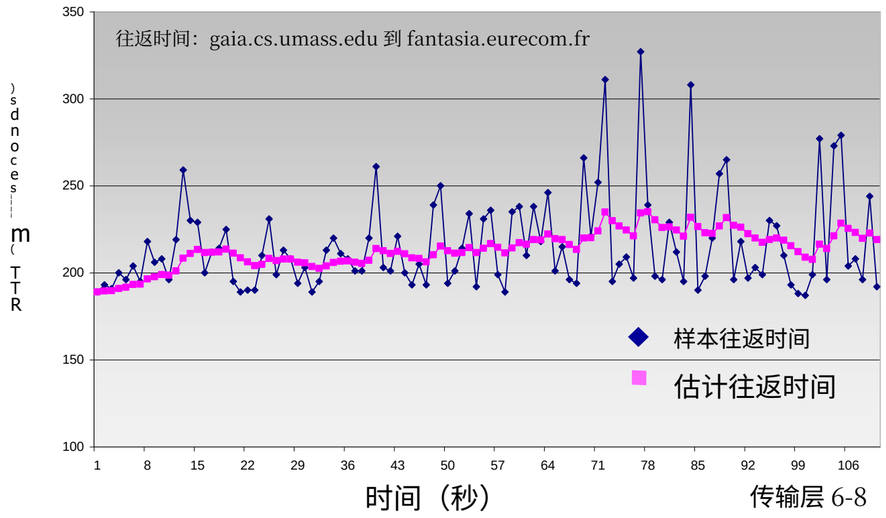





# TCP往返时间，超时

**估计往返时间 = (1- α)*估计往返时间 + α*样本往返时间**

- 指数加权移动平均
- 过去样本的影响呈指数级快速衰减
- 典型值：α = 0.125



# TCP round trip time, timeout

- timeout interval: **EstimatedRTT** plus "safety margin"
  - large variation in **EstimatedRTT** -> larger safety margin
- estimate SampleRTT deviation from EstimatedRTT:

DevRTT = (1-β)*DevRTT +
β*|SampleRTT-EstimatedRTT|

**(typically, β = 0.25)**

**TimeoutInterval = EstimatedRTT + 4*DevRTT**

estimated RTT

"safety margin"





# TCP往返时间，超时

- 超时间隔：**估计往返时间**加上"安全边际"
  - **估计往返时间** - >变化较大，需要更大的安全边际
- 估计样本往返时间偏差：

DevRTT = (1-β)*DevRTT +β*|样本往返时间-
估计往返时间|

**(通常，β = 0.25)**

**超时间隔= 估计往返时间+ 4*往返时间偏差**估计往返时间"安全裕度"



# More on Sender Policies



- Doubling the Timeout Interval
  - Used by most TCP implementations
  - If timeout occurs then, after retransmisison, Timeout Interval is doubled
  - Intervals grow exponentially with each consecutive timeout
- When Timer restarted because of (i) new data from above or (ii) ACK received, then Timeout Interval is reset as described previously using Estimated RTT and DevRTT.
- Limited form of Congestion Control



# 更多关于发送方策略



- 将超时间隔加倍
  - 大多数TCP实现使用
  - 如果 超时发生 那么，在重传后，超时间隔 会加倍
  - 每个连续超时后，间隔呈指数增长
- 当计时器因为 (i) 来自上层的新数据 或 (ii) 收到确认后重启时，超时 间隔 会像之前描述的那样使用估计往返时间和往返时间偏差重置
- 有限形式的拥塞控制

# TCP fast retransmit

- time-out period often relatively long:
  - long delay before resending lost packet
- detect lost segments via duplicate ACKs.
  - sender often sends many segments back-to-back
  - if segment is lost, there will likely be many duplicate ACKs.

*TCP fast retransmit*

if sender receives "triple duplicate ACKs" for same data, resend unacked segment with smallest seq #

- likely that unacked segment lost, so don't wait for timeout



# TCP快速重传

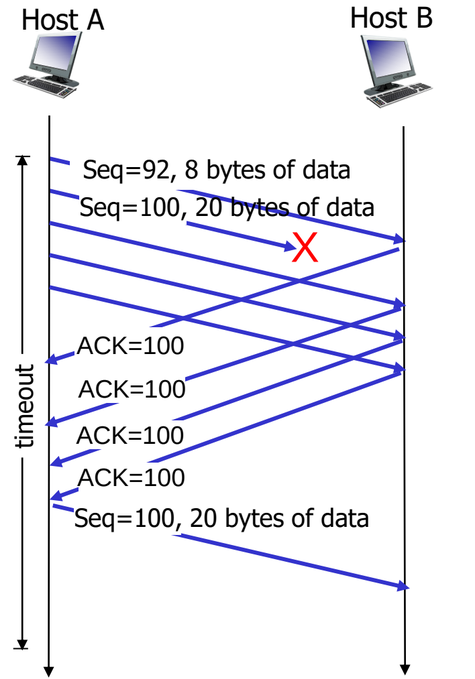
- 超时时间通常相对较长：
  - 重传前存在长延迟丢包
- 通过重复确认检测丢失的分段。
  - 发送方通常会连续发送多个分段
  - 如果分段丢失，很可能会出现许多重复确认。

TCP快速重传

如果发送方收到针对相同数据的"三重重复确认"，则使用最小的序列号重传未确认分段。

- 很可能未确认的分段丢失，因此不必等待超时

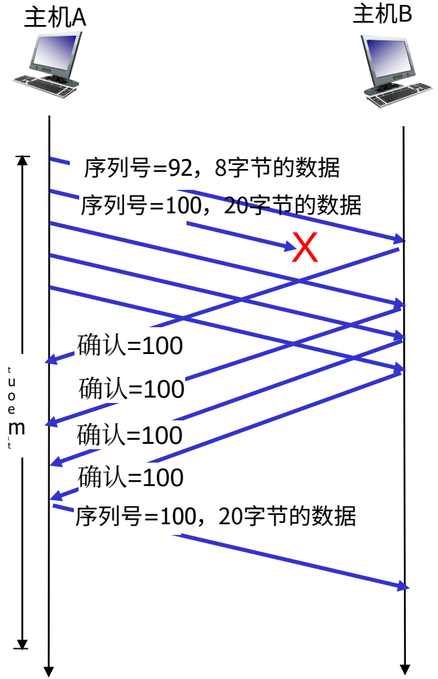
# TCP fast retransmit




fast retransmit after sender receipt of triple duplicate ACK



# TCP快速重传




发送方快速重传
三重重复ACK的接收

# TCP ACK Generation [RFC 1122, RFC 2581]

| Event at Receiver | TCP Receiver action |
|---|---|
| Arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed | Delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK |
| Arrival of in-order segment with expected seq #. One other segment has ACK pending | Immediately send single cumulative ACK, ACKing both in-order segments |
| Arrival of out-of-order segment higher-than-expect seq. # . Gap detected | Immediately send *duplicate ACK*, indicating seq. # of next expected byte |
| Arrival of segment that partially or completely fills gap | Immediate send ACK, provided that segment starts at lower end of gap |

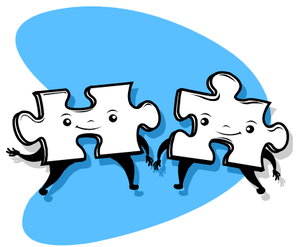


# TCP 确认生成[RFC 1122, RFC 2581]

| 接收者处事件 | TCP接收器动作 |
|---|---|
| 按序到达分段，包含<br>预期序列号。所有数据至<br>预期序列号部分已确认 | 延迟确认。等待最长500毫秒<br>以等待下一个分段。如果没有下一个分段，<br>发送确认 |
| 按序到达分段，包含<br>预期序列号。另有一个<br>分段正在等待确认 | 立即发送单个累积确认<br>确认所有有序分段 |
| 乱序分段到达<br>高于预期序列号。<br>检测到间隔 | 立即发送 重复确认，<br>指示下一个预期字节的序列号 |
| 分段到达，该<br>部分或完全填补了间隙 | 立即发送确认，前提是<br>分段从间隙的较低端开始 |

# TCP: GBN or Selective Repeat?

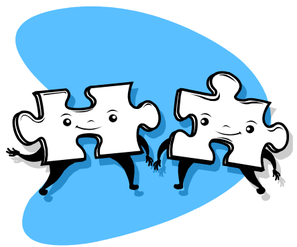
- Basic TCP looks a lot like GBN
- Many TCP implementations will buffer received out-of-order segments and then ACK them all after filling in the range
  - This looks a lot like Selective Repeat
- TCP is a hybrid





# TCP：GBN或选择重发?

- 基本的TCP很像GBN
- 许多TCP实现会缓存接收到的乱序分段，并在填满序号范围后确认所有分段
  - 这很像选择重发
- TCP是一种混合



# Exercises-3

1. A TCP connection has been established between host A and host B. host A sends three consecutive TCP segments to host B, including 300 bytes, 400 bytes and 500 bytes payload respectively. The sequence number of the third segment is 900. If host B receives only the first and third segments correctly, the confirmation sequence number sent by host B to host A is( )

(1)300
(2)500
(3)1200
(4)1400



# 练习-3

1. 主机A和主机B之间已建立TCP连接。主机A向主机B发送了三个连续的TCP数据段，分别为300字节、400字节和500字节的负载。第三个数据段的序列号是900。如果主机B只正确接收了第一个和第三个数据段，主机B向主机A发送的确认序列号是( )

(1)300(2)500(3)1200(4)1400

# Chapter 6 Outline

- 6.1 Transport-layer services
- 6.2 Multiplexing and demultiplexing
- 6.3 Connectionless transport: UDP
- 6.4 Connection-oriented transport: TCP
  - Segment structure
  - Connection management
  - Reliable data transfer
- 6.5 TCP congestion control

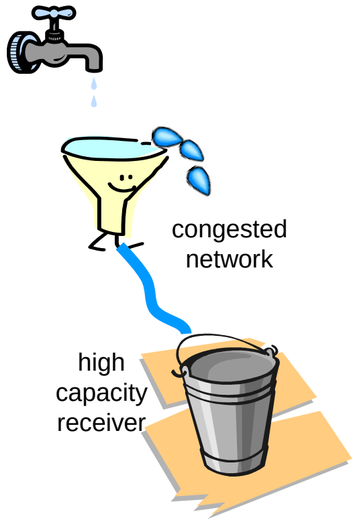
# 第6章概要

- 6.1 传输层服务
- 6.2 多路复用和分路复用
- 6.3 无连接传输：UDP
- 6.4 面向连接的传输：TCP
  - 分段结构
  - 连接管理
  - 可靠数据传输
- 6.5 TCP拥塞控制

# Principles of Congestion Control

## Congestion:

- Informally: "too many sources sending too much data too fast for *network* to handle"
- Different from flow control!
- Manifestations:
  - Lost packets (buffer overflow at routers)
  - Long delays (queueing in router buffers)
- A top-10 problem!




congestion control problem



# 拥塞控制原理

## 拥塞：

- 非正式： "太多源发送 太多数据太快 网络无法 处理"
- 与流量控制不同！
- 表现形式：
  - 丢失数据包（路由器缓冲区溢出）
  - 长时间延迟（路由器缓冲区队列）
- 十大问题之一！




拥塞控制问题

# Approaches towards Congestion Control

Two broad approaches towards congestion control:

## End-end congestion control:

- No explicit feedback from network
- Congestion inferred from end-system observed loss, delay
- Approach taken by TCP

## Network-assisted congestion control:

- Routers provide feedback to end systems
  - Single bit indicating congestion (SNA, DECbit, TCP/IP ECN, ATM)
  - Explicit rate sender should send at





# 拥塞控制方法

拥塞控制的两条主要方法：

## 端到端拥塞控制：

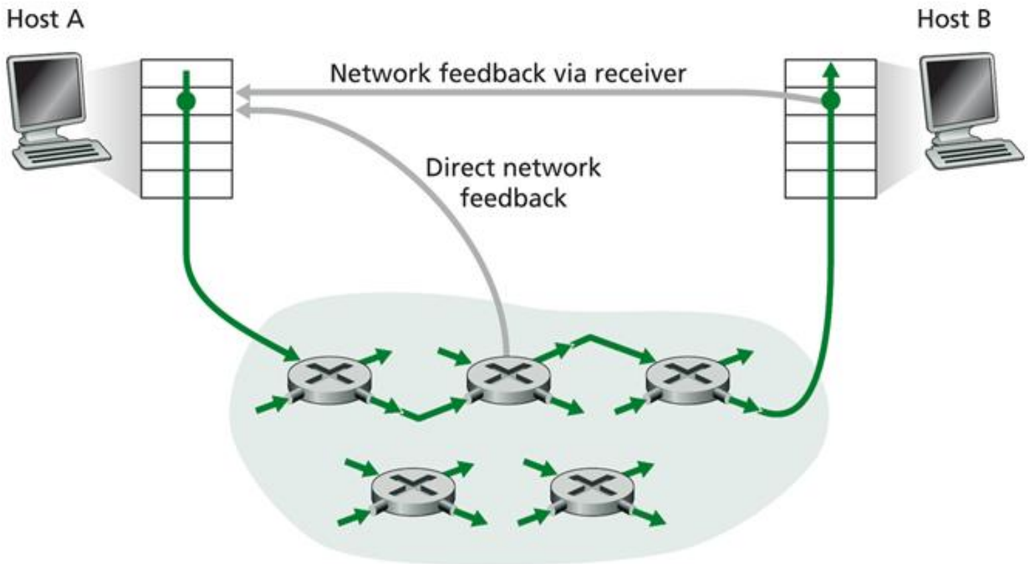
- 网络无显式反馈
- 通过终端系统观察到的丢包、延迟推断拥塞
- TCP采用的策略

## 网络辅助的拥塞控制：

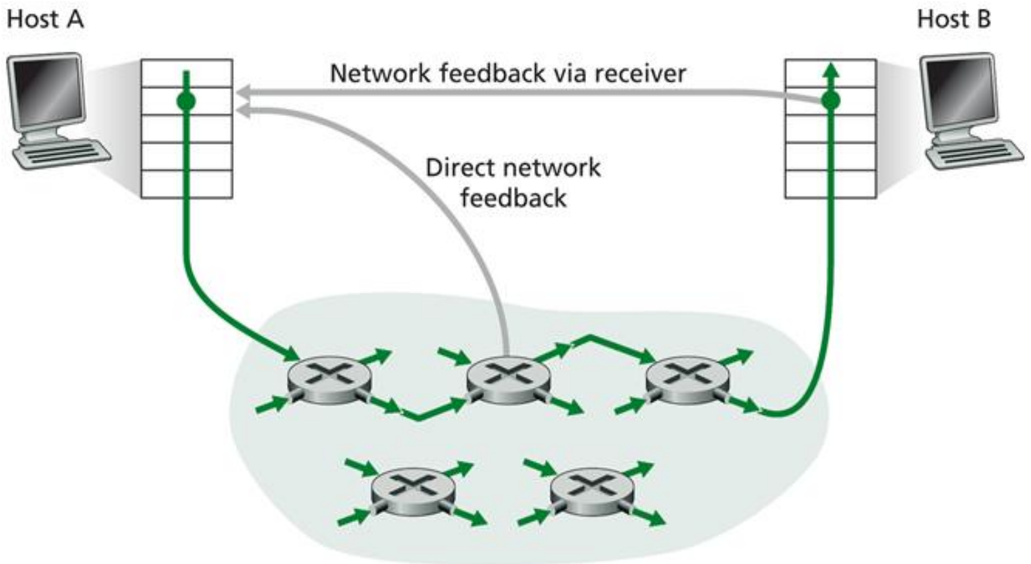
- 路由器向终端系统提供反馈
  - 单比特指示拥塞（SNA、DECbit、TCP/IP ECN、ATM）
  - 显式速率发送方应发送在



# Two Ways for Providing Feedback




# 提供反馈的两种方式



# TCP Congestion Control

- End-to-end control (no network assistance)
- Transmission rate limited by congestion window size, **Congwin**, over segments
  - Congwin dynamically modified to reflect perceived congestion
- Tools are "similar" to flow control
  sender limits transmission using:
  **LastByteSent-LastByteAcked ≤ CongWin**

## How does sender perceive congestion?

- loss event = timeout *or* 3 duplicate ACKs
- TCP sender reduces rate (**CongWin**) after loss event

# TCP拥塞控制

- 端到端控制（无需网络辅助）
- 传输速率受拥塞窗口大小限制，**Congwin**，通过分段
  - Congwin 动态修改以反映感知拥塞
- 工具与流量控制类似，发送方通过
  以下方式限制传输：
  **LastByteSent-LastByteAcked ≤ Congwin**

## 发送方如何感知拥塞?

- 丢包事件 = 超时 或 3个重复确认
- TCP发送方在丢包事件后降低速率（**CongWin**）

# TCP Congestion Control Algorithms

## Three mechanisms:

- AIMD = *Additive Increase Multiplicative Decrease*
- Slow start = `CongWin` set to 1 and then grows exponentially
- Conservative after timeout events



# TCP拥塞控制算法

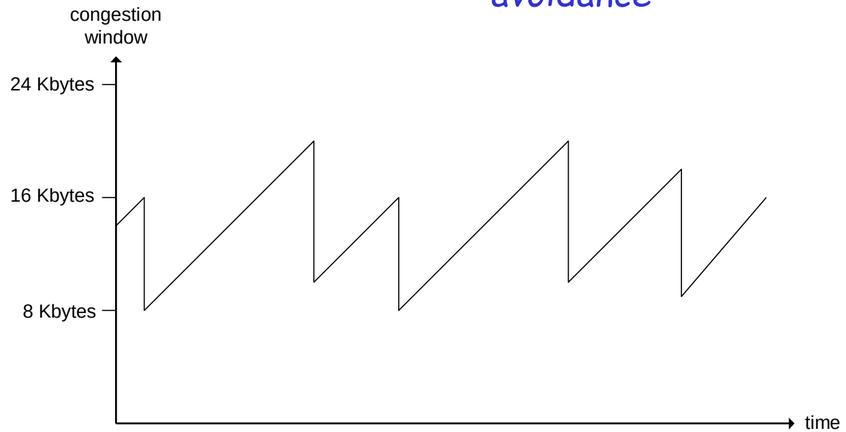
## 三种机制：

- AIMD = 加性增加乘性减少
- 慢启动= `CongWin` 设置为1然后呈指数增长
- 超时事件后保守处理

# TCP AIMD

Multiplicative decrease: cut **CongWin** in half after loss event

Additive increase: increase **CongWin** by 1 MSS every RTT in the absence of loss events: *probing* also known as *congestion avoidance*



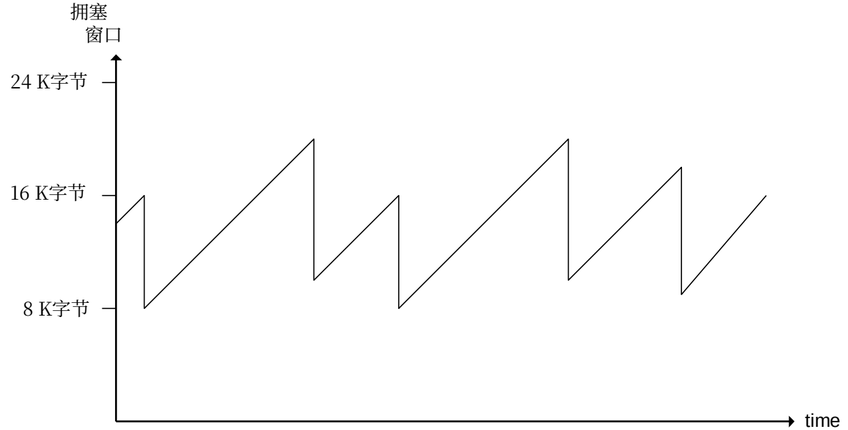

Long-lived TCP connection



# TCP AIMD

乘性减少：丢包事件后 cut **CongWin** 减半

加性增加：增加**CongWin** 在无丢包事件的情况下，每RTT增加1 MSS：探测也称为拥塞避免




长连接TCP连接

# TCP Slow Start

- When connection begins, **CongWin** = 1 MSS
  - Example: MSS = 500 bytes & RTT = 200 msec
  - Initial rate = 20 kbps
- Available bandwidth may be >> MSS/RTT
  - Desirable to quickly ramp up to respectable rate

- When connection begins, increase rate exponentially fast until first loss event

**Slow Start is NOT really Slow!**

# TCP慢启动

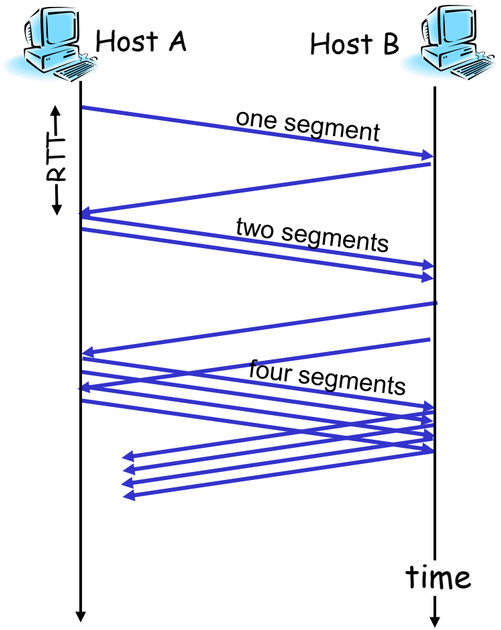
- 连接开始时，**CongWin** = 1 MSS
  - 示例：MSS = 500 字节 & 往返时间 = 200 毫秒 初始速率 = 20 千比特每秒
- 可用带宽可能是 >> MSS/往返时间
  - 希望快速提升到可接受的速度

- 连接开始时，以指数级速度增加速度，直到第一个丢包事件

**慢启动并非真的慢！**

# TCP Slow Start (more)

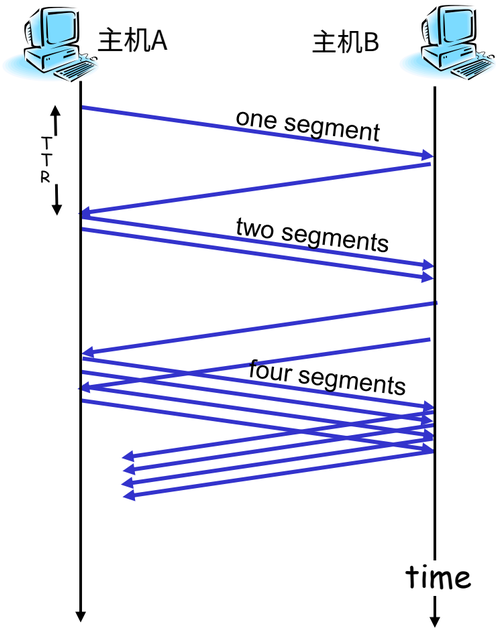
- When connection begins, increase rate exponentially until first loss event:
  - Double **CongWin** every RTT
  - Done by incrementing **CongWin** for every ACK received
- Summary: initial rate is slow but ramps up exponentially fast






# TCP慢启动 (更多)

- 连接开始时，指数级增加速率，直到第一个丢包事件：
  - 将<code>CongWin</code>翻倍，每收到一个<code>ACK</code>就递增<code>CongWin</code>
  - 完成
- 摘要：初始速率较慢但指数级快速提升



## So Far

- Slow-Start: ramps up exponentially
- Followed by AIMD: sawtooth pattern
- Q: When should the exponential increase switch to linear?

## Reality (TCP Reno)

- Introduce new variable **threshold:** initially very large
- Slow-Start exponential growth stops when reaches **threshold** and then switches to AIMD
- Two different types of loss events
  - 3 dup ACKS: cut **CongWin** in half and set **threshold=CongWin** (now in standard AIMD)
  - Timeout: set **threshold=CongWin/2**, **CongWin=1** and switch to Slow-Start



## 到目前为止

- 慢启动：指数级快速提升
- 随后是AIMD：锯齿形模式
- Q：何时应从指数增长切换到线性?

## 现实（TCP Reno）

- 引入新变量**阈值：**初始时非常大
- 慢启动指数增长在达到**阈值**时停止，然后切换到AIMD
- 两种不同的丢包事件
  - 3 重复ACK：将**CongWin**减半并设置**阈值=CongWin**（现在采用标准AIMD）
  - 超时：设置**阈值=CongWin/2**，**CongWin=1**并切换到慢启动

- Reason for treating 3 dup ACKS differently than timeout is that:
  - 3 dup ACKs indicates network capable of delivering some segments while
  - Timeout before 3 dup ACKs is "more alarming"

- Note that older protocol, TCP Tahoe, treated both types of loss events the same and **always** goes to slow-start with Congwin=1 after a loss event

- TCP Reno's skipping of the slow start for a 3-DUP-ACK loss event is known as fast-recovery



- 将3次重复ACK与超时区别对待的原因是：
  - 3 重复ACK表示网络能够传输一些 e 分段，而
  - 在3次重复ACK之前超时则“更令人担忧”

- 请注意，较旧的协议TCP Tahoe将两种类型的丢包事件视为相同，并且在发生丢包事件后**总是**进入慢启动状态并使用Congwin=1

- TCP Reno的对3-DUP-丢包事件跳过慢启动是称为快速恢复

# Summary: TCP Congestion Control

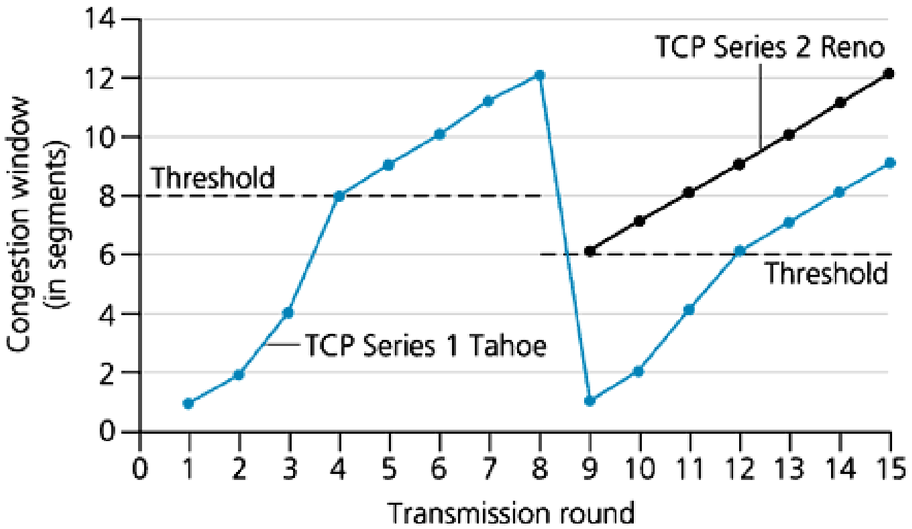
- When **CongWin** is below **Threshold**, sender in slow-start phase, window grows exponentially
- When **CongWin** is above **Threshold**, sender is in congestion-avoidance phase, window grows linearly
- When a triple duplicate ACK occurs, **Threshold** set to **CongWin/2** and **CongWin** set to **Threshold** *(only in TCP Reno)*
- When timeout occurs, **Threshold** set to **CongWin/2** and **CongWin** is set to 1 MSS *(TCP Tahoe does this for 3 Dup Acks as well)*



# 总结：TCP拥塞控制

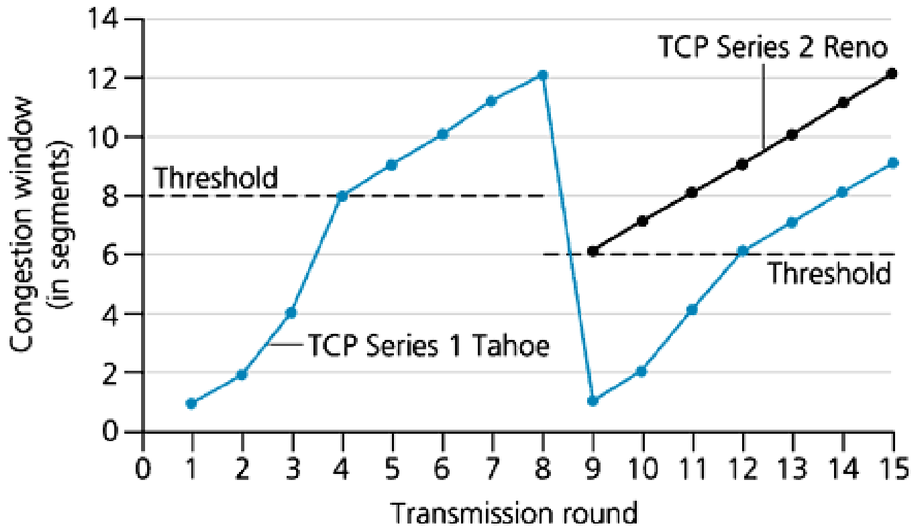
- 当**CongWin**低于**阈值**时，发送方在慢启动阶段，窗口指数增长
- 当 **Congwin**高于 **阈值**时，发送方处于拥塞避免 阶段，窗口线性增长
- 当出现 三重重复ACK时， **阈值** 设置为 **Congwin/2** ，且 **Congwin** 设置为 **阈值**(仅在TCP Reno中如此)
- 当出现 超时时， **阈值**设置为**Congwin/2**，且 **Congwin** 设置为1 MSS (TCP Tahoe对3次重复ACK也这样做)

# The Big Picture






# 整体概览



# TCP Sender Congestion Control

| Event | State | TCP Sender Action | Commentary |
|---|---|---|---|
| ACK receipt for previously unacked data | Slow Start (SS) | CongWin = CongWin + MSS, If (CongWin > Threshold) set state to "Congestion Avoidance" | Resulting in a doubling of CongWin every RTT |
| ACK receipt for previously unacked data | Congestion Avoidance (CA) | CongWin = CongWin+MSS * (MSS/CongWin) | Additive increase, resulting in increase of CongWin by 1 MSS every RTT |
| Loss event detected by triple duplicate ACK | SS or CA | Threshold = CongWin/2, CongWin = Threshold, Set state to "Congestion Avoidance" | Fast recovery, implementing multiplicative decrease. CongWin will not drop below 1 MSS |
| Timeout | SS or CA | Threshold = CongWin/2, CongWin = 1 MSS, Set state to "Slow Start" | Enter slow start |
| Duplicate ACK | SS or CA | Increment duplicate ACK count for segment being acked | CongWin and Threshold not changed |



# TCP发送方拥塞控制

| 事件 | 状态 | TCP发送方动作 | 注释 |
|---|---|---|---|
| 确认接收<br>针对先前<br>未确认数据 | 慢启动<br>(SS) | CongWin = CongWin + MSS,<br>If (CongWin > 阈值)<br>将状态设置为 "拥塞避免<br>" | 导致 CongWin 每次往返时间翻倍<br>确认接收 |
| 对于之前<br>的<br>未确认数据 | 拥塞<br>避免<br>(CA) | Congwin = Congwin+MSS *<br>(MSS/CongWin) | 加性增加，导致<br>CongWin增加<br>每RTT增加1 MSS |
| 丢包事件<br>检测到<br>三重重复<br>ACK | 慢启动或拥塞<br>避免 | 阈值 = Congwin/2,<br>Congwin = 阈值,<br>设置状态为 "拥塞避免<br>快速恢复" | 快速恢复,<br>实现乘性<br>减少。CongWin 不会<br>低于 1 MSS |
| 超时 | 慢启动或拥塞<br>避免 | 阈值 = CongWin/2,<br>CongWin = 1 MSS,<br>将状态设置为 "慢启动" | 进入慢启动 |
| 重复ACK | SS或CA | 增加重复ACK计数<br>对于被确认的分段 | Congwin 和 阈值 不变<br>changed |

# Exercises-4

1. A TCP connection always sends TCP segments with 1KB MSS. The sender has enough data to send. A timeout event occurs when the congestion window is 16KB. If the transmission of TCP segments in the next four RTTS is successful, when all TCP segments sent in the fourth RTT receive positive responses, the size of the congestion window is( )

A. 7KB
B. 8KB
C. 9KB
D. 16KB



# 习题-4

1. 一个TCP连接总是以1KB MSS发送TCP数据段。发送方有足够的数据可以发送。当拥塞窗口为16KB时发生超时事件。如果接下来四个往返时间(RTT)内TCP数据段的传输都成功，当第四个RTT中所有发送的TCP数据段都收到积极响应时，拥塞窗口的大小是( )

A. 7KB
B. 8KB
C. 9KB
D. 16KB

# Chapter 6: Summary

- Principles behind transport layer services:
  - Multiplexing, demultiplexing
  - Reliable data transfer
  - Congestion control
- Instantiation and implementation in the Internet
  - UDP
  - TCP



# 第6章：总结

- 传输层服务的原理：
  - 复用，解复用
  - 可靠数据传输
  - 拥塞控制
- 互联网中的实例化和实现
  - UDP
  - TCP

# Homework

- P538:14,15
- P322:27
- P323:28
- P326:42,43

# 作业

- P538:14,15
- P322:27
- P323:28
- P326:42,43